

CANN
V100R020C20

TensorFlow Network Model Porting and Training Guide

Issue 01
Date 2021-02-09



HUAWEI TECHNOLOGIES CO., LTD.

Copyright © Huawei Technologies Co., Ltd. 2021. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions



HUAWEI and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Contents

1 About This Document.....	1
2 Restrictions and Limitations.....	2
3 Migration Workflow.....	3
4 Auto Migration and Manual Migration.....	4
4.1 Model Preparation and Evaluation.....	4
4.2 Auto Migration Using Tool.....	4
4.3 Manual Migration.....	6
4.3.1 tf.config.optimizer.set_experimental_options Options to Migrate.....	6
4.3.2 tf.case, tf.cond, and tf.while_loop Options to Migrate.....	7
4.3.3 tf.ConfigProto Options to Migrate.....	7
4.3.4 Keras Options to Migrate.....	8
4.3.5 Configuration Options for Distributed Training Migration.....	8
4.3.5.1 Typical Scenarios.....	8
4.3.5.2 Distributed Training Based on the AllReduce Architecture.....	13
4.3.5.3 Distributed Training Based on the PS-Worker Architecture.....	17
4.3.5.4 Distributed Training Based on the Horovod Framework.....	20
5 Model Training.....	23
5.1 Training Service Workflow.....	23
5.2 Training in a Bare Metal Environment.....	26
6 Performance Tuning.....	31
6.1 Network Throughput Analysis.....	31
6.2 Network-wide Performance Optimization.....	32
6.2.1 Iteration Offloading.....	32
6.2.2 Data Preprocessing Performance.....	37
6.2.3 Distributed Performance Improvement.....	38
6.2.4 Mixed Precision.....	41
6.2.5 Loss Scaling.....	43
6.3 Operator Performance Optimization.....	45
6.3.1 Automatic Operator Tuning.....	45
6.3.2 Operator Profiling.....	45
7 Accuracy Tuning.....	47

7.1 Operator Accuracy Analysis.....	47
7.2 Operator Overflow Detection.....	49
8 Model Conversion and Storage.....	55
9 More Functions.....	59
9.1 Mixed Computing.....	59
9.2 Log and Summary Operators.....	60
9.3 Collective Communication APIs.....	62
9.3.1 API Overview.....	62
9.3.2 Initializing Collective Communication.....	63
9.3.3 Specifying Devices to Perform Collective Communication.....	64
9.3.4 Obtaining the Group Information.....	64
9.3.5 Setting the Backward Gradient Segmentation Policy.....	64
9.3.6 Computing Tensor Nodes for Collective Communication.....	65
10 Reference.....	67
10.1 Migration Examples.....	67
10.1.1 ResNet-50 Model Training Using the ImageNet Dataset.....	67
10.1.1.1 Preparations.....	67
10.1.1.2 Training Workflow.....	68
10.1.1.3 Training Code Directory Structure.....	69
10.1.1.4 Data Preprocessing.....	70
10.1.1.5 Model Building.....	72
10.1.1.6 Run Configuration.....	75
10.1.1.7 Training.....	76
10.1.1.8 Script Execution.....	79
10.1.2 More Migration Cases.....	80
10.2 API Reference.....	80
10.2.1 Available TensorFlow APIs.....	80
10.2.2 TF Adapter APIs.....	109
10.2.2.1 Overview.....	109
10.2.2.2 npu_bridge.estimator.npu.npu_config.....	110
10.2.2.2.1 NPURunConfig Constructor.....	110
10.2.2.2.2 ProfilingConfig Constructor.....	125
10.2.2.2.3 DumpConfig Constructor.....	127
10.2.2.3 npu_bridge.estimator.npu.npu_estimator.....	129
10.2.2.3.1 NPUEstimator Constructor.....	129
10.2.2.3.2 NPUEstimatorSpec Constructor.....	130
10.2.2.4 npu_bridge.estimator.npu.npu_hook.....	132
10.2.2.4.1 NPUCheckpointSaverHook Constructor.....	132
10.2.2.4.2 NPUOutputTensorHook Constructor.....	133
10.2.2.5 npu_bridge.estimator.npu.npu_optimizer.....	135
10.2.2.5.1 NPUDistributedOptimizer Constructor.....	135

10.2.2.5.2 NPUOptimizer Constructor.....	136
10.2.2.6 npu_bridge.estimator.npu.npu_loss_scale_optimizer.....	138
10.2.2.6.1 NPULossScaleOptimizer Constructor.....	138
10.2.2.7 npu_bridge.estimator.npu.npu_loss_scale_manager.....	140
10.2.2.7.1 FixedLossScaleManager Constructor.....	140
10.2.2.7.2 ExponentialUpdateLossScaleManager Constructor.....	140
10.2.2.8 npu_bridge.estimator.npu_ops.....	141
10.2.2.8.1 dropout.....	141
10.2.2.8.2 LARSV2.....	142
10.2.2.8.3 initialize_system.....	143
10.2.2.8.4 shutdown_system.....	149
10.2.2.9 npu_bridge.estimator.npu.npu_rnn.....	149
10.2.2.9.1 npu_dynamic_rnn.....	150
10.2.2.10 npu_bridge.estimator.npu.npu_scope.....	151
10.2.2.10.1 without_npu_compile_scope.....	151
10.2.2.10.2 keep_dtype_scope.....	152
10.2.2.11 npu_bridge.estimator.npu.util.....	152
10.2.2.11.1 set_iteration_per_loop.....	152
10.2.2.11.2 create_iteration_per_loop_var.....	153
10.2.2.11.3 load_iteration_per_loop_var.....	154
10.2.2.12 npu_bridge.estimator.npu.keras_to_npu.....	155
10.2.2.12.1 model_to_npu_estimator.....	155
10.2.2.13 Session Configuration in sess.run Mode.....	156
10.2.3 Collective Communication APIs.....	173
10.2.3.1 Overview.....	173
10.2.3.2 hccl.manage.api.....	173
10.2.3.2.1 create_group.....	174
10.2.3.2.2 destroy_group.....	176
10.2.3.2.3 get_rank_size.....	177
10.2.3.2.4 get_local_rank_size.....	177
10.2.3.2.5 get_rank_id.....	178
10.2.3.2.6 get_local_rank_id.....	179
10.2.3.2.7 get_world_rank_from_group_rank.....	180
10.2.3.2.8 get_group_rank_from_world_rank.....	181
10.2.3.3 hccl.split.api.....	182
10.2.3.3.1 set_split_strategy_by_idx.....	182
10.2.3.3.2 set_split_strategy_by_size.....	184
10.2.3.4 npu_bridge.hccl.hccl_ops.....	185
10.2.3.4.1 allreduce.....	185
10.2.3.4.2 allgather.....	187
10.2.3.4.3 broadcast.....	187
10.2.3.4.4 reduce_scatter.....	189

10.2.3.4.5 send.....	190
10.2.3.4.6 receive.....	191
10.3 Environment Variables.....	192
10.3.1 JOB_ID.....	192
10.3.2 ASCEND_DEVICE_ID.....	192
10.3.3 RANK_TABLE_FILE.....	192
10.3.4 RANK_ID.....	193
10.3.5 RANK_SIZE.....	193
10.3.6 GE_USE_STATIC_MEMORY.....	193
10.3.7 PROFILING_MODE.....	194
10.3.8 PROFILING_OPTIONS.....	194
10.3.9 SKT_ENABLE.....	195
10.3.10 OP_NO_REUSE_MEM.....	195
10.3.11 ENABLE_NETWORK_ANALYSIS_DEBUG.....	196
10.3.12 DUMP_GE_GRAPH.....	196
10.3.13 DUMP_GRAPH_LEVEL.....	196
10.3.14 TE_PARALLEL_COMPILER.....	196
10.3.15 ASCEND_MAX_OP_CACHE_SIZE.....	197
10.3.16 ASCEND_REMAIN_CACHE_SIZE_RATIO.....	197
10.3.17 HCCL_INTRA_ROCE_ENABLE.....	197
10.3.18 HCCL_INTRA_PCIE_ENABLE.....	197
10.3.19 ASCEND_SLOG_PRINT_TO_STDOUT.....	198
10.3.20 ASCEND_GLOBAL_LOG_LEVEL.....	198
10.3.21 ASCEND_GLOBAL_EVENT_ENABLE.....	199
10.3.22 ASCEND_LOG_DEVICE_FLUSH_TIMEOUT.....	199
10.4 Device Resource Configuration File Templates.....	199
11 FAQ.....	204
11.1 What Do I Do If Network Size Reaches Threshold?.....	204
11.2 What Do I Do If Training Times Out Due to Too Many Dataset Shuffle Operations?.....	205
11.3 How Do I Determine fp_point and bp_point?.....	206
11.4 What Do I Do If uint8 and quint8 Addition/Subtraction Overflows?.....	208
12 Appendixes.....	209
12.1 How Do I Install GCC 7.3.0?.....	209
12.2 Change History.....	210

1 About This Document

Purpose

This guide provides step-by-step instructions on migrating a training script developed with the TensorFlow Python API to Ascend AI Processor.

Before You Start

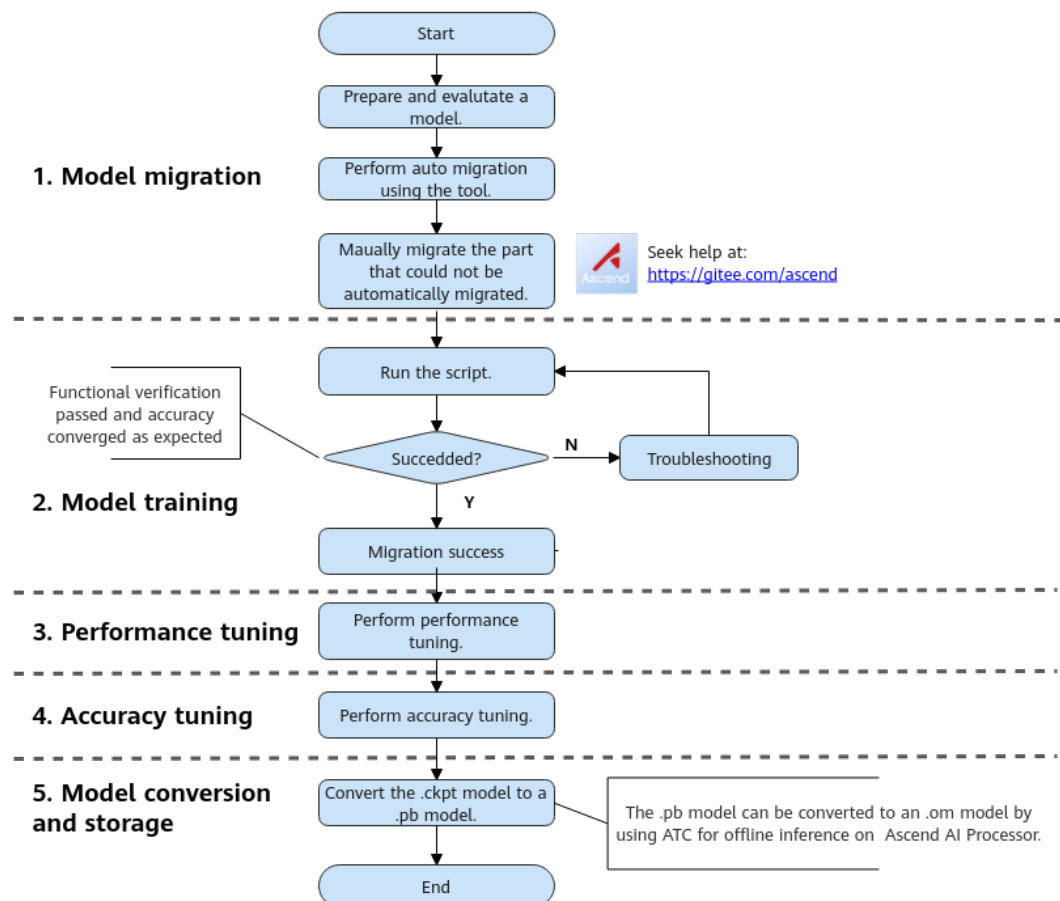
The code snippets in this document are only examples. Manual tweaking is needed.

2 Restrictions and Limitations

1. The compatible TensorFlow version is 1.15. For details about the support for TensorFlow APIs, see [10.2.1 Available TensorFlow APIs](#).
2. In the **inferredshape** phase, the operators do not support unknown shape inference.
3. Currently, the system supports the following formats: NCHW, NHWC, NC, HWCN, and CN.
4. If manual mixed precision has been implemented in the original script (for example, explicitly calling the cast operator for precision conversion), the system preferentially retains the source image precision by default. That is, when the operator does not support the float32 data type, the precision is reduced to float16.
5. For condition branches and loop branches, only **TF.cond**, **TF.while_loop**, and **tf.case** are supported.
6. In training with multiple devices, **NPURunconfig** does not support **save_checkpoints_secs**.
7. In training with multiple devices, it is not allowed to save the summary information of only a single device.
8. If the value of **iterations_per_loop** is greater than 1, the value of **save_checkpoints_steps** must be a positive integer multiple of **iterations_per_loop**; otherwise, checkpoint data is not saved as defined by **save_checkpoints_steps**. If the value of **iterations_per_loop** is greater than 1, data is not saved as defined by **save_summary_steps** and **log_step_count_steps**. For details, see [9.2 Log and Summary Operators](#).
9. Only the Summary, Log, and Data operators support the string data type.
10. The operators do not support the inf or nan inputs.
11. The restrictions on collective communication are as follows:
 - Allocation at only 1, 2, 4, or 8 Ascend AI Processors per server is supported.
 - AllReduce and ReduceScatter support only the int8, int32, float16, and float32 data types.
12. For data preprocessing, the getnext operator can be offloaded to the device only by using **tf.data.make_initializable_iterator**.

3 Migration Workflow

Model migration is to migrate models that have been implemented in the open-source community to Ascend AI Processor. The following figure shows the model migration workflow.



4 Auto Migration and Manual Migration

[4.1 Model Preparation and Evaluation](#)

[4.2 Auto Migration Using Tool](#)

[4.3 Manual Migration](#)

4.1 Model Preparation and Evaluation

Model Preparation

Before model migration to Ascend AI Processor, prepare a model developed on TensorFlow 1.15 and a matched dataset, run the model on GPU or CPU to test if the accuracy is converged as expected. In addition, record the accuracy and performance specifications for comparison on the Ascend AI Processor later on.

Model Evaluation

Currently, the Ascend platform works with TensorFlow 1.15 only. Check if all the APIs used in your model are listed in [10.2.1 Available TensorFlow APIs](#) before you move on to [4.2 Auto Migration Using Tool](#). Avoid using unsupported APIs in your model.

4.2 Auto Migration Using Tool

Overview

The Ascend platform provides the **TF1.x** script migration tool, which is applicable to the migration of native TensorFlow training scripts. You can use this tool to automatically migrate and adapt TensorFlow Python APIs and Horovod Python APIs for training on Ascend AI Processors. The migration tool can only migrate most content of your training script. Perform manual migration for the residual part by referring to [4.3 Manual Migration](#).

Tool Acquisition

Find the **TF1.x** script migration tool from **/tfplugin/python/site-packages/npu_bridge/conver_tf2npu/** in the TFPlugin installation directory.

Restrictions

Before using this tool, note the following restrictions:

Make sure the TensorFlow and Horovod modules have been imported to your training script in strict accordance with the following format:

```
import tensorflow as tf
import tensorflow.compat.v1 as tf
import horovod.tensorflow as hvd
```

Tool Usage

Run the following command in the directory where the migration tool is stored:

```
python3 main.py -i /home/BERT -o /home/out -r /home/report
```

The arguments are described as follows.

main.py: main function of the tool.

/home/BERT: path of the training script to be migrated. Must be a folder path.

/home/out: target path of script migration.

/home/report: path of the migration report.

NOTE

You can run the **python3 main.py -h** command to obtain the help information.

There are three types of migration reports:

- **success_report.txt**: migration success report.
For example, the following report indicates that **hvd.rank** in line 522 of **run_ner.py** has been migrated to **get_rank_id** of **NPU**.

```
/home/BERT/run_ner.py:522 "change hvd.rank to get_rank_id"
```
- **failed_report.txt**: migration failure report.
- **need_migration_doc.txt**: report about the part requiring manual migration.
The information provided in this report might be incomplete currently. Refer to [4.3 Manual Migration](#) for more information.

Options Migratable Using Tool

1. TensorFlow or Horovod APIs can be automatically migrated by the tool. The mapping of APIs before and after migration is summarized as follows:
 - "batch(xxx)" --> "batch(xxx, drop_remainder=True)"
 - "map_and_batch(xxx)" --> "map_and_batch(xxx, drop_remainder=True)"
 - "dataset.shard(xxx, xxx)" --> "dataset.shard(get_rank_size(), get_rank_id())"

- "tf.device(xxx)" --> "tf.device(/cpu:0)"
 - "tf.estimator.EstimatorSpec" --> "NPUEstimatorSpec"
 - "tf.estimator.RunConfig" --> "NPURunConfig"
 - "tf.estimator.Estimator" --> "NPUEstimator"
 - "tf.nn.dropout" --> "npu_ops.dropout"
 - "tf.compat.v1.layers.max_pooling2d" -->
"tf.compat.v1.nn.max_pool_with_argmax"
 - "tf.layers.max_pooling2d" --> "tf.nn.max_pool_with_argmax"
 - "hvd.init" --> "None"
 - "hvd.DistributedOptimizer" --> "NPUDistributedOptimizer"
 - "hvd.rank" --> "get_rank_id"
 - "hvd.local_rank" --> "get_local_rank_id"
 - "hvd.size" --> "get_rank_size"
 - "BroadcastGlobalVariablesHook" --> "None"
2. The following function can be automatically migrated by the tool.
The **gelu** functions in your script will be replaced with **npu_unary_ops.gelu**.
See the following example.
- Script before migration:
- ```
def gelu(x):
 cdf = 0.5 * (1.0 + tf.tanh(
 (np.sqrt(2 / np.pi) * (x + 0.044715 * tf.pow(x, 3)))))
 return x*cdf
layers = gelu()
```
- Script after migration:
- ```
layers = npu_unary_ops.gelu(x)
```
3. For a Python script, the "from npu_bridge.npu_init import *" line will be added to the script for importing dependent NPU library files.

4.3 Manual Migration

4.3.1 tf.config.optimizer.set_experimental_options Options to Migrate

If the **tf.config.optimizer.set_experimental_options** API is used in your training script, note the following restrictions:

- The following configuration option is deactivated by default and should remain deactivated:
rewrite_options.disable_model_pruning
- The following configuration options are activated by default and should remain activated:
 - rewrite_options.function_optimization
 - rewrite_options.constant_folding
 - rewrite_options.shape_optimization
 - rewrite_options.arithmetic_optimization

- `rewrite_options.loop_optimization`
- `rewrite_options.dependency_optimization`
- `rewrite_options.layout_optimizer`
- `rewrite_options.memory_optimization`

4.3.2 `tf.case`, `tf.cond`, and `tf.while_loop` Options to Migrate

If the input has a static shape, **`tf.case`**, **`tf.cond`**, and **`tf.while_loop`** are supported by Ascend AI Processor naturally. Therefore, the script requires no more editing.

If the input has a dynamic shape, upgrade the control flow operators of the V1 version to those of the V2 version to support the dynamic shape function. Only TensorFlow V2 control flow operators (such as `If`, `Case`, `While`, `For`, and `PartitionedCall`) support dynamic shape. TensorFlow V1 control flow operators (such as `Switch`, `Merge`, `Enter`, `LoopCond`, `NextIteration`, `Exit` and `ControlTrigger`) corresponding to the **`tf.case`**, **`tf.cond`**, and **`tf.while_loop`** APIs do not support dynamic shape. In addition, if there are many branch structures on the network, using the control flow operator of the V1 version may result in excessive streams.

```
[ERROR] DRV(13489,rtstest_host):2020-10-29-05:26:01.002.033 [hardware/dev_plat/./dev_core/tsdrv/  
tsdrv_id.c:172][devdrv] [drvStreamIdAlloc 172] ioctl failed, devId(0) tsId(0), strategy(0), ret(-1).  
[ERROR] RUNTIME(13489,rtstest_host):2020-10-29-05:26:01.002.050 [runtime/feature/src/npv_driver.cc:  
102]13489 StreamIdAlloc:drvStreamIdAlloc failed: errorcode=17, device_id=0, tsId=0, strategy=0  
[ERROR] RUNTIME(13489,rtstest_host):2020-10-29-05:26:01.002.061 [runtime/feature/src/stream.cc:  
184]13489 Setup:fail to alloc stream id
```

In this case, you also need to upgrade the control flow operators of the V1 version to the V2 version to solve the problem.

Specifically:

Call the following API at the beginning of the training script:

```
tf.enable_control_flow_v2()
```

Perform necessary adaptations. For example, the following error may be reported during script execution:

```
TypeError: In op 'NovoGrad/update_Loss_Optimization/FP32-master-copy/ForwardPass/w2l_encoder/conv11/  
kernel/ApplyMomentum', input types([tf.float32,tf.float32,tf.float32,tf.float32,tf.float32]) are not  
compatible with expected types ([tf.float32_ref,tf.float32_ref,tf.float32,tf.float32,tf.float32])
```

As the message tells, `input0` and `input1` of the `ApplyMomentum` operator are of type reference, instead of variable. In this case, convert the inputs of the `ApplyMomentum` operator from variables to resource variables:

```
tf.enable_resource_variables()
```

4.3.3 `tf.ConfigProto` Options to Migrate

If your training script uses the **`sess.run`** method, add the following lines in bold in the training session after automatic migration is complete using the migration tool:

```
config = tf.ConfigProto() # If your script does not contain tf.ConfigProto, you need to manually add this  
line.  
custom_op = config.graph_options.rewrite_options.custom_optimizers.add() # Manually add this line.  
custom_op.name = "NpuOptimizer" # Manually add this line.  
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Manually add this line.  
sess = tf.Session(config=config)
```

```
sess.run(init)
train_batches_per_epoch=int(np.floor(train_size/batch_size))
```

4.3.4 Keras Options to Migrate

If you are using a Keras training script, the script migrated to the Ascend platform will lose support of certain features such as the dynamic learning rate. Therefore, you are not advised to migrate Keras scripts to the Ascend platform. To run a Keras script on the Ascend platform, you need to edit the script as follows:

Create a TensorFlow session, register Keras, and activate **use_off_line** for training on Ascend AI Processor. Remember to close the session when the training is complete.

```
import tensorflow as tf
import tensorflow.python.keras as keras
from tensorflow.python.keras import backend as K
from npu_bridge.npu_init import *

sess_config = tf.ConfigProto()
custom_op = sess_config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
sess_config.graph_options.rewrite_options.remapping = RewriterConfig.OFF
sess = tf.Session(config=sess_config)
K.set_session(sess)

# Preprocess the data...
# Construct a model...
# Build the model...
# Train the model...

sess.close()
```

With the migrated training script, the number of training iterations performed by the Ascend AI Processor is fixed to 1 in every **session.run** call. To reduce data transfers between the host and device and shorten the training duration, use the **model_to_npu_estimator** API to convert the model constructed by Keras to an **NPUEstimator** object and set **iterations_per_loop** (number of training iterations performed by the Ascend AI Processor every **session.run** call) in **NPURunConfig** to a value that suits your needs. For details, see [Setting iterations_per_loop with Keras](#).

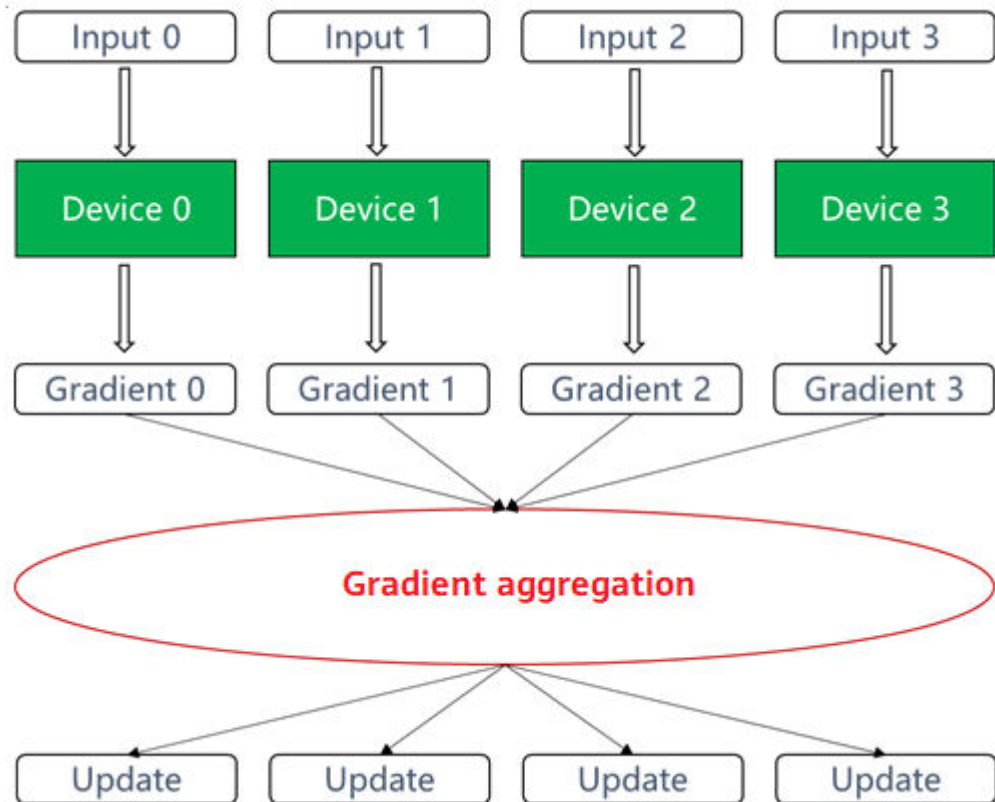
4.3.5 Configuration Options for Distributed Training Migration

4.3.5.1 Typical Scenarios

Overview

In a large-scale AI training cluster, training is generally completed in a data parallel manner. Data parallelism means that all devices use the same model and different training samples, and gradient data calculated by each device needs to be aggregated for parameter update.

Figure 4-1 Schematic diagram of data-parallel training

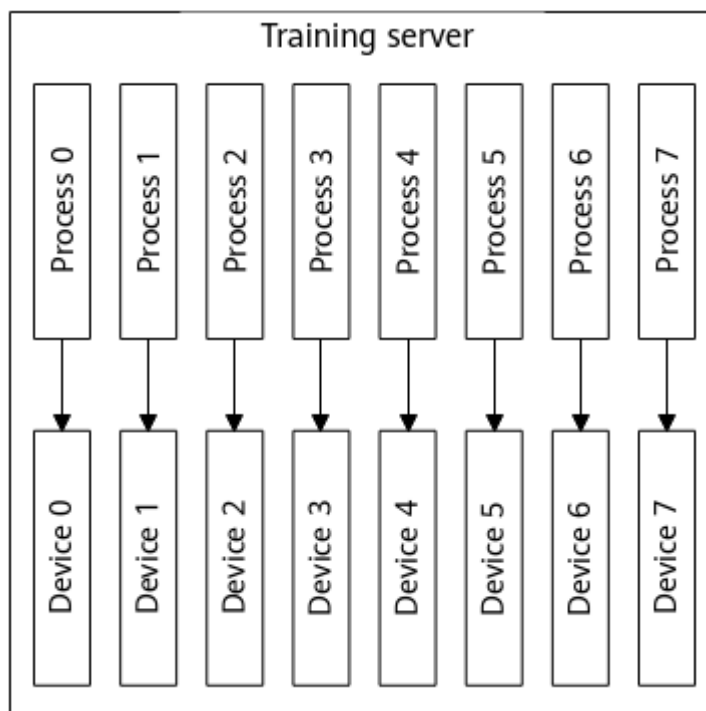


If classification is performed in gradient aggregation manner, the mainstream implementation of data parallelism includes the Parameter Server-Workers (PS-Worker) architecture and AllReduce collective communication. The Ascend platform supports both implementation types. For details, see [4.3.5.2 Distributed Training Based on the AllReduce Architecture](#) and [4.3.5.3 Distributed Training Based on the PS-Worker Architecture](#).

Single-Server Scenario

In the single-server scenario, one training server is used to complete the training. The server consists of eight Ascend AI Processors. Only one, two, four, or eight devices participate in collective communication, and devices 0 to 3 and devices 4 to 7 form a network respectively. When two or four devices are used for training, cross-network cluster creation is not supported.

Figure 4-2 Single-server training



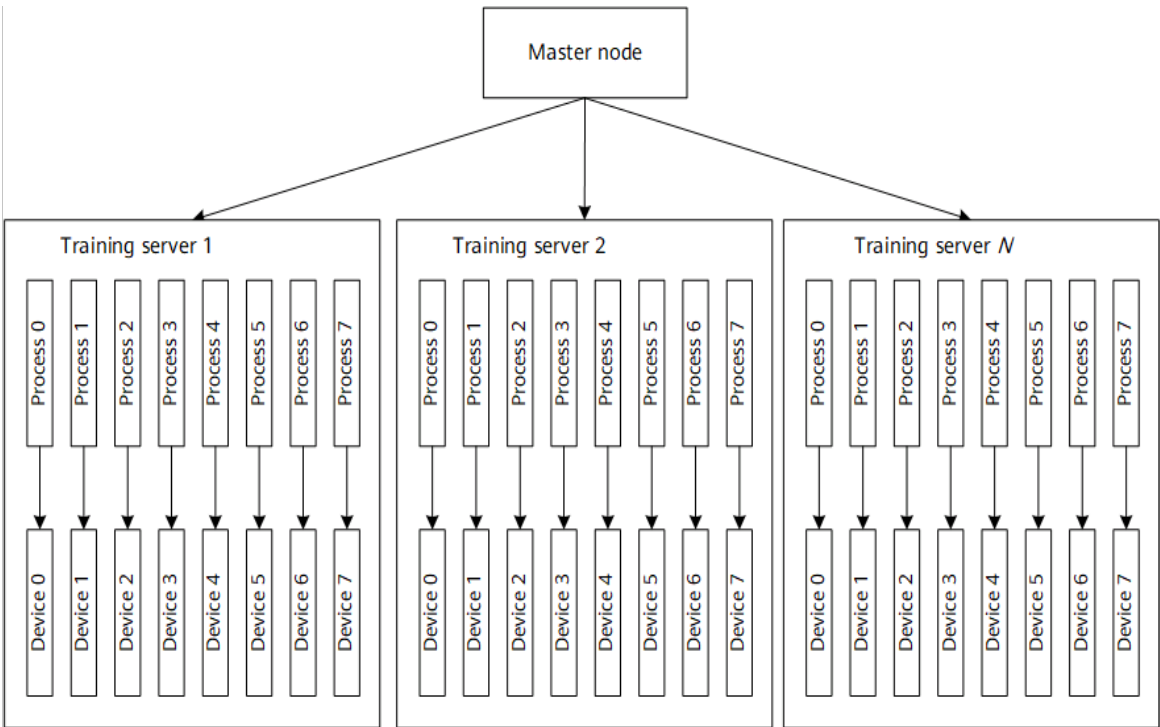
Server Cluster Scenario

In a server cluster scenario, a training server cluster consists of the master node for cluster management and a bunch of slave training servers. Currently, a server cluster can have up to 128 servers. Each server consists of eight Ascend AI Processors. In a server cluster, the number of Ascend AI Processors that participate in collective communication equals to one, two, four, or eight times the number of the training servers. The cluster performance is maximized when the number of training servers is a power of 2, which is therefore highly recommended.

NOTE

The number and positions of the Ascend AI Processors across the servers must be consistent. When the number of Ascend AI Processors used for training is 2 or 4 times the number of the training servers, devices 0 to 3 and devices 4 to 7 form a network respectively. In this case, cross-network cluster creation is not supported.

Figure 4-3 Cluster training

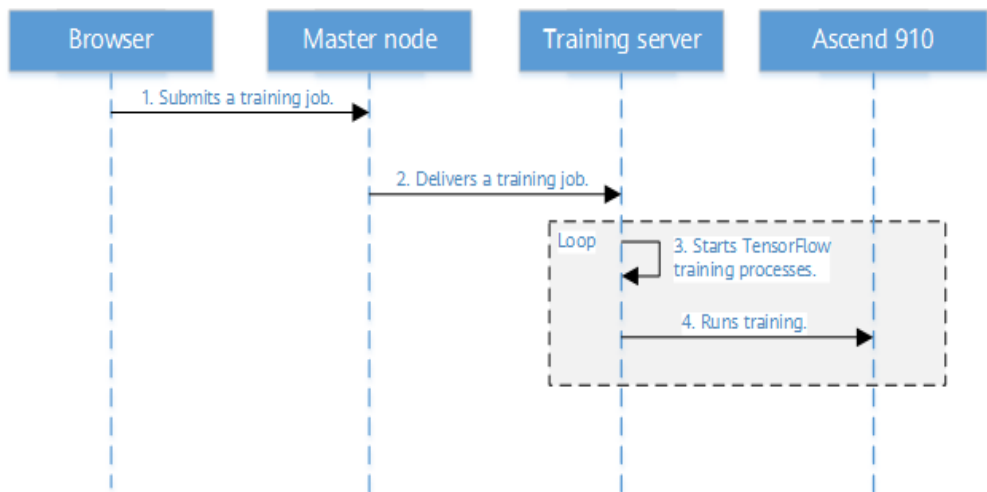


 NOTE

The master cluster management node manages the cluster and devices in the cluster, and supports distributed job management in the entire cluster.

In the cluster training scenario, a distributed training flow is as follows.

Figure 4-4 Distributed training workflow



The training job is delivered to the training server through the master node. The job agent on each server starts a number of TensorFlow processes to perform training based on the number of devices specified by the application. One TensorFlow process corresponds to one Ascend AI Processor.

Typically, the eight network ports of each server are enabled to implement collective communication between servers. In some scenarios, training can be implemented with just some of the network ports. That is, you can enable just one, two, or four network ports on each server.

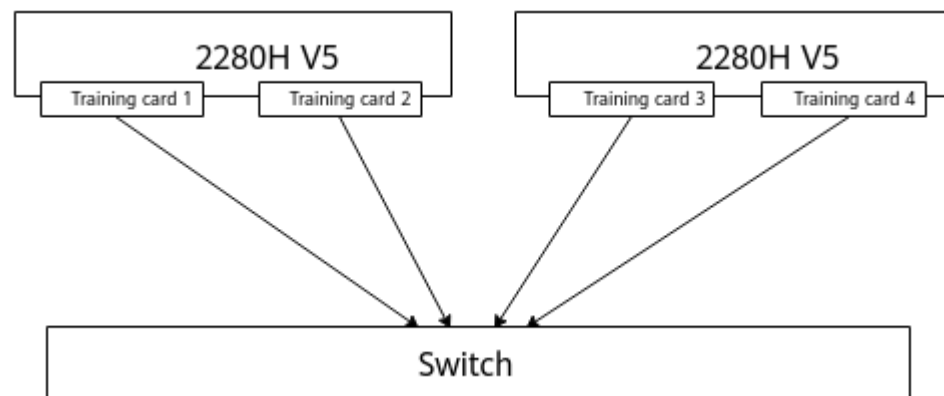
- One, two, or four network ports of each server are enabled to network a training cluster. Recommendation for network port selection is as follows:
 - To enable only one network port on each server, the performance of each network port is the same.
 - To enable two network ports on each server, select network ports [0, 5], [1, 4], [2, 7], or [3, 6] for optimal performance.
 - To enable four network ports on each server, select network ports [0, 2, 5, 7] or [1, 3, 4, 6] for optimal performance.
- Broadcast, AllReduce, ReduceScatter, and AllGather of the training cluster is implemented through intra-node and inter-node collective communication. Data communication between nodes can be implemented only through the available one, two, or four network ports.
- The number of Ascend AI Processors participated in collective communication must be 8 times the number of the training servers.

Atlas 300T training card (model: 9000)

Currently, training using the training cards allows two scenarios: single-server single-device training and multi-server multi-device distributed training. A single training card is equipped with one Ascend AI Processor.

For distributed training with multiple servers, the 100G ETH ports provided by each training card are used for communication between servers, and the Ring and Halving-doubling algorithms are used to implement collective communication.

Figure 4-5 Networking diagram



Precautions:

1. The number of training cards in different servers must be the same.
2. In the entire network, the IP addresses of the NICs of each training card are in the same network segment.
3. Currently, only AllReduce, AllGather, Broadcast, and ReduceScatter are supported.

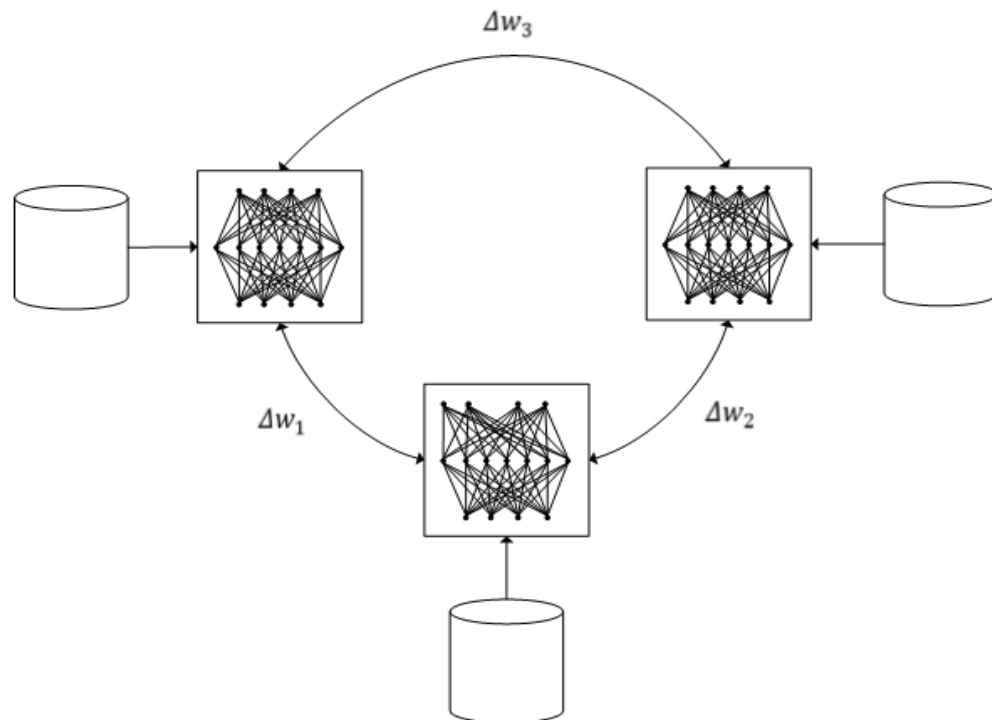
4. Before training, use the **HCCL_INTRA_PCIE_ENABLE** and **HCCL_INTRA_ROCE_ENABLE** environment variables to set the communication mode between devices. The PCIe path is used by default. The RoCE path is recommended. For details, see [10.3 Environment Variables](#).
5. The number of Ascend AI Processors participating in training specified in the ranktable configuration file cannot be greater than the number of available Ascend AI Processors on the server. Template 1 must be used. For details, see [10.4 Device Resource Configuration File Templates](#).

4.3.5.2 Distributed Training Based on the AllReduce Architecture

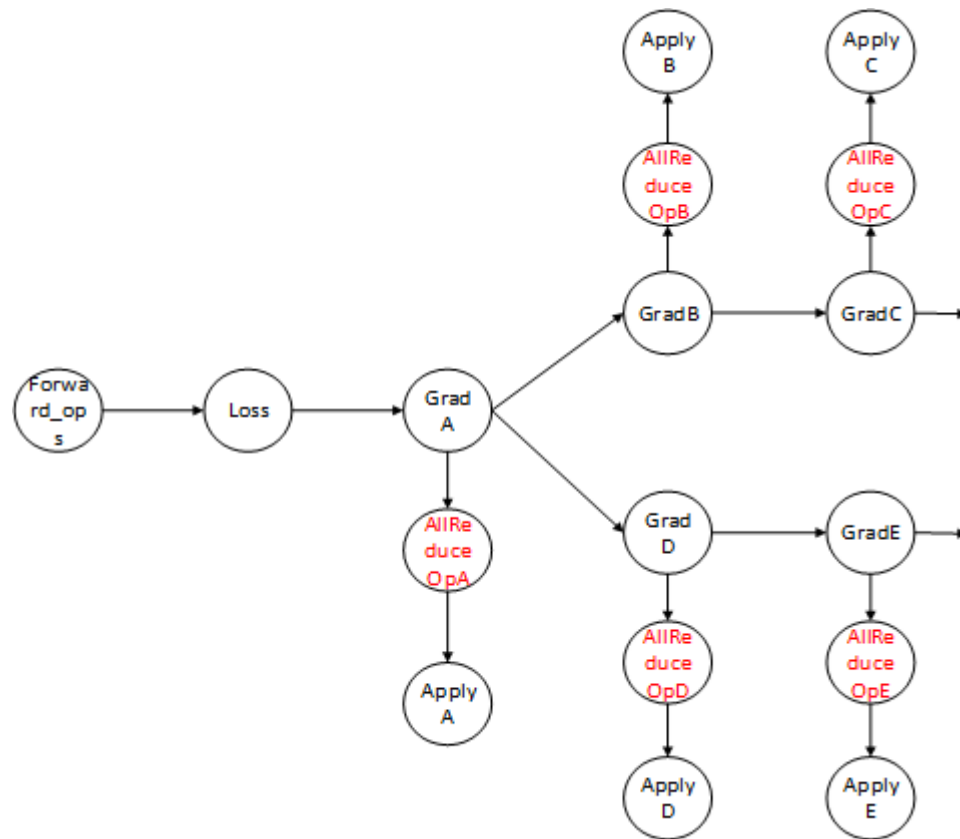
Overview

In the AllReduce architecture, all devices used for training form a logical ring, as shown in [Figure 4-6](#). There is no central node to aggregate the calculated gradients. Each device receives data from the upstream device and transmits data to the downstream device to fully utilize the bandwidth.

Figure 4-6 AllReduce architecture



The Ascend platform provides the **NPUDistributedOptimizer** high-level distributed API to implement gradient aggregation in the AllReduce architecture. **NPUDistributedOptimizer** encapsulates the single-server training optimizer into an NPU-based distributed training optimizer, to support single-server multi-device and multi-server multi-device scenarios. Gradients are calculated in each device before aggregation. After the **NPUDistributedOptimizer** optimizer is called, AllReduce operators are inserted between the gradient computation and update operators in the generated training graph.



Distributed Training with Estimator

To perform distributed training in **Estimator** mode, modify the training script as follows:

1. TensorFlow uses **train_distribute** in **Runconfig** to specify a distributed training policy. The Ascend platform does not support **train_distribute**. You need to delete related code.

2. During model training, class **NPUDistributedOptimizer** encapsulates the single-server training optimizer into an NPU distributed training optimizer.

```

from npu_bridge.npu_init import *

def cnn_model_fn(features, labels, mode):
    # Construct the network.
    xxx
    # Calculate the loss.
    xxx

    #Configure the TrainingOp(for TRAIN mode)
    if mode == tf.estimator.ModeKeys.TRAIN:
        optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001) # Use the SGD optimizer.
        distributedOptimizer=NPUDistributedOptimizer(optimizer) # Use the NPU-based distributed
        computing to update gradients.
        train_op=distributedOptimizer.minimize(loss=loss,global_step=tf.train.get_global_step()) # Minimize
        the loss.
        return tf.estimator.EstimatorSpec(mode=mode,loss=loss,train_op=train_op)
  
```

NOTE

- If the original script uses a TensorFlow API to calculate the gradients, for example, `grads = tf.gradients(loss, tvars)`, after **NPUDistributedOptimizer** is constructed, replace the API with the `compute_gradients` and `apply_gradients` methods of **NPUDistributedOptimizer**.
- In **Estimator** mode, when **NPUDistributedOptimizer** is used to implement the AllReduce function, **NPUBroadcastGlobalVariablesHook** is automatically added to **NPUEstimator**. Therefore, you do not need to manually implement the broadcast function.

Distributed Training with `sess.run`

To perform distributed training by using the `sess.run` method, modify the training script as follows:

1. When creating a session, you need to manually add the **GradFusionOptimizer** of the NPU to fuse broadcast operators and ensure that associated optimization options such as pruning of TensorFlow are enabled.

```
from npu_bridge.npu_init import *

# Create a session.
config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map[["use_off_line"].b] = True
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF
config.graph_options.rewrite_options.optimizers.extend(["pruning",
                                                         "function",
                                                         "constfold",
                                                         "shape",
                                                         "arithmetic",
                                                         "loop",
                                                         "dependency",
                                                         "layout",
                                                         "memory",
                                                         "GradFusionOptimizer"])

sess = tf.Session(config=config)
```

2. After the variables are initialized and before the training, the variables are broadcast through the collective communication API **broadcast**.

```
from npu_bridge.npu_init import *

def broadcast_global_variables(root_rank, index):
    """Broadcasts all global variables from root rank to all other processes.
    Arguments:
    root_rank: rank of the process from which global variables will be broadcasted
    to all other processes.
    index: rank_id
    """
    op_list = []
    for var in tf.global_variables():
        # the input and out tensor of HCOMBroadcast interface are list
        if "float" in var.dtype.name:
            inputs = [var]
            outputs = hcc_ops.broadcast(tensor=inputs, root_rank=root_rank)
            if outputs is not None:
                op_list.append(outputs[0].op)
                op_list.append(tf.assign(var, outputs[0]))

    return tf.group(op_list)

...
```

```
bcast_op = broadcast_global_variables(root_rank, index)
sess = tf.Session()
...
sess.run(bcast_op)
```

- During the training, after the gradient data of each device is calculated, the gradient data is aggregated through the collective communication API **allreduce**.

```
from npu_bridge.npu_init import *
grads = [ hccL_ops.allreduce(grad, "sum") for grad in grads ]
```

Alternatively, use the **NPUDistributedOptimizer** distributed training optimizer to aggregate gradient data.

```
from npu_bridge.npu_init import *
optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001) # Use the SGD optimizer.
distributedOptimizer=NPUDistributedOptimizer(optimizer) # Use the NPU-based distributed
computing to update gradients.
```

Distributed Training with Keras

To perform distributed training by using the **Keras** method, modify the training script as follows:

- Modify the optimizer during Keras model build. Use the TensorFlow single-server training optimizer (do not use the Keras optimizer) and use class **NPUDistributedOptimizer** to encapsulate the single-server training optimizer. For example:

```
from npu_bridge.npu_init import *

opt = tf.compat.v1.train.AdamOptimizer(learning_rate=0.1)
opt = NPUDistributedOptimizer(opt)
keras_model.compile(optimizer=opt,loss='sparse_categorical_crossentropy')
```

In the distributed scenario, the dynamic learning rate cannot be set in the callback function.

- (Optional) If a session is created, you need to manually add the **GradFusionOptimizer** optimizer.

```
import tensorflow as tf
import tensorflow.python.keras as keras
from tensorflow.python.keras import backend as K
from tensorflow.core.protobuf.rewriter_config_pb2 import RewriterConfig
from npu_bridge.npu_init import *

sess_config = tf.ConfigProto()
custom_op = sess_config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
sess_config.graph_options.rewrite_options.remapping = RewriterConfig.OFF
sess_config.graph_options.rewrite_options.optimizers.extend(["GradFusionOptimizer"]) # Add
for distributed training
sess = tf.Session(config=sess_config)
K.set_session(sess)

# Preprocess the data...
# Construct a model...
# Build the model...
# Train the model...

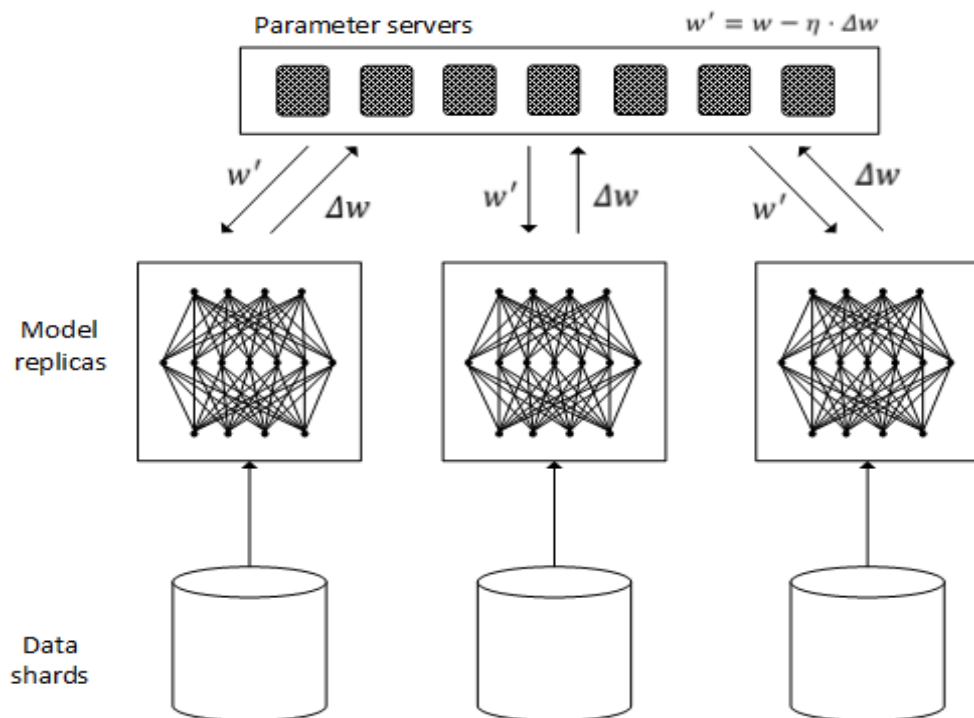
sess.close()
```

4.3.5.3 Distributed Training Based on the PS-Worker Architecture

Overview

In the PS-Worker architecture, servers in a cluster are classified into two types: parameter servers and workers. Parameter servers store model parameters, and workers calculate the gradients. In each iteration, workers obtain parameters from parameter servers, and then return the calculated gradients to the parameter servers. Parameter servers aggregate the calculated gradients, update the parameters, and then broadcast the updated parameters to workers.

Figure 4-7 PS-Worker architecture



This section describes how to migrate the training scripts developed using TensorFlow Python APIs, so as to perform distributed training on Ascend AI Processor based on the PS-Worker architecture.

NOTICE

1. Currently, distributed training on Ascend AI Processor based on the PS-Worker architecture supports only the **NPUEstimator** mode.
2. Process of a worker can only be executed on one device.
3. In the PS-Worker architecture scenario, you are advised to use high-speed NICs.

Configuring the Cluster

In the PS-Worker architecture, cluster is configured using the **TF_CONFIG** environment variable, which contains the **cluster** and **task** components. **cluster**

provides information about the entire cluster, namely the workers and parameter servers in the cluster. **task** provides information about the current task. For details, visit https://tensorflow.google.cn/tutorials/distribute/multi_worker_with_estimator.

The following uses the two-server (each server has one parameter server and 8 workers) scenario as an example:

1. Set **TF_CONFIG**.

```
os.environ['TF_CONFIG'] = json.dumps({
    'cluster': {
        #'chief': chief_hosts, # Optional
        'worker': worker_hosts,
        'ps': ps_hosts,
        'evaluator': evaluator_hosts, # Not required if evaluation is not performed
    },
    'task': {'type': job_name, 'index': task_index}
})
```

2. Configure **ps_hosts** and **worker_hosts** using **FLAGS** as follows.

```
ps_hosts = FLAGS.ps_hosts.split(',')
worker_hosts = FLAGS.worker_hosts.split(',')
evaluator_hosts = FLAGS.evaluator_hosts.split(',')
task_index = FLAGS.task_index
job_name = FLAGS.job_name
flags.DEFINE_string("ps_hosts", '192.168.1.100:2222,192.168.1.200:2222',
    '192.168.1.100:2223, 192.168.1.100:2224, 192.168.1.100:2225, 192.168.1.100:2226,'
    '192.168.1.100:2227, 192.168.1.100:2228, 192.168.1.100:2229, 192.168.1.100:2230,'
    '192.168.1.200:2223, 192.168.1.200:2224, 192.168.1.200:2225, 192.168.1.200:2226,'
    '192.168.1.200:2227, 192.168.1.200:2228, 192.168.1.200:2229, 192.168.1.200:2230',)
flags.DEFINE_string("evaluator_hosts", '192.168.1.100:2231')
flags.DEFINE_string("job_name", "", "One of 'ps', 'worker', 'evaluator', 'chief'")
flags.DEFINE_integer("task_index", 0, "Index of task within the job")
```

Configuration description:

- **worker_hosts** and **ps_hosts**: Separate the items by commas (,) without spaces.
- **chief_hosts**: Only one argument can be set at most, which can also be left empty as in the preceding example. If **chief** is not specified, the first worker is used as the chief by default. The chief worker performs model training as other workers, and also manages other work, for example, checkpoint saving and restoration, as well as summary writing.
- **evaluator_hosts**: Only one argument can be set, which is not required if evaluation is not performed.

Next, you need to configure **TF_CONFIG** for all workers.

Defining the ParameterServerStrategy Instance

To support distributed training in the PS-Worker architecture, the **tf.distribute.experimental.ParameterServerStrategy** instance needs to be defined first. For details about this strategy, see [tf.distribute.experimental.ParameterServerStrategy](#).

```
strategy = tf.distribute.experimental.ParameterServerStrategy()
```

Training and Evaluating the Model

In **NPURunConfig**, you need to specify the distribution strategy for **NPUEstimator** by using the **distribute** parameter, and then call [tf.estimator.train_and_evaluate](#) to train and evaluate the model.

Ensure that **NPURunConfig.model_dir** of all workers is set to the same directory. For example, for a shared file system that can be read and written by all workers, if a directory is set for worker 1, this shared directory must be mounted to worker 2, and the values of **NPURunConfig.model_dir** must be the same.

```
from npu_bridge.npu_init import *

run_config = NPURunConfig(
    model_dir=flags_obj.model_dir,
    session_config=session_config,
    keep_checkpoint_max=5,
    save_summary_steps=1,
    log_step_count_steps=1,
    save_checkpoints_steps=100,
    enable_data_pre_proc=True,
    mix_compile_mode=True, # PS-Worker supports only mixed precision.
    iterations_per_loop=1, # This value must be 1 in mixed precision mode.
    precision_mode='allow_mix_precision',
    distribute=strategy)

classifier = tf.estimator.NPUEstimator(
    model_fn=model_fn,
    model_dir='/tmp/multiworker',
    config=run_config)

tf.estimator.train_and_evaluate(
    classifier,
    train_spec=tf.estimator.TrainSpec(input_fn=input_fn),
    eval_spec=tf.estimator.EvalSpec(input_fn=input_fn))
```

NOTICE

The evaluation process can be executed on the device or the host CPU. You can decide as required.

The following uses the single-server, 8-device scenario as an example. One parameter server process and 8 worker processes are needed, and the 8 worker processes are executed on the device side.

- **To perform evaluation and training at the same time**, the number of processes started by the evaluator and workers at the same time cannot exceed the number of devices on the server (that is, 8 in the given example). Since the 8 devices are already used by the worker processes in the example, evaluation needs to be performed on the host CPU. In this case, although training and evaluation can be performed in parallel, the compute capability of Ascend AI Processor is not utilized for evaluation. If evaluation is performed in this mode, it is recommended that checkpoint storage duration be set longer than the evaluation duration.

To perform evaluation on the host side, call the native TensorFlow **Estimator**. **Estimator** should not be converted into **NPUEstimator** to avoid using device resources. Otherwise, evaluation fails because the devices are already used for training.

- **To perform evaluation after training**, ensure that the evaluator is executed after the workers complete the training. In this case, both the training and evaluation processes are executed on the devices to achieve optimal performance.

Running the Script

To run the script using the **ps_hosts** and **worker_hosts** information in the Python script (**chief** is not defined in the Python script):

```
python resnet50_ps_strategy.py --job_name=ps --task_index=0
python resnet50_ps_strategy.py --job_name=ps --task_index=1
python resnet50_ps_strategy.py --job_name=worker --task_index=0
python resnet50_ps_strategy.py --job_name=worker --task_index=1
python resnet50_ps_strategy.py --job_name=worker --task_index=2
python resnet50_ps_strategy.py --job_name=worker --task_index=3
python resnet50_ps_strategy.py --job_name=worker --task_index=4
python resnet50_ps_strategy.py --job_name=worker --task_index=5
python resnet50_ps_strategy.py --job_name=worker --task_index=6
python resnet50_ps_strategy.py --job_name=worker --task_index=7
```

To redefine **ps_hosts** and **worker_hosts** (**chief** is not defined in the Python script):

```
python resnet50_ps_strategy.py \
    --ps_hosts=192.168.1.79:2222,192.168.1.80:2222 \
    --
worker_hosts=192.168.1.79:2223,192.168.1.79:2224,192.168.1.79:2225,192.168.1.79:2226,192.168.1.79:2227,192.168.1.79:2228,192.168.1.79:2229,192.168.1.79:2230,192.168.1.80:2223,192.168.1.80:2224,192.168.1.80:2225,192.168.1.80:2226,192.168.1.80:2227,192.168.1.80:2228,192.168.1.80:2229,192.168.1.80:2230 \
    --job_name=ps \
    --task_index=0
```

To run **chief** and **evaluator**, modify **job_name** to the defined type value as follows.

```
python resnet50_ps_strategy.py --job_name=chief --task_index=0
python resnet50_ps_strategy.py --job_name=evaluator --task_index=0
```

NOTE

For details about the dependent environment variables, see [5.2 Training in a Bare Metal Environment](#).

4.3.5.4 Distributed Training Based on the Horovod Framework

If your model is trained in distributed mode based on the Horovod framework, you need to manually edit your script after auto migration using the migration tool.

Original Horovod code:

```
import tensorflow as tf
import horovod.tensorflow as hvd

# Initialize Horovod
hvd.init()

# Pin GPU to be used to process local rank (one GPU per process)
config = tf.ConfigProto()
config.gpu_options.visible_device_list = str(hvd.local_rank())

# Build model...
loss = ...
opt = tf.train.AdagradOptimizer(0.01 * hvd.size())

# Add Horovod Distributed Optimizer
opt = hvd.DistributedOptimizer(opt)

# Add hook to broadcast variables from rank 0 to all other processes during
# initialization.
hooks = [hvd.BroadcastGlobalVariablesHook(0)]

# Make training operation
```

```
train_op = opt.minimize(loss)

# Save checkpoints only on worker 0 to prevent other workers from corrupting them.
checkpoint_dir = '/tmp/train_logs' if hvd.rank() == 0 else None

# The MonitoredTrainingSession takes care of session initialization,
# restoring from a checkpoint, saving to a checkpoint, and closing when done
# or an error occurs.
with tf.train.MonitoredTrainingSession(checkpoint_dir=checkpoint_dir,
                                       config=config,
                                       hooks=hooks) as mon_sess:
    while not mon_sess.should_stop():
        # Perform synchronous training.
        mon_sess.run(train_op)
```

Code after migration:

```
# The following libraries are already automatically imported by the migration tool.
import tensorflow as tf
from npu_bridge.npu_init import *

# In this sample, the HCCL group management API is called. Therefore, HCCL needs to be initialized
# manually when a new session is started.
npu_int = npu_ops.initialize_system()
npu_shutdown = npu_ops.shutdown_system()
config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF
init_sess = tf.Session(config=config)
init_sess.run(npu_int)

# Pin GPU to be used to process local rank (one GPU per process)
config.gpu_options.visible_device_list = str(get_local_rank_id()) # Editing completed by the migration tool.

# Build model...
loss = ...
opt = tf.train.AdagradOptimizer(0.01 * get_rank_size()) # Editing completed by the migration tool.

# Add NPU Distributed Optimizer
opt = NPUDistributedOptimizer(opt) # Editing completed by the migration tool.

# Add hook to broadcast variables from rank 0 to all other processes during
# initialization.
# hooks = [hvd.BroadcastGlobalVariablesHook(0)] # Editing completed by the migration tool.

# Manual editing: Call the collective communication API broadcast to broadcast variables in sess.run mode.
input = tf.trainable_variables()
bcast_global_variables_op = hccl_ops.broadcast(input, 0)

# Make training operation
train_op = opt.minimize(loss)

# Save checkpoints only on worker 0 to prevent other workers from corrupting them.
checkpoint_dir = '/tmp/train_logs' if get_rank_id() == 0 else None # Editing completed by the migration
tool.

# The MonitoredTrainingSession takes care of session initialization,
# restoring from a checkpoint, saving to a checkpoint, and closing when done
# or an error occurs.
with tf.train.MonitoredTrainingSession(checkpoint_dir=checkpoint_dir,
                                       config=config,
                                       hooks=hooks) as mon_sess:
    # Manually broadcast the variables.
    mon_sess.run(bcast_global_variables_op)
    while not mon_sess.should_stop():
        # Perform synchronous training.
        mon_sess.run(train_op)
```

```
# Manual editing: Call shutdown_system and close the session after the training is complete.  
init_sess.run(npu_shutdown)  
init_sess.close()
```

5 Model Training

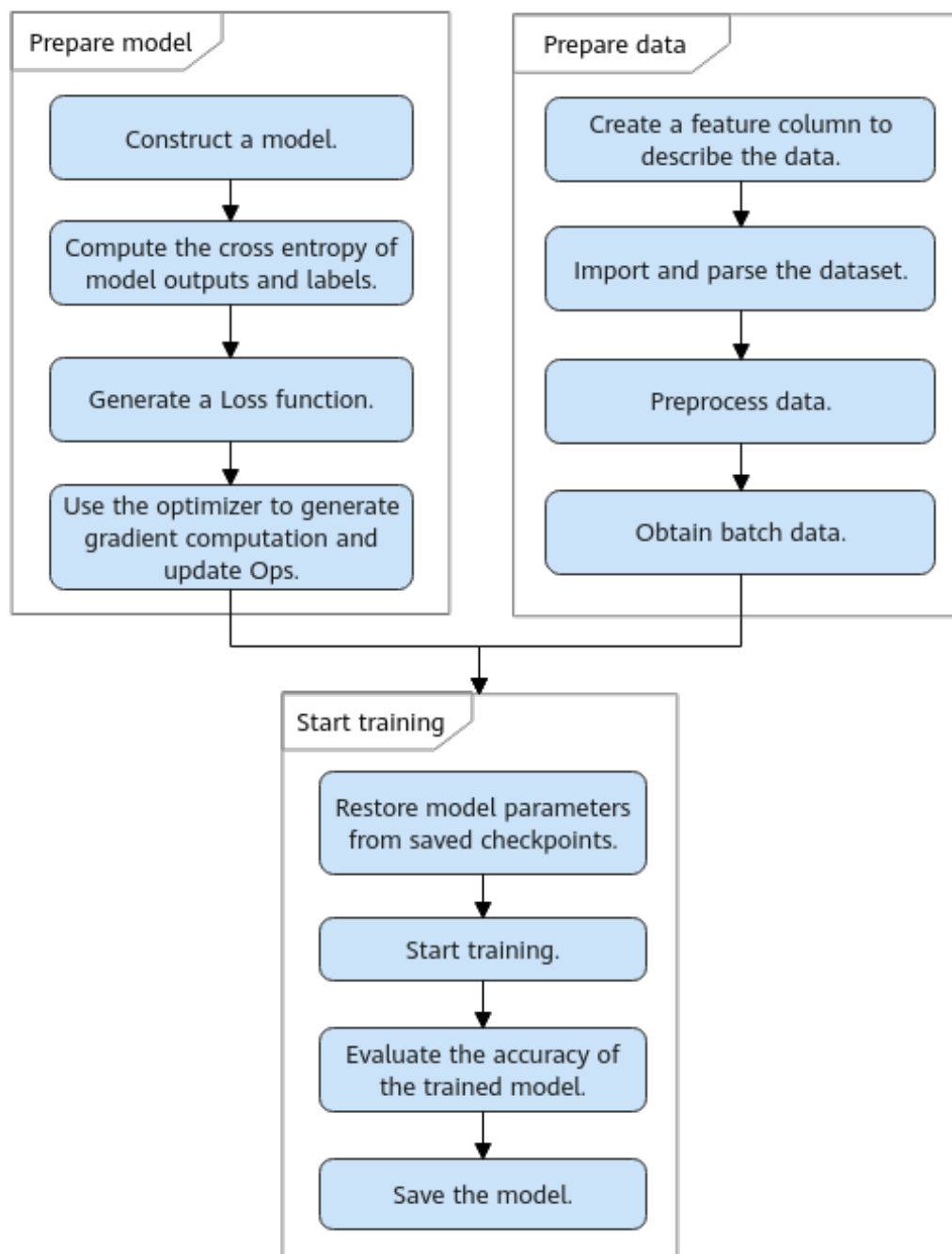
[5.1 Training Service Workflow](#)

[5.2 Training in a Bare Metal Environment](#)

5.1 Training Service Workflow

Typical Training Service Workflow

A typical training service workflow usually includes the model preparation, data preparation, training execution, and model storage phases.

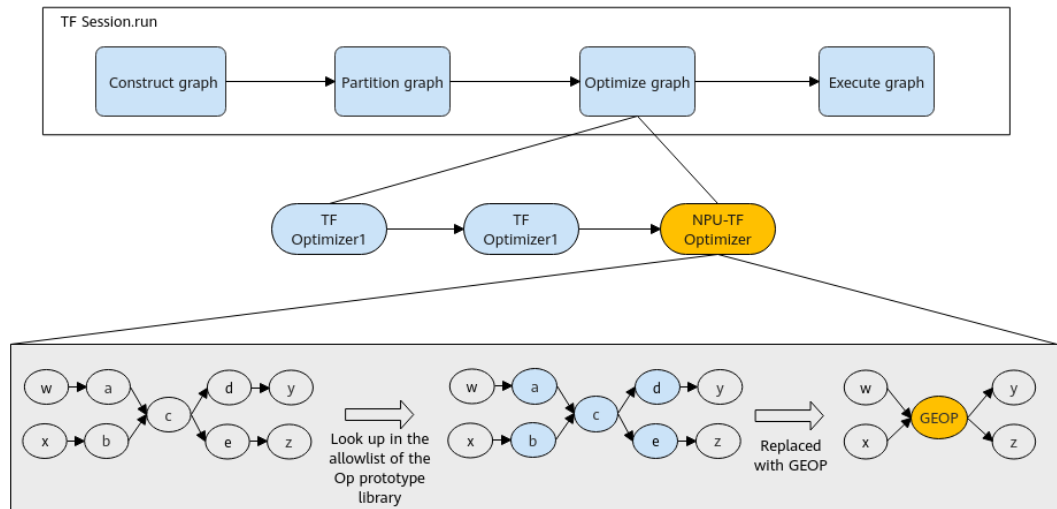


- Prepare a model: Use the customized expressions of the NN framework to construct a model and associated loss functions, and specify an appropriate optimizer to define the gradient computation and tuning algorithms.
- Prepare data: Import and parse data files and preprocess the data. The NN framework encapsulates common data preprocessing and enhancement operations and allows you to read and preprocess data using third-party libraries or in a user-defined manner.
- Train the model: Initialize your model based on the network structure defined in the model preparation phase and restore the pre-trained model from the model file. Read the training data and start the training iterations. The model accuracy is evaluated along the training process. When the model accuracy

reaches a certain threshold, the training is stopped and the trained model is saved.

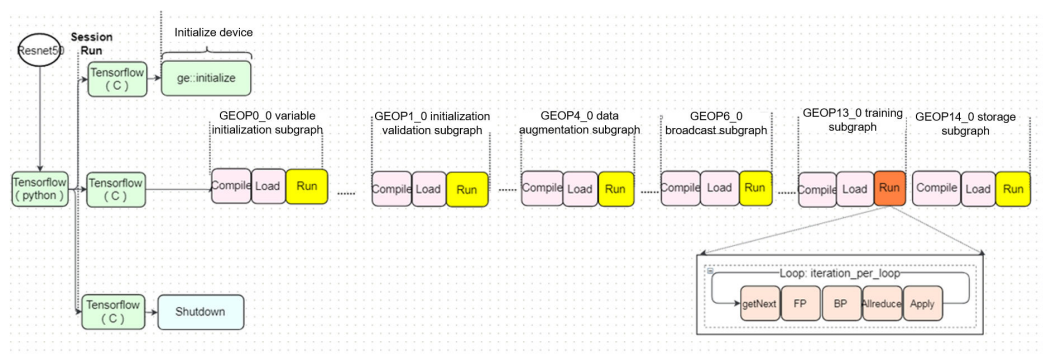
Training Service Data Flow

TF Adapter registers the NPU-TF Optimizer to partition the computational graph at the TensorFlow backend. The operators that can be offloaded to the device side are fused into one or more GEOP structures. Attr reflects all TensorFlow operators in the graph.



The data flow diagram of the training service contains a range of GEOP subgraphs:

- Data processing subgraph, including data reading and data augmentation. When both training and evaluation are performed, two data processing subgraphs are generated to process the training dataset and evaluation dataset, respectively.
- Model initialization subgraph, including variable initialization, variable verification, and variable broadcast (optional in distributed scenarios).
- Model training subgraph, whose input comes from the output of the training data processing subgraph. It performs forward propagation based on the network structure, calculates the loss, calculates the gradient, and updates the model parameters based on the optimization algorithm. It also saves the trained model and outputs logs in the training process. All the preceding steps are encapsulated in a loop body and are repeated iteratively according to the training configuration.
- Model evaluation subgraph, whose input comes from the output of the data processing subgraph. It performs forward propagation based on the network structure and calculates the model accuracy based on the defined algorithm.



5.2 Training in a Bare Metal Environment

Overview

This section describes how to execute a TensorFlow training job in a bare-metal environment. You are welcome to visit [Ascend ModelZoo](#) for more samples that are ready to run on Ascend AI Processors. The following uses the single-device and two-device training scenarios as examples to describe the general training procedure on Ascend AI Processors.

NOTICE

One device corresponds to one training process. It is not supported to run multiple training processes on a single device.

Prerequisite

- You have prepared a bare metal server environment.
- You have prepared a dataset.
- Your TensorFlow training script have been migrated.

Single-Device Training

Step 1 Prepare a single-device resource configuration file. If your training script does not involve calls to collective communication APIs, skip this step.

Assume that the configuration file is named **rank_table_1p.json**. The following provides a template of the file content.

```
{
  "server_count": "1",
  "server_list": [
    {
      "device": [
        {
          "device_id": "0",
          "device_ip": "192.168.1.8",
          "rank_id": "0"
        }
      ],
      "server_id": "10.0.0.10"
    }
  ]
}
```



```
}  
],  
"status":"completed",  
"version":"1.0"  
}
```

For details about the configuration file, see [10.4 Device Resource Configuration File Templates](#).

Step 2 Configure the environment variables required for starting the training process.

```
# Set the environment variables for the installation paths of the training component dependencies as follows:  
# Method 1: Install Ascend-CANN-Toolkit for training on an Ascend AI device in the development environment.  
export install_path=/home/HwHiAiUser/Ascend/ascend-toolkit/latest# Ascend-CANN-Toolkit installation path. Replace it as required.  
# Method 2: Install Ascend-CANN-NNAE for training on an Ascend AI device.  
export install_path=/home/HwHiAiUser/Ascend/nnae/latest# Ascend-CANN-NNAE installation path. Replace it as required.  
# Driver dependency  
export LD_LIBRARY_PATH=/usr/local/Ascend/driver/lib64/common:/usr/local/Ascend/driver/lib64/driver:$LD_LIBRARY_PATH # Required only in the containerized training scenario  
# Fwkacllib dependency  
export PATH=${install_path}/fwkacllib/ccec_compiler/bin:${install_path}/fwkacllib/bin:$PATH  
export LD_LIBRARY_PATH=${install_path}/fwkacllib/lib64:$LD_LIBRARY_PATH  
export PYTHONPATH=${install_path}/fwkacllib/python/site-packages:$PYTHONPATH  
# TFPlugin dependency  
export PYTHONPATH=${install_path}/tfplugin/python/site-packages:$PYTHONPATH  
export PYTHONPATH=/home/HwHiAiUser/Ascend/tfplugin/latest/tfplugin/python/site-packages:$PYTHONPATH # Replace the TFPlugin installation path with the actual one.  
# OPP dependency  
export ASCEND_OPP_PATH=${install_path}/opp  
# AI CPU dependency  
export ASCEND_AICPU_PATH=${install_path}  
# Script directory  
export PYTHONPATH=/home/test:$PYTHONPATH  
  
export JOB_ID=10086  
export ASCEND_DEVICE_ID=0  
export RANK_ID=0  
export RANK_SIZE=1  
export RANK_TABLE_FILE=/home/test/rank_table_1p.json // Unnecessary if your training script does not involve calls to collective communication APIs.
```

Step 3 Run the training script to start the training process.

python3.7 /home/test/xxx.py

NOTE

The preceding lists only the environment variables necessary for starting a training process. For details about all available environment variables, see [10.3 Environment Variables](#).

If GCC in the training environment (such as CentOS, Debian, and BCLinux) needs to be upgraded, add **\${install_path}/lib64** (replace **{install_path}** with the GCC installation path) to this variable. For details, see [Step 5](#).

----End

Multi-Device Training

For distributed training on multiple devices, you need to start the training processes in sequence. In the following distributed training example, two devices are used.

Step 1 Prepare a 2-device resource configuration file. Assume that the file is named **rank_table_2p.json**. The following provides a template of the file content.

```
{
  "server_count": "1",
  "server_list":
  [
    {
      "device": [
        {
          "device_id": "0",
          "device_ip": "192.168.1.8",
          "rank_id": "0"
        },
        {
          "device_id": "4",
          "device_ip": "192.168.1.9", // The two devices must be in the same network segment.
          "rank_id": "1"
        }
      ],
      "server_id": "10.0.0.10"
    }
  ],
  "status": "completed",
  "version": "1.0"
}
```

For details about the configuration file, see [10.4 Device Resource Configuration File Templates](#).

Step 2 Start the training processes in different shell windows.

Start training process 0.

```
...
Configure the dependency components. This can be done by following the same process as above.
...
export PYTHONPATH=/home/test:$PYTHONPATH
export JOB_ID=10086
export ASCEND_DEVICE_ID=0
export RANK_ID=0
export RANK_SIZE=2
export RANK_TABLE_FILE=/home/test/rank_table_2p.json
python3.7 /home/test/xxx.py
```

Start training process 1.

```
...
Configure the dependency components. This can be done by following the same process as above.
...
export PYTHONPATH=/home/test:$PYTHONPATH
export JOB_ID=10086
export ASCEND_DEVICE_ID=4
export RANK_ID=1
export RANK_SIZE=2
export RANK_TABLE_FILE=/home/test/rank_table_2p.json
python3.7 /home/test/xxx.py
```

NOTE

Alternatively, you can create a startup script to start training processes by using a loop.
Click [here](#) for more information.

----End

Viewing Training Result

Step 1 Check whether error logs are generated during system runtime. Pay attention to the following important phases:

- Figure 5-1** Training process started

[illegible]

- Figure 5-2** A training iteration started

```

INFO:tensorflow:global_step = 87
INFO:tensorflow:ent_loss = 0.42675993, global_step = 87, reg_loss = 0.1262866, top1 = 1.0, top5 = 1.0, total_loss = 0.54701954 (0.851 sec)
INFO:tensorflow:tf.nn.CheckpointSaveHook and...
2020-02-27 15:49:09.546329: I tensorflow/contrib/offline_train/kernels/gpu_nn.cc:436] [GP] Begin G6p0::Computeskys, kernel_name=G6p4_0, num_inputs=8, num_outputs=1
2020-02-27 15:49:09.546352: I tensorflow/contrib/offline_train/kernels/gpu_nn.cc:418] [GP] the graph 4 no need to rebuild
2020-02-27 15:49:09.546351: I tensorflow/contrib/offline_train/kernels/gpu_nn.cc:618] [GP] Call ge session RunGraphXxx, kernel_name=G6p4_0, ret=0
2020-02-27 15:49:09.546361: I tensorflow/contrib/offline_train/kernels/gpu_nn.cc:613] [GP] End ge session RunGraphXxx, kernel_name=G6p4_0, ret=0
2020-02-27 15:49:09.547148: I tensorflow/contrib/offline_train/kernels/gpu_nn.cc:660] [GP] RunGraphXxx callback, status=0, kernel_name=G6p4_0

```

- Figure 5-3 All training iterations completed**

```

2020-02-27 16:41:51.51729: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_18
2020-02-27 16:41:51.51769: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_17
2020-02-27 16:41:51.51807: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_16
2020-02-27 16:41:51.51845: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_15
2020-02-27 16:41:51.51884: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_14
2020-02-27 16:41:51.51938: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_13
2020-02-27 16:41:51.52006: tensorflow/contr/kernels/data_host_queue_deferred.cc:125: (GEP) Gpu data host queue ready ut.
2020-02-27 16:41:51.52006: tensorflow/contr/kernels/data_host_queue_deferred.cc:125: End training.
2020-02-27 16:41:52.55848: tensorflow/contr/kernels/data_host_queue_deferred.cc:125: The client closes the iterator.
2020-02-27 16:41:52.55886: tensorflow/contr/kernels/data_host_queue_deferred.cc:125: dclose
2020-02-27 16:41:52.55924: tensorflow/contr/kernels/data_host_queue_deferred.cc:125: The host queue's iterator is released.
2020-02-27 16:41:52.55943: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_12
2020-02-27 16:41:52.55959: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_11
2020-02-27 16:41:52.56018: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_10
2020-02-27 16:41:52.56186: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_9
2020-02-27 16:41:52.56208: tensorflow/contr/Offline_train/Kernels/gpu_npu.cc:331: (GEP) Gpu Finalize success, gpu_num_8

```

- Figure 5-4** Model saved successfully

```
INFO:tensorflow:SavedModel written to: ./train url/model/saved_model.pb
INFO:tensorflow:The model is exported successfully.
```

- **model:** stores checkpoint files and model files. Whether this directory is generated depends on the script implementation.
- **kernel_meta:** stores the .o and .json files of the operator. The files can be used to locate GE and FE errors. By default, the files are deleted. You can set **op debug level** to **3** in the training script to retain the .o and .json files.

----End

If the script execution fails, analyze and locate the fault based on the following logs:

Host log: **\$HOME/ascend/log/plog/plog_*.log**. Replace **\$HOME** with the root directory of the user on the host.

Device log: **\$HOME/ascend/log/device-id/device-id *.log**

Log format:

[Level] ModuleName(PID,PName):DateTimeMS LogContent

NOTE

For more information, see [Log Reference](#).

You can identify the error module and determine the cause by using ERROR-level log messages.

Figure 5-5 Error log example

```
[ERROR] TEFUSION(51553,python3.6):2020-02-27-17:14:18.955.503 [tensor_engine/te_fusion/fusion_op.cc:3904]PrintPyException Traceback (most recent call last): File "/usr/local/python3/lib/python3.6/site-packages/topi/cce/util.py", line 127, in in_wrapper return func(*args, **kwargs) File "/usr/local/Huawei/runtime/ops/op_impl/built-in/ai_core/tbe/impl/l2_loss.py", line 51, in l2_loss util.check_shape_rule(shape) UnboundLocalError: local variable 'shape' referenced before assignment
```

Table 5-1 Fault locating techniques

Module Name	Error Process	Solution
System error	Environment and version mismatch	Check the version mapping and system installation.
Data Preprocessing	Data preprocessing	N/A
GE	GE graph build or verification	Specific error causes are provided for verification errors. You only need to modify the network script as prompted.
FE	Operator selection or operator fusion	N/A
TEFUSION	Operator build	N/A
RUNTIME	Initialization or graph execution failure due to an environment exception	If initialization fails, check the environment configuration and whether the environment is preempted.

6 Performance Tuning

6.1 Network Throughput Analysis

6.2 Network-wide Performance Optimization

6.3 Operator Performance Optimization

6.1 Network Throughput Analysis

You can add the following performance data printing statement to your training script.

```
class LogSessionRunHook(tf.train.SessionRunHook):
    def __init__(self, config, warmup_steps=5):
        # def __init__(self, global_batch_size, num_records, display_every=10, logger=None):
        self.global_batch_size = config['global_batch_size']
        self.iterations_per_loop = config['iterations_per_loop']
        self.warmup_steps = warmup_steps
        self.iter_times = []
        self.num_records = config['num_training_samples']
        self.display_every = config['display_every']
        self.logger = get_logger(config['log_name'], config['log_dir'])
        rank0log(self.logger, 'PY' + str(sys.version) + 'TF' + str(tf.__version__))

    def after_create_session(self, session, coord):
        rank0log(self.logger, 'Step Epoch Speed Loss FinLoss LR')
        self.elapsed_secs = 0.
        self.count = 0

    def before_run(self, run_context):
        self.t0 = time.time()
        return tf.train.SessionRunArgs(
            fetches=[tf.train.get_global_step(), 'loss:0', 'total_loss:0', 'learning_rate:0'])
#            'loss:0', 'loss:0', 'learning_rate:0'])

    def after_run(self, run_context, run_values):
        batch_time = time.time() - self.t0
        self.iter_times.append(batch_time)
        self.elapsed_secs += batch_time
        self.count += 1
        global_step, loss, total_loss, lr = run_values.results
        if global_step == 1 or global_step % self.display_every == 0:
            dt = self.elapsed_secs / self.count
            img_per_sec = self.global_batch_size * self.iterations_per_loop / dt
            epoch = global_step * self.global_batch_size / self.num_records
            self.logger.info('step:%6i epoch:%5.1f FPS:%7.1f loss:%6.3f total_loss:%6.3f lr:%7.5f' %
                            (global_step, epoch, img_per_sec, loss, total_loss, lr))
            self.elapsed_secs = 0.
            self.count = 0

    def get_average_speed(self):
        avg_time = np.mean(self.iter_times[self.warmup_steps:])
        speed = self.global_batch_size / avg_time
        return speed
```

After the training script is executed, you can run the following command to check the logs to see whether the throughput meets the requirements:

gerp FPS log.txt

```
[root@bms-aiserver-125 mains]# grep FPS log.txt
step: 1000 epoch: 0.4 FPS: 5838.8 loss: 6.866 total_loss: 8.203 lr:0.00314
step: 2000 epoch: 0.8 FPS:12015.4 loss: 6.590 total_loss: 7.918 lr:0.01235
step: 3000 epoch: 1.2 FPS:19141.7 loss: 6.335 total_loss: 7.643 lr:0.02706
step: 4000 epoch: 1.6 FPS:19401.8 loss: 5.891 total_loss: 7.165 lr:0.04635
step: 5000 epoch: 2.0 FPS:18856.9 loss: 5.737 total_loss: 6.969 lr:0.06900
step: 6000 epoch: 2.4 FPS:19408.8 loss: 5.399 total_loss: 6.587 lr:0.09360
step: 7000 epoch: 2.8 FPS:19587.2 loss: 5.136 total_loss: 6.291 lr:0.11860
step: 8000 epoch: 3.2 FPS:19020.2 loss: 4.659 total_loss: 5.795 lr:0.14243
step: 9000 epoch: 3.6 FPS:19633.7 loss: 4.847 total_loss: 5.983 lr:0.16360
step: 10000 epoch: 4.0 FPS:19095.4 loss: 4.758 total_loss: 5.909 lr:0.18078
step: 11000 epoch: 4.4 FPS:19283.6 loss: 4.395 total_loss: 5.574 lr:0.19290
step: 12000 epoch: 4.8 FPS:19233.6 loss: 4.124 total_loss: 5.334 lr:0.19918
step: 13000 epoch: 5.2 FPS:19367.1 loss: 4.201 total_loss: 5.444 lr:0.19999
step: 14000 epoch: 5.6 FPS:19463.7 loss: 3.817 total_loss: 5.090 lr:0.19994
step: 15000 epoch: 6.0 FPS:19228.0 loss: 3.703 total_loss: 5.004 lr:0.19984
```

Set the number of printing steps to **log_every_n_steps=1** or **log_every_n_steps=10** as needed.

If the throughput is not as expected, improve the network performance by referring to [6.2 Network-wide Performance Optimization](#). If the throughput is still unsatisfactory, improve the performance of specific operators by referring to [6.3 Operator Performance Optimization](#).

6.2 Network-wide Performance Optimization

6.2.1 Iteration Offloading

Overview

Iterations_per_loop is the number of iterations per training loop performed on the device side per **sess.run()** call. Training is performed according to the specified number of iterations per loop (**iterations_per_loop**) on the device side and then the result is returned to the host. This parameter can save unnecessary interactions between the host and device and reduce the training time consumption. Note the following:

- The default value of **iterations_per_loop** is **1**, and the total number of training iterations must be an integer multiple of **iterations_per_loop**.
- If the value of **iterations_per_loop** is greater than 1, the value of **save_checkpoints_steps** must be a positive integer multiple of **iterations_per_loop**; otherwise, checkpoint data is not saved as defined by **save_checkpoints_steps**. If the value of **iterations_per_loop** is greater than 1, data is not saved as defined by **save_summary_steps** and **log_step_count_steps**. For details, see [9.2 Log and Summary Operators](#).
- In mixed computing mode (**mix_compile_mode** is set to **True**), this parameter must be set to 1.
- The getNext operator can be scheduled to the device side only when **enable_data_pre_proc** is enabled and iterator **tf.data.make_initializable_iterator()** is used. In this case, the **iterations_per_loop** parameter takes effect only when it is set to a value greater than 1.

When **enable_data_pre_proc** is disabled or other data preprocessing iterators, such as **tf.data.make_one_shot_iterator()**, are used, the getNext operator will not be scheduled to the device side. Therefore, **iterations_per_loop** does not take effect when it is set to a value greater than 1.

In **Estimator** mode, if the return value of **input_fn** is dataset, **tf.data.make_initializable_iterator()** is implicitly called in internal processing of **Estimator**.

- During network commissioning, you are advised to set **iterations_per_loop** to **1** to facilitate log printing every iteration. After the network is set up correctly, you can set the **iterations_per_loop** parameter to shorten the training time.

Iteration Offloading with Estimator

In **Estimator** mode, configure **iterations_per_loop** in **NPURunConfig** as follows.

```
from npu_bridge.npu_init import *

session_config=tf.ConfigProto()
config = NPURunConfig(session_config=session_config, iterations_per_loop=10)
```

Iteration Offloading with sess.run()

In **sess.run** mode, configure the **iterations_per_loop** parameter by using **set_iteration_per_loop** and change the number of **sess.run()** calls to the original number of calls divided by the value of **iterations_per_loop**. The following shows how to configure **iterations_per_loop**.

```
from __future__ import print_function
import input_data
from npu_bridge.npu_init import *

mnist = input_data.read_data_sets("/test/", one_hot=True)

import tensorflow as tf

# Set the model.
# Set the learning rate.
learning_rate = 0.01
# Set the number of training iterations.
training_epochs = 10
# Set the batch size.
batch_size = 100
# Set the number of iterations after which the loss is displayed once.
display_step = 1

x = tf.placeholder(tf.float32, [None, 784])
y = tf.placeholder(tf.float32, [None, 10])

# Set the model parameters.
W = tf.Variable(tf.zeros([784, 10]))
b = tf.Variable(tf.zeros([10]))

# Build the model.
pred = tf.nn.softmax(tf.matmul(x, W) + b)

# Define the loss function: cross entropy.
cost = tf.reduce_mean(-tf.reduce_sum(y*tf.log(pred), reduction_indices=1))

# Optimize the gradient descent.
optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)

# Initialize all variables.
```

```
init = tf.global_variables_initializer()

config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True # Perform training on the Ascend AI Processor.
custom_op.parameter_map["mix_compile_mode"].b = False # Disable the mixed computing function. Set
this parameter as required. This function is disabled by default.
custom_op.parameter_map["iterations_per_loop"].i = 10 # Determine whether the training iteration is
offloaded. Must be equal to iterations_per_loop of set_iteration_per_loop.
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

# Train the model.
with tf.Session(config=config) as sess:
    sess.run(init)
    # Set the number of iterations per loop to 10 in sess.run mode.
    train_op = util.set_iteration_per_loop(sess, optimizer, 10)

    for epoch in range(training_epochs):
        avg_cost = 0
        total_batch = int(mnist.train.num_examples / batch_size)

        for i in range(total_batch):
            batch_xs, batch_ys = mnist.train.next_batch(batch_size)
            _, c = sess.run([train_op, cost], feed_dict={x: batch_xs, y: batch_ys})

            avg_cost += c / total_batch
```

The preceding API involves graph modification. If a graph cannot be modified (for example, the graph is frozen or a session is created using **tf.train.Supervisor**), you cannot use the **set_iteration_per_loop** API to set the loops and iterations per loop. In this case, use the **create_iteration_per_loop_var** and **load_iteration_per_loop_var** APIs to set the number of iterations per loop. The following is an example.

```
from __future__ import print_function
import input_data
from npu_bridge.npu_init import *

mnist = input_data.read_data_sets("/test/", one_hot=True)

import tensorflow as tf

# Set the model.
# Set the learning rate.
learning_rate = 0.01
# Set the number of training iterations.
training_epochs = 10
# Set the batch size.
batch_size = 100
# Set the number of iterations after which the loss is displayed once.
display_step = 1

x = tf.placeholder(tf.float32, [None, 784])
y = tf.placeholder(tf.float32, [None, 10])

# Set the model parameters.
W = tf.Variable(tf.zeros([784, 10]))
b = tf.Variable(tf.zeros([10]))

# Build the model.
pred = tf.nn.softmax(tf.matmul(x, W) + b)

# Define the loss function: cross entropy.
cost = tf.reduce_mean(-tf.reduce_sum(y*tf.log(pred), reduction_indices=1))

# Optimize the gradient descent.
optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)
```



```
# Initialize all variables.
init = tf.global_variables_initializer()

config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True # Perform training on the Ascend AI Processor.
custom_op.parameter_map["mix_compile_mode"].b = False # Disable the mixed computing function. Set
this parameter as required. This function is disabled by default.
custom_op.parameter_map["iterations_per_loop"].i = 10 # Used for functional validation. Must be equal
to iterations_per_loop of set_iteration_per_loop.
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

# Train the model.
with tf.Session(config=config) as sess:
    sess.run(init)
    # Set the number of iterations per loop to 10 in sess.run mode.
    iteration = util.IterationPerLoop()
    train_op = iteration.create_iteration_per_loop_var(optimizer) # Modify the graph.
    tf.train.Supervisor(logdir="/home/xxxx",init_op=init) # Freeze the graph.
    iteration.load_iteration_per_loop_var(sess, 10) # Set the number of iterations per loop.

    for epoch in range(training_epochs):
        avg_cost = 0
        total_batch = int(mnist.train.num_examples / batch_size)

        for i in range(total_batch):
            batch_xs, batch_ys = mnist.train.next_batch(batch_size)
            _, c = sess.run([train_op, cost], feed_dict={x: batch_xs, y: batch_ys})

            avg_cost += c / total_batch
```

Setting iterations_per_loop with Keras

On the Ascend platform, you can directly use the native Keras API for training. However, the number of iterations per training loop on Ascend AI Processor is fixed at 1 in each **sess.run()** call. To reduce the number of interactions between the host and devices and shorten the training duration, you need to use the **model_to_npu_estimator** API to convert the model constructed by using Keras into an **NPUEstimator** object. Besides, you need to specify the number of iterations per training loop on Ascend AI Processor per **sess.run()** call by using the **iterations_per_loop** parameter in **NPURunConfig**.

Original TensorFlow code:

```
from keras.layers import Input, Dense
from keras.models import Model

# This returns a tensor
inputs = Input(shape=(224, 224, 3))

# This creates a model that includes
# the Input layer and three Dense layers
keras_model = ResNet50(input_tensor=inputs, weights=None, include_top=True)
keras_model.compile(optimizer='rmsprop', loss='sparse_categorical_crossentropy')

keras_model.fit_generator(
    train_generator,
    steps_per_epoch=100,
    epochs=10)
```

Code after migration:

```
from npu_bridge.npu_init import *
```

```
run_config = NPURunConfig(save_checkpoints_steps=2,
                           model_dir=model_path,
                           iterations_per_loop=10)
# Convert the model constructed by using Keras to an NPUEstimator object.
est_resnet = keras_to_npu.model_to_npu_estimator(keras_model=keras_model, config=run_config)
# Perform training.
est_resnet.train(input_fn=lambda: input_fn(), max_steps=1000)
```

In addition, you need to migrate the Keras data preprocessing part to **input_fn** in **NPUEstimator**. The following is an example. In the following example, Keras reads image data from the folder, automatically labels the data, performs data augmentation operations such as data resize, normalization, and horizontal flip, and finally outputs the data. In **Estimator** mode, data is preprocessed in the same way as reading data from the file list. The difference is that the file name list needs to be read in advance and each image needs to be labeled to output the label list. The data is output after the same data enhancement operations such as normalization, resize, and horizontal flip.

Original TensorFlow code:

```
# Keras reads images from the folder.
train_datagen = ImageDataGenerator(rescale=1./255,
                                    horizontal_flip=True)

train_generator = train_datagen.flow_from_directory('data/',
                                                    target_size=(224, 224, 3),
                                                    batch_size=32,
                                                    class_mode='sparse')
```

Code after migration:

```
# The function is used to read the image files corresponding to the file names and resize the image files to
a unified size.
def _parse_function(filename, label):
    image = tf.read_file(filename)
    image = tf.image.decode_image(image)
    image = image / 255.0
    image = tf.image.resize_images(image, [224, 224, 3])
    image = tf.image.random_flip_left_right(image)
    return image, label

def input_fn():
    # List of image files. The image list needs to be generated by yourself.
    filenames = tf.constant(["/data/image1.jpg", "/data/image2.jpg", ...])
    # label[i] is the label of the filenames[i] image. The label list needs to be generated by yourself.
    labels = tf.constant([0, 5, ...])
    # Now an element in the dataset is (filename, label).
    dataset = tf.data.Dataset.from_tensor_slices((filenames, labels)).repeat(10)
    # Now an element in the dataset is (image_resized, label).
    dataset = dataset.map(_parse_function)
    # Now an element in the dataset is (image_resized_batch, label_batch).
    dataset = dataset.shuffle().batch(32)
    return dataset
```

Note that the callback function of Keras cannot be used after being converted to an **NPUEstimator** object.

Checking Whether iterations_per_loop Takes Effect

Search for "Insert op success" in the host log file to check whether **iterations_per_loop** takes effect, as shown in [Figure 6-1](#).

Figure 6-1 Log message (`iterations_per_loop > 1`)

```
[root@localhost ~]# cat /var/log/npu/slog | grep -rn "Insert op success"
host-0/host-0_20200423104329152.log:15101:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.952.828 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:ge_global_step, type:Variable.
host-0/host-0_20200423104329152.log:15105:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.953.232 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:npu_runconfig/loop_cond, type:Variable.
host-0/host-0_20200423104329152.log:15107:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.953.389 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:npu_runconfig/one, type:Variable.
host-0/host-0_20200423104329152.log:15109:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.953.556 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:npu_runconfig/zero, type:Variable.
host-0/host-0_20200423104329152.log:15111:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.953.722 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:npu_runconfig/iterations_per_loop, type:Variable.
host-0/host-0_20200423104329152.log:15112:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.953.808 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:Node_Output_IteratorCtrl_StreamSwitch, type:StreamSwitch.
host-0/host-0_20200423104329152.log:15113:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.953.894 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:FlowCtrl_LoopCond_ASSIGN, type:Assign.
host-0/host-0_20200423104329152.log:15115:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.954.011 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:FlowCtrl_LoopCond_ASSIGNAdd, type:AssignAdd.
host-0/host-0_20200423104329152.log:15116:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.954.058 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:Node_Output_IteratorCtrl_StreamSwitch_StreamActive, type:StreamActive.
host-0/host-0_20200423104329152.log:15118:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.954.478 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:IteratorV2_IteratorCtrl_StreamSwitch, type:StreamSwitch.
host-0/host-0_20200423104329152.log:15119:[INFO] GE(26217,python3.7):2020-04-23-10:43:36.954.516 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
ess, name:IteratorV2_IteratorCtrl_StreamSwitch_StreamActive, type:StreamActive.
```

Figure 6-2 shows the related log information when the value of `iterations_per_loop` is set to 1 (the default value).

Figure 6-2 Log message (`iterations_per_loop = 1`)

```
[root@localhost ~]# cat /var/log/npu/slog | grep -rn "Insert op success"
host-0/host-0_20200423103535695.log:12241:[INFO] GE(6856,python3.7):2020-04-23-10:36:08.019.905 [framework/domi/graph/passes/flow_ctrl_pass.cc:141]InsertOp:Insert op succ
```

6.2.2 Data Preprocessing Performance

Balancing the Schedule of Data Preprocessing Operators

When the data volume is large, the data preprocessing performance might not meet the compute performance requirements of the downstream layers in a graph. In this case, you can balance the data preprocessing operators executed on the host and device sides to improve the training performance.

Determine whether a data preprocessing operator is scheduled to the device side as follows: From the bottom up, schedule the preprocessing operators that can run on the device side to the device side, stop at the operator that cannot run on the device side, and schedule the downstream preprocessing operators to the host.

Currently, the following data preprocessing operators can run on the device side: `map`, `batch`, and `map_and_batch`. Other preprocessing operators run only on the host.

The following is a schedule example.

Original TensorFlow code:

TFRecordDataset and shuffle cannot run on the device side. Therefore, only the `map` and `batch` operators are scheduled on the device side.

```
train_dataset=tf.contrib.data.TFRecordDataset("./train_new.tfrecords")
train_dataset=train_dataset.shuffle(1000)
train_dataset=train_dataset.map(parse_tf)
train_dataset=train_dataset.batch(batch_size)
```

Code after migration:

```
train_dataset=tf.contrib.data.TFRecordDataset("./train_new.tfrecords")
train_dataset=train_dataset.shuffle(1000)
train_dataset=train_dataset.map(parse_tf)
train_dataset=train_dataset.batch(batch_size)
train_dataset = train_dataset.prefetch(buffer_size=buffer_size)
```

Insert a `prefetch` operator between the `map` and `batch` operators. Since the `prefetch` operator cannot run on the device side, all its downstream operators are scheduled to the host.

Binding Training Process to CPU

In the multi-device scenario, to evenly schedule the host CPU cores and further improve the training performance, you can bind each training process to specify host CPU cores. The following uses the 8-device scenario as an example.

1. Query the total number of host CPU cores, for example, 96.

```
[root@localhost ~]# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                 96
On-line CPU(s) list:   0-95
```

2. Calculate the number (n) of host CPU cores allocated to each training process.
$$n = \text{Total CPU cores} / 8 = 12$$
3. Modify the training process startup script. Before starting the training script, use **taskset -c** to bind the processes to the specified host CPU cores, for example:

Device 0:

```
taskset -c 0-11 python3.7 /home/test/xxx.py /
```

Device 7:

```
taskset -c 84-95 python3.7 /home/test/xxx.py /
```

NOTE

For details about the startup script, see [5.2 Training in a Bare Metal Environment](#).

6.2.3 Distributed Performance Improvement

Background

In the distributed training scenario, gradient aggregation is performed after gradients between devices are calculated. Gradient data is generated in order and does not change after being generated. To improve training performance, the gradient parameter data may be segmented. Gradient aggregation may be immediately started after gradient data of a segment is generated, so that some gradient parameter data is aggregated and forward and backward time is executed in parallel.

The default segmentation policy is two segments with the first taking up 96.54% of the data volume, and the second segment taking up 3.46% of the data volume (In some cases, the data is not segmented). This segmentation policy may not be applicable to other networks due to the data volume and calculation time differences of different network gradients. You can adjust the distributed gradient segmentation policy by referring to this section to improve the training performance in distributed scenarios.

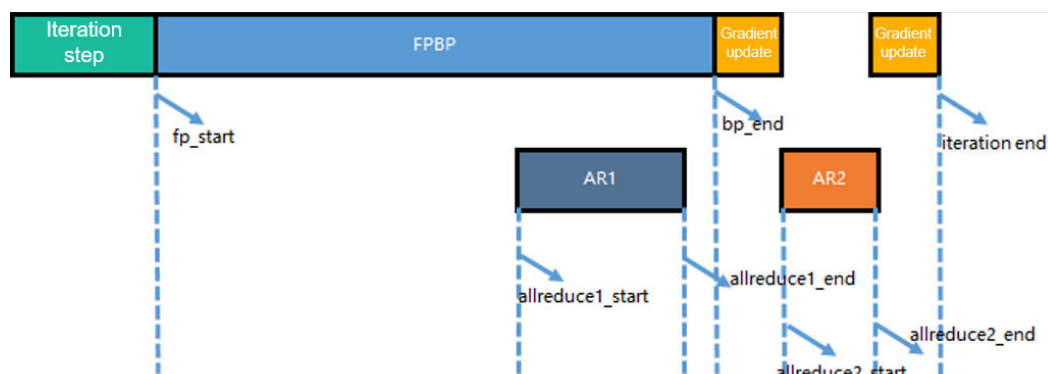
Determining Gradient Segmentation Policy

You need to use the Profiling tool to analyze the iteration traces of the training process to determine the gradient segmentation policy and improve the training performance in distributed scenarios.

NOTE

For details, see *Profiling Tool Instructions*.

Iteration tracing is to trace the software status of a training job and the Ascend AI Software Stack, which can be used to analyze the performance of a training job. If the default two-segment gradient segmentation policy is applied, the following iteration traces of a training job are printed to describe the job execution status in an iteration: **fp_start**, **bp_end**, **allreduce1_start**, **allreduce1_end**, **allreduce2_start**, **allreduce2_end**, and **Iteration_end** in the training job.



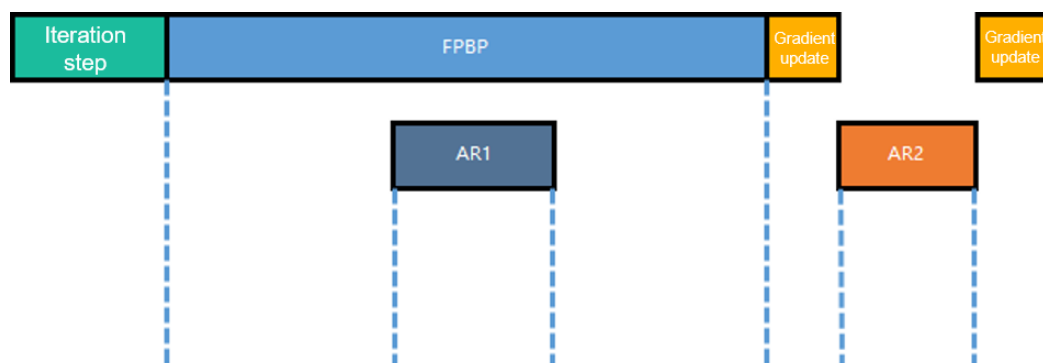
An optimal gradient data segmentation policy meets the following rules:

- AR1 is hidden within the BPFP period.
- The time of the AR2 is as short as possible, so as to reduce the hangover caused by collective communication after the computation.

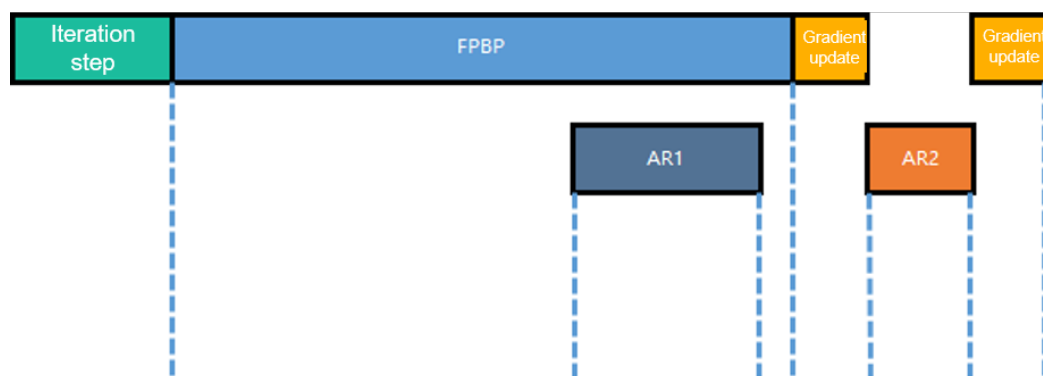
Based on the preceding segmentation rules, you can adjust the gradient segmentation policy to improve the training performance in distributed scenarios. The following uses two-segment gradient segmentation as an example to describe how to determine a gradient segmentation policy with three optimization scenarios.

[Optimization Scenario 1] Since AR1 starts earlier than AR2, the segmentation point can be moved backward to shorten the time of AR2.

For example, if the two gradient segments have the same data volume, the segmentation diagram is as follows.

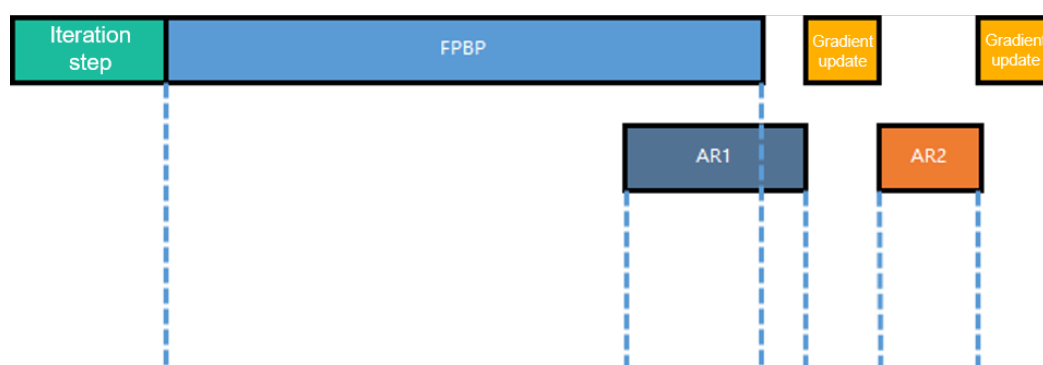


If the data volume of the first gradient segment is increased to 80%, the segmentation diagram is as follows.

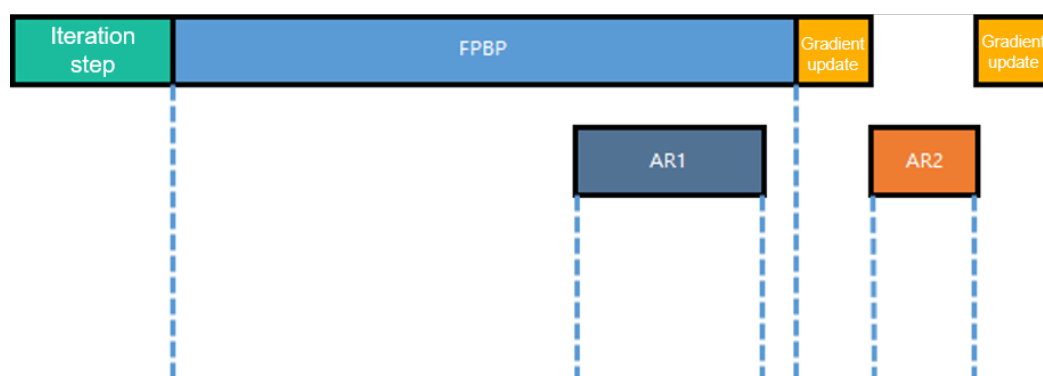


[Optimization Scenario 2] Since AR1 starts later than BFPF, the segmentation point can be moved forward to hide AR1 between BFPFs.

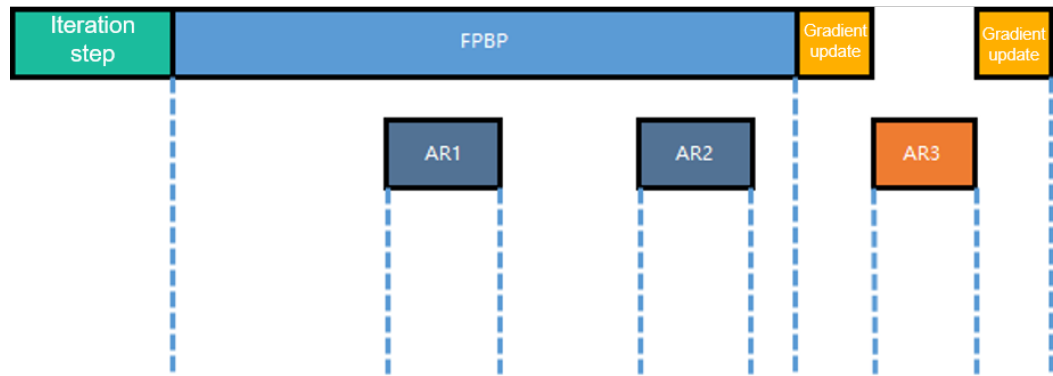
If the data volume of the first gradient segment is 90%, the segmentation diagram is as follows.



If the data volume of the first gradient segment is decreased to 80%, the segmentation diagram is as follows.



[Optimization Scenario 3] Since the BFPF data volume is large and the computation time is long, in the two-segment gradient segmentation scenario, if most gradient data is stored in AR1, the hangover time is long. For details, see optimization scenario 2. If most gradient data is stored in AR2, AR2 will take a long time. For details, see optimization scenario 1. However, there is still a relatively large amount of time in the BFPF to be utilized. In this case, more segmentation points may be added, to improve the parallelism.



Adjusting Gradient Segmentation Policy

You can call the gradient segmentation API in the training script to set the AllReduce segmentation and fusion policy in the backward propagation phase.

set_split_strategy_by_idx: sets the backward gradient segmentation policy in the collective communication group based on the gradient index ID.

```
from npu_bridge.npu_init import *
set_split_strategy_by_idx([20, 100, 159])
```

set_split_strategy_by_size: sets the backward gradient segmentation policy in the collective communication group by ratio.

```
from npu_bridge.npu_init import *
set_split_strategy_by_size([60, 20, 20])
```

Call either of the preceding APIs before the **allreduce** call (the collective communication API must be initialized). For example:

```
import tensorflow as tf
from npu_bridge.npu_init import *

npu_init = npu_ops.initialize_system()
npu_shutdown = npu_ops.shutdown_system()

config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
# Enable profiling iterative trace collection.
custom_op.parameter_map["profiling_mode"].b = True
custom_op.parameter_map["profiling_options"].s = tf.compat.as_bytes("training_trace")
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

with tf.Session(config=config) as sess:
    sess.run(npu_init)
    # Set the backward gradient segmentation policy.
    set_split_strategy_by_size([80, 20])
    # Perform AllReduce.
    # Perform training...
    sess.run(npu_shutdown)
```

6.2.4 Mixed Precision

Overview

Mixed precision is the combined use of the float16 and float32 data types in training deep neural networks, which reduces memory usage and access

frequency. Mixed-precision training makes it easier to deploy larger networks without compromising the network accuracy with float32. Currently, Ascend AI Processor supports the following training precision modes. Choose one as needed in the training script.

- **allow_fp32_to_fp16**: The original precision is preferentially retained. If an operator does not support the float32 data type, the float16 precision is used. Currently, the float32 type is not supported by convolution operators, such as Conv2D and DepthwiseConv2D. These operators are precision-insensitive and do not reduce the accuracy of the entire network. This is the default precision mode.
- **force_fp16**: If an operator supports both float16 and float32 data types, float16 is forcibly selected.
- **must_keep_origin_dtype**: The original precision is retained. In this mode, if a network contains a Conv2D operator, the training process will be interrupted when the input type of the original graph is float32 because the operator supports only the float16 type.
- **allow_mix_precision**: Mixed precision is allowed. For operators of the float32 data type on a network, the precision of some float32 operators can be automatically reduced to float16 based on the built-in optimization policy. In this way, the system performance is improved and the memory usage is reduced with little accuracy loss. Note that Ascend AI Processor supports only float32 to float16 casting. It is recommended that [6.2.5 Loss Scaling](#) be enabled after mixed precision is enabled to compensate for the accuracy loss caused by precision reduction. If the original script has implemented the manual mixed precision, for example, the cast operator has been explicitly called to convert the compute precision, mixed precision of Ascend AI Processor does not need to be enabled.

Setting the Precision Mode with Estimator

In **Estimator** mode, the precision mode is specified by **precision_mode** in **NPURunConfig**.

```
from npu_bridge.npu_init import *

npu_config=NPURunConfig(
    model_dir=FLAGS.model_dir,
    save_checkpoints_steps=FLAGS.save_checkpoints_steps,
    session_config=tf.ConfigProto(allow_soft_placement=True,log_device_placement=False),
    precision_mode="allow_mix_precision"
)
```

If **allow_mix_precision** is enabled, you can make adjustment based on the built-in optimization policy to specify the operators whose precision can be reduced and the operators whose precision cannot.

1. Go to **/opp/op_impl/built-in/ai_core/tbe/config/ascend910** in the OPP installation path.
2. Grant the write permission on the **aic-ascend910-ops-info.json** file.
`chmod u+w aic-ascend910-ops-info.json`
3. Modify or add the **precision_reduce** field of the corresponding operator in the **aic-ascend910-ops-info.json** file of the operator information library.

```
"precision_reduce":{
    "flag": "true"
}
```


- **true**: allows precision reduction to float16 for operators supporting float32.
- **false**: forbids precision reduction to float16 for operators supporting float32.
- If not specified, the current operator uses the same mixed precision processing as the upstream operator.

Setting Precision Mode with `sess.run()`

In `sess.run()` mode, set the precision mode by using the session configuration option **precision_mode**.

```
import tensorflow as tf
from npu_bridge.npu_init import *

config = tf.ConfigProto()

custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
custom_op.parameter_map["precision_mode"].s = tf.compat.as_bytes("allow_mix_precision")
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

with tf.Session(config=config) as sess:
    print(sess.run(cost))
```

The parameter description and configuration method are the same as those in the **Estimator** method.

6.2.5 Loss Scaling

Overview

Loss scaling is used to solve the underflow problem that occurs during the gradient calculation due to the small representation range of float16. The loss calculated in the forward pass is multiplied by the loss scale *S* to amplify the gradient during the backward gradient calculation. In the mixed precision training scenario on some networks, the loss scaling function needs to be enabled. Otherwise, the loss does not converge.

Using Loss Scaling

If the loss scaling function is used on the original network, you need to migrate **LossScaleOptimizer** to the **NPULossScaleOptimizer** or **NPUOptimizer** constructor. The following uses **NPULossScaleOptimizer** as an example.

- Static loss scaling: You can use a fixed loss scaling factor during mixed precision training.

When using the static loss scaling, before creating **NPULossScaleOptimizer**, instantiate a **FixedLossScaleManager** class to specify the loss scaling factor.

- Dynamic loss scaling: You can adjust the loss scaling factor based on the abnormal status of floating-point computation during mixed precision training.

When using the dynamic loss scaling, before creating

NPULossScaleOptimizer, instantiate a

ExponentialUpdateLossScaleManager class to dynamically specify the loss scaling factor.

In addition, when **NPULossScaleOptimizer** is used, set **is_distributed** to **True** to support the loss scaling function in the distributed training scenario.

Original TensorFlow code:

```
if FLAGS.use_fp16 and (FLAGS.bert_loss_scale not in [None, -1]):
    opt_tmp = opt
    if FLAGS.bert_loss_scale == 0:
        loss_scale_manager =
tf.contrib.mixed_precision.ExponentialUpdateLossScaleManager(init_loss_scale=2**32,
incr_every_n_steps=1000, decr_every_n_nan_or_inf=2, decr_ratio=0.5)
    elif FLAGS.bert_loss_scale >= 1:
        loss_scale_manager = tf.contrib.mixed_precision.FixedLossScaleManager(loss_scale=FLAGS.bert_loss_scale)
    else:
        raise ValueError("Invalid loss scale: %d" % FLAGS.bert_loss_scale)
    opt = tf.contrib.mixed_precision.LossScaleOptimizer(opt_tmp, loss_scale_manager)
```

Code after migration:

```
from npu_bridge.npu_init import *

if FLAGS.use_fp16 and (FLAGS.bert_loss_scale not in [None, -1]):
    opt_tmp = opt
    if FLAGS.bert_loss_scale == 0:
        loss_scale_manager = ExponentialUpdateLossScaleManager(init_loss_scale=2**32,
incr_every_n_steps=1000, decr_every_n_nan_or_inf=2, decr_ratio=0.5)
    elif FLAGS.bert_loss_scale >= 1:
        loss_scale_manager = FixedLossScaleManager(loss_scale=FLAGS.bert_loss_scale)
    else:
        raise ValueError("Invalid loss scale: %d" % FLAGS.bert_loss_scale)
    # Check whether the number of devices is greater than 1. If yes, perform distributed training.
    if ops_adapter.size() > 1:
        opt_tmp = NPUDistributedOptimizer(opt_tmp)
        opt = NPULossScaleOptimizer(opt_tmp, loss_scale_manager, is_distributed=True)
    else:
        opt = NPULossScaleOptimizer(opt_tmp, loss_scale_manager)
```

Updating the Global Step

After the loss scaling function is enabled, the step where the loss scaling overflow occurs needs to be discarded. For details, see the update step logic of the optimizer.

- In most cases, for example, **tf.train.MomentumOptimizer** used on the ResNet-50HC network updates the global step in **apply_gradients**, the step does not need to be updated when overflow occurs. Therefore, the script does not need to be modified.
- However, for the BERT network, the global step update is implemented in **create_optimizer**, including the judgment logic. In this case, the global step update needs to be performed in the optimizer. The following is a migration example:

In the original TensorFlow code, the global step is updated in **create_optimizer**, including the judgment logic.

```
def create_optimizer(loss, init_lr, num_train_steps, num_warmup_steps, hvd=None, manual_fp16=False,
use_fp16=False, num_accumulation_steps=1,
optimizer_type="adam", allreduce_post_accumulation=False):
    ...
    if tf.flags.FLAGS.npu_bert_clip_by_global_norm:
        new_global_step = tf.cond(all_are_finite, lambda: global_step + 1, lambda: global_step)
    else:
        new_global_step = global_step + 1
```

```
new_global_step = tf.identity(new_global_step, name='step_update')
train_op = tf.group(train_op, [global_step.assign(new_global_step)])
return train_op
```

During the migration to the Ascend platform, you need to update the global step in the optimizer as follows:

1. Comment out the global step update logic implemented in **create_optimizer** in the script.

```
def create_optimizer(loss, init_lr, num_train_steps, num_warmup_steps, hvd=None,
                    manual_fp16=False, use_fp16=False, num_accumulation_steps=1,
                    optimizer_type="adam", allreduce_post_accumulation=False):
    ...
    #if tf.flags.FLAGS.npu_bert_clip_by_global_norm:
    # new_global_step = tf.cond(all_are_finite, lambda: global_step + 1, lambda: global_step)
    #else:
    # new_global_step = global_step + 1
    #new_global_step = tf.identity(new_global_step, name='step_update')
    #train_op = tf.group(train_op, [global_step.assign(new_global_step)])
    return train_op
```

2. Before the last return statement of the **apply_gradients** function, add the logic for updating the global step in the **AdamWeightDecayOptimizer** and **LAMBOptimizer** classes, respectively. The **apply_gradients** function is called only when overflow is not found in the status check during loss scaling.

```
def apply_gradients(self, grads_and_vars, global_step=None, name=None,
                  manual_fp16=False):
    assignments = []
    for (grad, param) in grads_and_vars:
        ...
        new_global_step = global_step + 1
        new_global_step = tf.identity(new_global_step, name='step_update')
        assignments.extend([global_step.assign(new_global_step)])
    return tf.group(*assignments, name=name)
```

6.3 Operator Performance Optimization

6.3.1 Automatic Operator Tuning

If the network performance is still not satisfactory after the network-wide performance optimization actions are taken, you can look into tuning particular operators by identifying the optimal tiling policies with the Auto Tune tool. For details, see "[Auto Tune Tool Instructions](#)" in *CANN Auxiliary Development Tool Guide*.

6.3.2 Operator Profiling

Overview

The Profiling tool provides an economical solution for achieving optimal performance by accurate location of bottlenecks in software and hardware, efficient analysis, and specific optimization in the training process. Currently, the following options can be traced in profiling.

- **training_trace**: iteration tracing. Collects software profile data of a training job and the AI Software Stack to profile the training job. Focuses on data augmentation, forward and backward propagation, and gradient aggregation and update.

- **task_trace**: task tracing. Collects the HWTS and AI Core hardware information of the Ascend AI Processor and the start and end of each task.
- **aicpu**: AI CPU data augmentation tracing.

If you need to enable profile data collection during training (disabled by default), modify the training script as follows.

Collecting Profile Data with Estimator

```
from npu_bridge.npu_init import *

profiling_options = '{"output":"/tmp/profiling","training_trace":"on","fp_point":"resnet_model/conv2d/Conv2Dresnet_model/batch_normalization/FusedBatchNormV3_Reduce","bp_point":"gradients/AddN_70"}'
profiling_config = ProfilingConfig(enable_profiling=True, profiling_options = profiling_options)
session_config=tf.ConfigProto()

config = NPURunConfig(profiling_config=profiling_config, session_config=session_config)
```

Collecting Profile Data with sess.run()

In **sess.run()** mode, use **profiling_mode** and **profiling_options**, session configuration options, to enable tracing.

```
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
custom_op.parameter_map["profiling_mode"].b = True
custom_op.parameter_map["profiling_options"].s = tf.compat.as_bytes('{"output":"/tmp/profiling","training_trace":"on","fp_point":"resnet_model/conv2d/Conv2Dresnet_model/batch_normalization/FusedBatchNormV3_Reduce","bp_point":"gradients/AddN_70"}')
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

with tf.Session(config=config) as sess:
    sess.run()
```

Collecting Profile Data with Environment Variables

In addition to the preceding two methods, you can modify the environment variables in the startup script to enable profile data collection.

```
export PROFILING_MODE=true
export PROFILING_OPTIONS='{"output":"/tmp/profiling","training_trace":"on","task_trace":"on","aicpu":"on","fp_point":"resnet_model/conv2d/Conv2Dresnet_model/batch_normalization/FusedBatchNormV3_Reduce","bp_point":"gradients/AddN_70","aic_metrics":"PipeUtilization"}'
```

For details about environment variable configuration, see [Environment Variable Configuration](#).

Analyzing Profile Data

After the training is complete, find the collected profile data in the **output** directory. You can use the Profiling tool to analyze the profile data by referring to "[Profiling Tool Instructions](#)" in *CANN Auxiliary Development Tool Guide*.

If the profiling reports show that it is the custom operators on the network that lower the network performance, you can tune the custom operators by referring to the "Performance Optimization" sections in [TBE Custom Operator Development Guide](#).

7 Accuracy Tuning

[7.1 Operator Accuracy Analysis](#)

[7.2 Operator Overflow Detection](#)

7.1 Operator Accuracy Analysis

Overview

During the training process, the compute result (dump data) of each operator can be collected and then compared with that of the counterpart standard operator (such as TensorFlow) to facilitate accuracy tuning. Currently, the following options can be dumped.

- **input:** dumps operator inputs only.
- **output:** dumps operator outputs only.
- **all:** dumps both the operator inputs and outputs.

If you need to enable dumping during training (disabled by default), modify the training script as follows.

Precautions

- Currently, all iterations can be dumped. You can specify the iterations to be dumped. If the training dataset is large, the dump data volume of each iteration can reach about dozens of GB or even more. You are advised to control the number of iterations.
- Data dump and overflow detection are mutually exclusive.
- Currently, only dump data of AI Core and AI CPU operators can be collected. Dump data of collective communication operators cannot be collected.

Dumping with Estimator

In **Estimator** mode, use **dump_config** in **NPURunConfig** to collect dump data. Before **NPURunConfig** is created, instantiate a **DumpConfig** class for dump configuration, including the dump path, iteration data to be dumped, and whether to dump the inputs or outputs of the operator.

For details, see the **DumpConfig** constructor.

```
from npu_bridge.npu_init import *

# dump_path: dump path. Create the specified path in advance in the training environment (either in a
# container or on the host). The running user configured during installation must have the read and write
# permissions on this path.
# enable_dump: dump enable.
# dump_step: iterations to dump.
# dump_mode: dump mode, which can be set to input, output, or all
dump_config = DumpConfig(enable_dump=True, dump_path = "/home/HwHiAiUser/output",
dump_step="0|5|10", dump_mode="all")

session_config=tf.ConfigProto()

config = NPURunConfig(
    dump_config=dump_config,
    session_config=session_config
)
```

Dumping with sess.run()

In **sess.run** mode, set the dump parameters by setting the session configuration items **enable_dump**, **dump_path**, **dump_step**, and **dump_mode**.

```
config = tf.ConfigProto()

custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True

# enable_dump: dump enable
custom_op.parameter_map["enable_dump"].b = True
# dump_path: dump path. Create the specified path in advance in the training environment (either in a
# container or on the host). The running user configured during installation must have the read and write
# permissions on this path.
custom_op.parameter_map["dump_path"].s = tf.compat.as_bytes("/home/HwHiAiUser/output")
# dump_step: iterations to dump
custom_op.parameter_map["dump_step"].s = tf.compat.as_bytes("0|5|10")
# dump_mode: dump mode, which can be set to input, output, or all
custom_op.parameter_map["dump_mode"].s = tf.compat.as_bytes("all")
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF

with tf.Session(config=config) as sess:
    print(sess.run(cost))
```

Viewing Dump Data

If data dump data is collected during training, a dump file is generated to the **{dump_path}/{time}/{deviceid}/{model_name}/{model_id}/{data_index}** directory, for example, **/home/HwHiAiUser/output/20200808163566/0/ge_default_20200808163719_121/11/0**. In addition, a GE graph file, for example, **ge_proto_xxxx_Build.txt**, is generated in the same directory of the training script.

The fields in the dump data path and file are described as follows.

- **dump_path**: dump path, for example, **/home/HwHiAiUser/output**
- **time**: timestamp (for example, **20200317020343**)
- **deviceid**: device ID.
- **model_name**: subnetwork name. If the **model_name** directory contains more than one folder, dump data in the folder of the computational graph is used.

After the training script is executed, one or multiple GE graphs will be generated in the directory of the training script. Take **ge_proto_****_Build.txt** as an example. To select the computational graph, view each GE graph, respectively. The GE graph with the IteratorV2, Iterator, or GetNext operator is the computational graph. Value of the **name** field in the computational graph is used as the name of the computational graph.

- **model_id**: subnetwork ID.
- **data_index**: iterations to dump. If **dump_step** is specified, **data_index** equals **dump_step**. If not, **data_index** is indexed starting at 0 and is incremented by 1 with each dump.
- **dump_file**: formatted as **{op_type}.{op_name}.{taskid}.{timestamp}**
- Periods (.), forward slashes (/), backslashes (\), and spaces in **model_name**, **op_type** or **op_name** are replaced by underscores (_).

NOTE

- In the multi-device training scenario where more than one Ascend AI Processor is used, since the processes are not started at the same time as defined in the training script, multiple timestamp directories are generated when data is dumped.
- When the command is executed in a Docker, the generated data is stored in the Docker.

Analyzing Dump Data

You can use the Model Accuracy Analyzer to analyze the dump data by referring to "[Model Accuracy Analyzer Instructions](#)" in *CANN Auxiliary Development Tool Guide*.

More Cases

- [Accuracy Troubleshooting in Ascend 910 ALBERT-Base Migration](#)
- [\[Ascend Model Training Challenge\] Ascend 910 BERT-Base Migration Workflow](#)
- [What Do I Do If Loss Converges but Accuracy Drops?](#)
- [PWCNet Accuracy Tuning](#)
- [InceptionV4 Training Accuracy Tuning](#)

If the analysis reports show that it is the custom operators on the network that lower the network accuracy, you can tune the custom operators by referring to the section "Special Topics > Accuracy Optimization in DSL Mode" in [TBE Custom Operator Development Guide](#).

7.2 Operator Overflow Detection

Overview

For deep networks, a large amount of data will be dumped during operator accuracy analysis. In addition, due to the randomness of the network, it is difficult to locate the operators with accuracy drop compared with third-party equivalents. In this case, you can choose to enable the overflow detection function. Currently, the following three overflow detection modes are provided:

- **aicore_overflow**: detects AI Core operator overflow. With normal inputs, abnormal extreme outputs (such as float16 65500, 38400, and 51200) are detected. Once such fault is detected, analyze the cause of the overflow and modify the operator implementation based on the network requirements and operator logic.
- **atomic_overflow**: detects Atomic Add overflow. Atomic Add overflows are detected when data is moved from the UB to the external storage after AI Core computation.
- **all**: detects both AI Core operator overflow and Atomic Add overflow.

With the overflow detection result, you can locate the error operator, dump data of the particular operator, and analyze the dump data to solve the accuracy drop issue.

Precautions

- The data dump and overflow detection functions are mutually exclusive.
- The overflow detection function or data dump function might cause disk space insufficiency due to the generation of result files. You are advised to limit the number of iterations appropriately.

Detecting Overflow with Estimator

In **Estimator** mode, use **dump_config** in **NPURunConfig** to set the overflow detection mode. Before creating **NPURunConfig**, create instance of class **DumpConfig**. For details, see the **DumpConfig** constructor.

```
from npu_bridge.npu_init import *

# dump_path: dump path. Create the specified path in advance in the training environment (either in a
# container or on the host). The running user configured during installation must have the read and write
# permissions on this path.
# enable_dump_debug: overflow detection enable.
# dump_debug_mode: overflow detection mode select, which can be all, aicore_overflow, or
# atomic_overflow
dump_config = DumpConfig(enable_dump_debug = True, dump_path = "/home/HwHiAiUser/output",
dump_debug_mode = "all" )
session_config=tf.ConfigProto()

config = NPURunConfig(dump_config=dump_config, session_config=session_config)
```

Detecting Overflow with sess.run()

In **sess.run** mode, set the overflow detection mode by setting the session configuration options **dump_path**, **enable_dump_debug**, and **dump_debug_mode**.

```
config = tf.ConfigProto()

custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True

# dump_path: dump path. Create the specified path in advance in the training environment (either in a
# container or on the host). The running user configured during installation must have the read and write
# permissions on this path.
custom_op.parameter_map["dump_path"].s = tf.compat.as_bytes("/home/HwHiAiUser/output")
# enable_dump_debug: overflow detection enable.
custom_op.parameter_map["enable_dump_debug"].b = True
```



```
# dump_debug_mode: overflow detection mode select, which can be all, aicore_overflow, or  
atomic_overflow  
custom_op.parameter_map["dump_debug_mode"].s = tf.compat.as_bytes("all")  
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.  
  
with tf.Session(config=config) as sess:  
    print(sess.run(cost))
```

Viewing Overflow Data

If overflow data is collected during training, an overflow data file is generated to the `{dump_path}/{time}/{deviceid}/{model_id}/{data_index}` directory, for example, `/home/HwHiAiUser/output/20200808163566/0/11/0`.

If no overflow data is collected during training, that is, no overflow occurs, the preceding directory is not generated.

The fields in the dump data path and file are described as follows.

- **dump_path**: user-defined path for storing overflow data, for example, `/home/HwHiAiUser/output`.
- **time**: timestamp (for example, `20200808163566`)
- **deviceid**: device ID.
- **model_id**: subnetwork ID.
- **data_index**: iterations to detect overflow.
- Dump files:
 - The dump file of an overflow operator is named as: `{op_type}.{op_name}.{taskid}.{timestamp}`. Any period (.), slash (/), backslash (\), or space in the **op_type** or **op_name** field is replaced by an underscore (_).
 - You can identify an overflow operator based on its dump file name. To view the inputs and outputs of an overflow operator, refer to [Analyzing the Dump File of an Overflow Operator](#).
 - The operator overflow data file is named as: `OpDebug.Node_Opdebug.{taskid}.{timestamp}`, where **taskid** is not the task ID of the overflow operator and can be ignored.

To locate the overflow cause, follow the instructions in [Analyzing an Operator Overflow Data File](#).

NOTE

- In the multi-device training scenario where more than one Ascend AI Processor is used, since the processes are not started at the same time as defined in the training script, multiple timestamp directories are generated when data is dumped.
- When the command is executed in a Docker, the generated data is stored in the Docker.

Analyzing the Dump File of an Overflow Operator

Step 1 Upload the `{op_type}.{op_name}.{taskid}.{timestamp}` file to the environment installed with Toolkit.

- Step 2** Go to the directory where the parse script is stored. The following assumes that the Toolkit installation path is */home/HwHiAiUser/Ascend/ascend-toolkit/latest*.

```
cd /home/HwHiAiUser/Ascend/ascend-toolkit/latest/toolkit/tools/  
operator_cmp/compare
```

- Step 3** Run the **msaccucmp.pyc** script to convert the dump file into a NumPy file. The following is an example:

```
python3.7.5 msaccucmp.pyc convert -d /home/HwHiAiUser/dump -out /home/  
HwHiAiUser/dumptonumpy -v 2
```

 NOTE

The **-d** option enables the conversion of a single dump file or all dump files in a path.

- Step 4** Use Python to save the NumPy data into a text file. The following is an example:

```
$ python3.7.5  
  
>>> import numpy as np  
  
>>> a = np.load("/home/HwHiAiUser/dumptonumpy/  
Pooling.pool1.1147.1589195081588018.output.0.npy")  
  
>>> b = a.flatten()  
  
>>> np.savetxt("/home/HwHiAiUser/dumptonumpy/  
Pooling.pool1.1147.1589195081588018.output.0.txt", b)
```

The dimension and **Dtype** information no longer exist in the .txt file. For details, visit the NumPy website.

----End

Analyzing an Operator Overflow Data File

Since the generated overflow data is in binary format, you need to interpret the binary file into a readable format, such as JSON.

- Step 1** Upload the overflow data file **OpDebug.Node_Opdebug.{taskid}.{timestamp}** to the Toolkit installation environment.

 NOTE

You are advised to go to the **data_index** directory with the minimum value, and use the dump file with the minimum **timestamp** for analysis.

- Step 2** Go to the directory where the parse script is stored. The following assumes that the Toolkit installation path is */home/HwHiAiUser/Ascend/ascend-toolkit/latest*.

```
cd /home/HwHiAiUser/Ascend/ascend-toolkit/latest/toolkit/tools/  
operator_cmp/compare
```

- Step 3** Run the parse command:

```
python3.7.5 msaccucmp.pyc convert -d /home/HwHiAiUser/opdebug/  
Opdebug.Node_OpDebug.59.1597922031178434 -out /home/HwHiAiUser/  
result
```

The options are described as follows:

- **-d**: directory of the overflow data, including the file name
- **-out**: directory of the parsing result. If not specified, the current directory is used.

Step 4 Find the **result.txt** parse result as follows.

```
{
  "DHA Atomic Add": {
    "model_id": 0,
    "stream_id": 0,
    "task_id": 0,
    "task_type": 0,
    "pc_start": "0x0",
    "para_base": "0x0",
    "status": 0
  },
  "L2 Atomic Add": {
    "model_id": 0,
    "stream_id": 0,
    "task_id": 0,
    "task_type": 0,
    "pc_start": "0x0",
    "para_base": "0x0",
    "status": 0
  },
  "AI Core": {
    "model_id": 514,
    "stream_id": 563,
    "task_id": 57,
    "task_type": 0,
    "pc_start": "0x1008005b0000",
    "para_base": "0x100800297000",
    "kernel_code": "0x1008005ae000",
    "block_idx": 1,
    "status": 32
  }
}
```

NOTE

If both AI Core operator overflow detection and Atomic Add overflow detection are enabled, only the earliest overflow record is displayed.

In the preceding example, the earliest overflow record is an AI Core operator overflow.

The fields are described as follows:

- **model_id**: ID of the model where the overflow operator is located.
- **stream_id**: ID of the stream where the overflow operator is located.
- **task_id**: task ID of the overflow operator.
- **task_type**: task type of the overflow operator.
- **pc_start**: start of the code program of the overflow operator.
- **para_base**: parameter start address of the overflow operator.
- **kernel_code**: start of the code program of the overflow operator, which is equivalent to **pc_start**.
- **block_idx**: block ID of the overflow operator.
- **status**: status of the AI Core status register, including the overflow information. You can analyze the **status** value to identify the specific overflow error.

----End

Status Reference

- The **status** field that reflects the AI Core operator overflow detection result is in decimal format. You need to convert it into the hexadecimal format before locating the fault.

For example, assume that the value of **status** is **272**. The hexadecimal equivalent of the value is **0x00000110**. Therefore, the error cause is **0x00000010+0x00000100**.

- **0x00000008**: inversion overflow of the minimum negative sign bit of a signed integer
- **0x00000010**: integer addition, subtraction, multiplication, or multiplication overflow
- **0x00000020**: floating-point computation overflow
- **0x00000080**: negative input for floating-point to unsigned conversion
- **0x00000100**: FP32 to FP16 conversion or 32-bit signed integer to FP16 conversion overflow
- **0x00000400**: Cube accumulation overflow

Note: The preceding floating-point errors correspond to the hexadecimal bits, which might lead to combinations of floating-point errors.

- The **status** field that reflects the DHA Atomic Add overflow detection result is in decimal format. You need to convert it into a binary number and convert bits 8 to 15 to a hexadecimal number before locating the fault.

For example, assume that the value of **status** is **2546**. The binary equivalent of the value is **100111110010**. Convert bits 8 to 15 **1001** to obtain the hexadecimal number **0x9**, that is, the error cause.

- **0x9**: atomic overflow
- **0xA**: atomic underflow
- **0xB**: atomic srcnan (invalid source operand)
- **0xC**: atomic dstnan (invalid destination operand)
- **0xD**: atomic bothnan (invalid source operand and destination operand)

- The **status** field that reflects the L2 Atomic Add overflow detection result is in decimal format. You need to convert it into a binary number before locating the fault.

For example, assume that the value of **status** is **2546**. The binary equivalent of the value is **100111110010**, and bits 16 to 18 make **000**. It can be determined that no error occurs.

- **001**: atomic overflow
- **010**: atomic underflow
- **011**: atomic srcnan (invalid source operand)
- **100**: atomic dstnan (invalid destination operand)
- **101**: atomic bothnan (invalid source operand and destination operand)

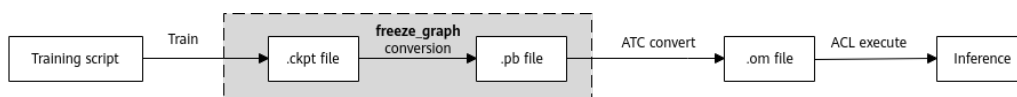
8 Model Conversion and Storage

Model Conversion and Storage with `sess.run()`

During TensorFlow training with `sess.run()`, `saver = tf.train.Saver()` and `saver.save()` are used to save the model. The following files are generated after each `saver.save()` call:

- **checkpoint**: a text file that records the latest checkpoint files and the list of other checkpoint files.
- **model.ckpt.data-00000-of-00001**: saves the current parameter settings.
- **model.ckpt.index**: saves the current parameter names.
- **model.ckpt.meta**: saves the current graph structure.

In this mode, the model weight data and model graph are saved separately. In the inference scenario, the TensorFlow **freeze_graph** function is used to combine the weight data and model graph into a .pb file, as shown in the dotted box in the following figure.



The workflow of generating a .pb file by using the TensorFlow **freeze_graph** function is as follows:

1. Specify the network model and checkpoint file path.
2. Define the input node. For example the input node for training is `IteratorV2`, but the input required for inference is a placeholder.
3. Define the output node. The output node required for training is the loss value, and the outputs required for inference are `Argmax` or `BiasAdd`.
4. Generally, an operator is processed in different ways in the training graph and inference graph (for example, `BatchNorm` and `dropout` operators). Therefore, you need to call the network model to generate an inference graph.
 - For the `BatchNorm` operator, the mean and variance of the `BatchNorm` operator are calculated based on the samples. However, during inference, the mean and variance of the `BatchNorm` operator are calculated based on the moving mean of the samples. Therefore, the mean calculation method of `BatchNorm` varies with the training or inference scenario.

- For the dropout operator: During inference, dropout needs to be masked by setting **rate** to **1**.

```
if is_training:
    x = npu_ops.dropout(x, 0.65)
else:
    x = npu_ops.dropout(x, 1.0)
```

Based on the preceding differences, you need to find the entry function of the inference test logic in the training script and set **is_training** to **False** during execution to generate an inference graph.

```
# Call the network to generate an inference graph. alexnet.inference is the entry function of the
inference test logic in the training script.
logits = alexnet.inference(inputs, version="he_uniform", num_classes=1000, is_training=False)
```

5. Call **tf.train.writegraph** to save the preceding inference graph to a .pb file as the input of the **freeze_graph** function.
6. Call **freeze_graph** to merge the .pb graph file generated by **tf.train.writegraph** and the checkpoint file to generate a .pb graph file for inference.

A code sample is provided as follows.

```
import tensorflow as tf
from tensorflow.python.tools import freeze_graph
from npu_bridge.npu_init import *

# Import the network model file.
import alexnet
# Specify the checkpoint path.
ckpt_path = "/opt/npu/model_ckpt/alexnet/model_8p/model.ckpt-0"

def main():
    tf.reset_default_graph()
    # Define the input node of the network.
    inputs = tf.placeholder(tf.float32, shape=[None, 224, 224, 3], name="input")
    # Call the network to generate an inference graph.
    logits = alexnet.inference(inputs, version="he_uniform",
                               num_classes=1000, is_training=False)
    # Define the output node of the network.
    predict_class = tf.argmax(logits, axis=1, output_type=tf.int32, name="output")
    with tf.Session() as sess:
        # Save the graph. The model.pb file is generated in the ./pb_model folder.
        # The model.pb file will be provided as input_graph to the following freeze_graph call.
        tf.train.write_graph(sess.graph_def, './pb_model', 'model.pb') # Generate a model file.
        freeze_graph.freeze_graph(
            input_graph='./pb_model/model.pb', # Pass the model file generated using write_graph.
            input_saver="",
            input_binary=False,
            input_checkpoint=ckpt_path, # Pass the checkpoint file generated in training.
            output_node_names='output', # Consistent with the output node of the inference network.
            restore_op_name='save/restore_all',
            filename_tensor_name='save/Const:0',
            output_graph='./pb_model/alexnet.pb', # Set to the name of the inference network to be
generated.
            clear_devices=False,
            initializer_nodes="")
        print("done")

if __name__ == '__main__':
    main()
```

The following describes the key parameters of **freeze_graph**. Retain the default values for parameters not described below.

- **input_graph**: model file generated by **write_graph**.
- **input_binary**: used in conjunction with **input_graph**. If set to **true**, **input_graph** is binary. If set to **false**, **input_graph** is a file. Defaults to **false**.

- **input_checkpoint**: path of the checkpoint file.
- **output_node_names**: name of the output node. Use commas (,) to separate multiple names.
- **output_graph**: path of the converted .pb file.

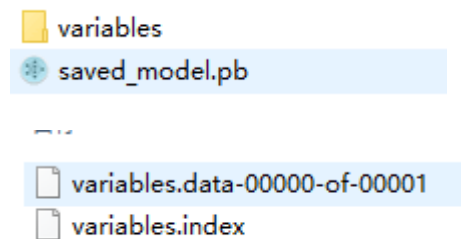
After the script is executed, the **alexnet.pb** file is generated in the **./pb_model/** folder. This file is the converted .pb image file used for inference.

NOTE

For details about the dependent environment variables, see [5.2 Training in a Bare Metal Environment](#).

Model Conversion and Storage with Estimator

Estimator can save models in ckpt and saved_model formats. The ckpt method is similar to the **sess.run** method. The saved_model format is recommended because the saved model is lightweight and possible errors are avoided. saved_model is generally saved using **estimator.export_savedmodel**, as shown in the following figures.



If you need to convert the saved_model model into a .pb model for inference, edit the training script as follows:

1. Define the input node.

The input received by **Estimator** during training is in Iterator format, which facilitates iteration between epochs. Before saving a model for inference, use the placeholder to define a specific input.

```
def serving_input_fn():
    input_ids = tf.placeholder(tf.int32, [None, FLAGS.max_seq_length], name='input_ids')
    input_fn = tf.estimator.export.build_raw_serving_input_receiver_fn({
        'input_ids': input_ids,
    })()
    return input_fn
```

2. Save the saved_model model.

Estimator can directly call the **export_savedmodel** function to save the model and automatically switch the mode and freeze the graph.

```
if FLAGS.do_export:
    estimator.evaluate()
    estimator.export_savedmodel(FLAGS.output_dir, serving_input_fn)
```

3. Freeze the .pb model.

Use the **freeze_graph** function of TensorFlow to freeze a graph into a .pb model. Note that if your model has an NPU custom operator, you need to import the NPU operator module to the source code of **freeze_graph**.

```
import tensorflow as tf
from tensorflow.python.tools import freeze_graph
from npu_bridge.npu_init import *

freeze_graph.freeze_graph(
    input_saved_model_dir='savedModel',
    output_node_names='output',
    output_graph='test.pb',
    initializer_nodes="",
    input_graph= None,
    input_saver= False,
    input_binary=False,
    input_checkpoint=None,
    restore_op_name=None,
    filename_tensor_name=None,
    clear_devices=False,
    input_meta_graph=False)
```

4. Run the **freeze_npu_savedModel.py** command to convert and save the model.

```
python3 freeze_npu_savedModel.py --input_saved_model_dir=savedModel --
output_node_names=loss/Softmax --output_graph=test.pb
```

NOTE

For details about the dependent environment variables, see [5.2 Training in a Bare Metal Environment](#).

Offline Model Generation for Offline Inference

Use the ATC tool to convert the obtained .pb model into an .om offline model, which can be executed for offline inference on the ***/home/HwHiAiUser/Ascend/ascend-toolkit/latest***. For details, see "[ATC Tool Instructions](#)" in *CANN Auxiliary Development Tool Guide*.

9 More Functions

9.1 Mixed Computing

9.2 Log and Summary Operators

9.3 Collective Communication APIs

9.1 Mixed Computing

Overview

By default, the fully offloaded mode is used on Ascend AI Processor. That is, the execution of compute operators is fully offloaded on the device side.

As a supplement to the fully offloaded mode, mixed computing allows some operators to be executed online in the frontend framework, improving the Ascend AI Processor flexibility for adapting to TensorFlow. Note the following:

- In mixed computing mode, **iteration offloading** is not supported. That is, **iterations_per_loop** must retain the default value **1**.
- In addition to the operators that are not offloaded by default, you can also configure the operators that are not offloaded by using **without_npu_compile_scope**.
- The FusedBatchNormV3 operator is released in 2019. Its fifth output is a CUDA-optimized output. It is not supported on Ascend AI Processor in mixed computing mode. If **tf.layers.batch_normalization** is used in your training script, you can use "with compat.forward_compatibility_horizon(2019, 5, 1):" to skip this operator.

Enabling Mixed Computing with Estimator

In **Estimator** mode, use **mix_compile_mode** of **NPURunConfig** to enable the mixed computing function.

```
from npu_bridge.npu_init import *  
  
session_config=tf.ConfigProto()  
config = NPURunConfig(session_config=session_config, mix_compile_mode=True, iterations_per_loop=1)
```

Enabling Mixed Computing with `sess.run()`

In `sess.run()` mode, use the session configuration option `mix_compile_mode` to enable the mixed computing function and use `without_npu_compile_scope` to configure operators not offloaded.

```
import tensorflow as tf
from npu_bridge.npu_init import *

X = tf.random_normal([2,])
Y = tf.random_normal([2,])

with npu_scope.without_npu_compile_scope():
    pred = tf.add(tf.multiply(X, 1.), 0.)
    cost = tf.reduce_sum(tf.abs(pred-Y))

config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
custom_op.parameter_map["mix_compile_mode"].b = True
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

with tf.Session(config=config) as sess:
    print(sess.run(cost)) # The reduce_sum node is executed on the host.
```

9.2 Log and Summary Operators

Background

The execution of Log and Summary operators is offloaded to the device side. If you need to capture the Log and Summary information on the device side and view the information of the corresponding step on the host side, modify the training script by referring to this section.

Log Printing

In **Estimator** mode, the system starts the dequeue thread when the Log information is returned to the host. The Log information on the device side can be directly printed. Therefore, no modification is needed.

```
print_op = tf.print(loss)
with tf.control_dependencies([print_op]):
    train_op = xxx # The Print operator depends on the nodes that can be executed on the graph.
    Otherwise, the Print operator does not take effect.
```

In **`sess.run`** mode, the dequeue thread is not started when the Log information is returned to the host. Therefore, you need to start the dequeue thread separately to obtain the buffered Log information.

```
from threading import Thread

import sys
def dequeue():
    tf.reset_default_graph()
    outfeed_log_tensors = npu_ops.outfeed_dequeue_op(
        channel_name="_npu_log",
        output_types=[tf.string],
        output_shapes=[])
    dequeue_ops = tf.print(outfeed_log_tensors, sys.stderr)
    with tf.Session() as sess:
        i = 0
        while i < max_train_steps: // max_train_steps indicates the maximum number of iterations.
```

```
sess.run(dequeue_ops)
i = i + 1

t1 = Thread(target=dequeue)
t1.start()
```

For training, the Assert or Print operator is used to print Log information.

```
print_op = tf.print(loss)
with tf.control_dependencies([print_op]):
    train_op = xxx # The Print operator depends on the nodes that can be executed on the graph.
Otherwise, the Print operator does not take effect.
```

Summary Printing

In **sess.run** mode, it is not supported to send summary back to the host for viewing.

In **Estimator** mode, you need to define a **host_call** function that contains the Summary information to be collected.

```
def _host_call_fn(gs, loss):
    with summary.create_file_writer(
        "./model", max_queue=1000).as_default():
        # Record summary every step.
        with summary.always_record_summaries():
            # Record summary every 2000 steps.
            #with summary.record_summaries_every_n_global_steps(2000, global_step=gs):
                summary.scalar("host_call_loss", loss, step=gs)
    return summary.all_summary_ops()
```

Then, pass **host_call** to the **NPUEstimatorSpec** constructor. The system starts the enqueue thread when the Summary operator is executed on the device side and starts the dequeue thread when the Summary information is sent back to the host, so that the information of each or *N* steps will be sent back to the host.

host_call is a tuple consisting of a function and a list or dictionary of tensors. It is used to return a list of tensors. **host_call** applies to **train()** and **evaluate()** calls.

```
from npu_bridge.npu_init import *

host_call = (_host_call_fn, [global_step, loss])
return NPUEstimatorSpec(mode=tf.estimator.ModeKeys.TRAIN, loss=loss, train_op=train_op,
    host_call=host_call)
```

The following is a complete code example.

```
from npu_bridge.npu_init import *

# Define a host_call function.
from tensorflow.contrib import summary
def _host_call_fn(gs, loss):
    with summary.create_file_writer(
        "./model", max_queue=1000).as_default():
        with summary.always_record_summaries():
            summary.scalar("host_call_loss", loss, step=gs)
    return summary.all_summary_ops()

def input_fn():
    "Build dataset"

# Call host_call in model_fn to capture the information to be viewed.
def model_fn():
    "Build a forward/backward model"
    model = ***
    loss = ***
    optimizer = tf.train.MomentumOptimizer(learning_rate=c, momentum=0.9)
```

```
global_step = tf.train.get_or_create_global_step()
grad_vars = optimizer.compute_gradients(loss)
minimize_op = optimizer.apply_gradients(grad_vars, global_step)
update_ops = tf.get_collection(tf.GraphKeys.UPDATE_OPS)
train_op = tf.group(minimize_op, update_ops)
host_call = ( _host_call_fn, [global_step, loss])
return NPUEstimatorSpec(mode=tf.estimator.ModeKeys.TRAIN, loss=loss, train_op=train_op,
host_call=host_call)

run_config = NPURunConfig()

classifier = NPUEstimator(model_fn=model_fn, config=run_config, params={ })
classifier.train(input_fn=lambda: input_fn(), max_steps=1000)
```

9.3 Collective Communication APIs

9.3.1 API Overview

The high-level API **NPU Distributed Optimizer** enables the user to automatically complete gradient aggregation without sensing AllReduce, implementing data parallel training. In addition, to meet users' requirements for flexibility, the API for atomic communication in collective communication is provided to implement native representation of data parallelism.

Currently, the **AllReduce**, **Broadcast**, **AllGather**, **ReduceScatter**, **Send**, as well as the **Receive** operations are supported, and a process group that participates in collective communication can be customized. For example, eight processors can be divided into groups of four for collective communication. Collective communication provides common APIs such as rank management, gradient segmentation, and collective communication prototype.

Table 9-1 Terminology

Term	Description
ranktable file	Describes the information of the cluster involved in collective communication, including the organization information about the servers, devices, and containers.
Rank	Indicates the processes that participate in collective communication.
Group	Indicates the process group that participates in collective communication. The process group includes: • HCCL world group: default global group, including all ranks that participate in collective communication. The group is created using the ranktable file. • Custom group: a subset of the process groups contained in the HCCL world group. The ranks in the ranktable can be defined as different groups through the create_group API, and the collective communication algorithms can be executed in parallel.

Term	Description
Rank size	- Rank size: indicates the number of ranks in a group. The maximum value is 4096. - Local rank size: indicates the number of ranks in a group on the server where the processes are located. The value can be 1, 2, 4 or 8.
Rank ID	- Rank ID: indicates the ID of a process in a group. The value ranges from 0 to the value of rank size – 1. For a custom group, the rank starts from 0 in the group. For an HCCL world group, the rank ID is the same as the world rank ID. - World rank ID: indicates the rank ID of a process in a HCCL world group. The value ranges from 0 to rank size – 1. - Local rank ID: indicates the rank ID of a process in a group on the server where the process is located. The value ranges from 0 to local rank size – 1.
Gradient segmentation	In the backward pass phase of the neural network training, AllReduce of multiple parameters in the GE optimization computational graph is fused and segmented. AllReduce is performed segment by segment based on the gradient generation sequence to update the backward pass parameters of the network, implementing compute resource optimization, as well as parallel optimization of communication time and backward pass time.

9.3.2 Initializing Collective Communication

If you call an HCCL API such as **get_local_rank_id**, **get_rank_size**, or **get_rank_id** before calling **sess.run()** or **estimator.train()**, you need to start another session and execute **initialize_system** to initialize collective communication. After the training is complete, execute **shutdown_system** and close the session.

```
import tensorflow as tf
from npu_bridge.npu_init import *

npu_int = npu_ops.initialize_system()
npu_shutdown = npu_ops.shutdown_system()

# If some specific functions need to be enabled during training, pass related arguments here. For details,
# see the description of the initialize_system API.
config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

init_sess = tf.Session(config=config)
init_sess.run(npu_int)

# Call an HCCL API...
# Perform training. If another session is started to perform training, set the same run parameters as the
# preceding ones.

init_sess.run(npu_shutdown)
init_sess.close()
```

Or:

```
import tensorflow as tf
from npu_bridge.npu_init import *

npu_init = npu_ops.initialize_system()
npu_shutdown = npu_ops.shutdown_system()

# If some specific functions need to be enabled during training, pass related arguments here. For details,
# see the description of the initialize_system API.
config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

with tf.Session(config=config) as sess:
    sess.run(npu_init)
    # Call an HCCL API...
    # Perform training...
    sess.run(npu_shutdown)
```

9.3.3 Specifying Devices to Perform Collective Communication

Call **create_group** to specify the devices that participate in collective communication. If this API is not called to create a custom group, all devices are created as the global **hccl_world_group** by default based on the **ranktable** file.

For example, you can select the first two devices in **hccl_world_group** as a group for collective communication:

```
from npu_bridge.npu_init import *
create_group("myGroup", 2, [0, 1])
```

9.3.4 Obtaining the Group Information

You can call the group management API to obtain the group information.

get_rank_size: obtains the number of all devices in the current group.

```
from npu_bridge.npu_init import *
rankSize = get_rank_size("myGroup")
```

get_local_rank_size: obtains the number of devices in a group on the server where the current device is located.

```
from npu_bridge.npu_init import *
localRankSize = get_local_rank_size("myGroup")
```

get_rank_id: obtains the logical device ID (rank ID) of the current device in a group.

```
from npu_bridge.npu_init import *
rankId = get_rank_id("myGroup")
```

get_local_rank_id: obtains the logical device ID (local rank ID) of the current device on the server.

```
from npu_bridge.npu_init import *
localRankId = get_local_rank_id("myGroup")
```

9.3.5 Setting the Backward Gradient Segmentation Policy

You can call the gradient segmentation APIs to set the AllReduce segmentation and fusion policy in the backward pass phase.

set_split_strategy_by_idx: sets the backward gradient segmentation policy in the collective communication group based on the gradient index ID.

```
from npu_bridge.npu_init import *
set_split_strategy_by_idx([20, 100, 159])
```

set_split_strategy_by_size: sets the backward gradient segmentation policy in the collective communication group by ratio.

```
from npu_bridge.npu_init import *
set_split_strategy_by_size([60, 20, 20])
```

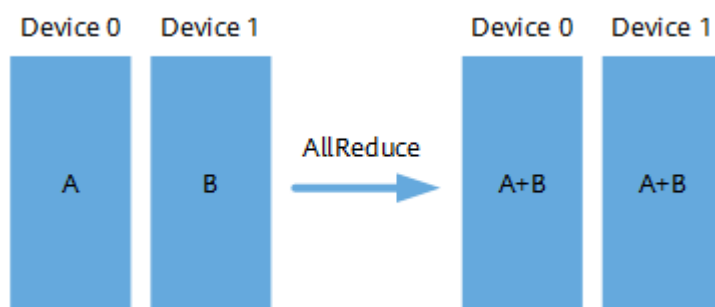
9.3.6 Computing Tensor Nodes for Collective Communication

You can call the operator prototype APIs to compute the tensors involved in collective communication.

AllReduce

allreduce: performs reduction on tensors with the same name across ranks within a collective communication group. The Reduce operation is specified by the **reduction** parameter.

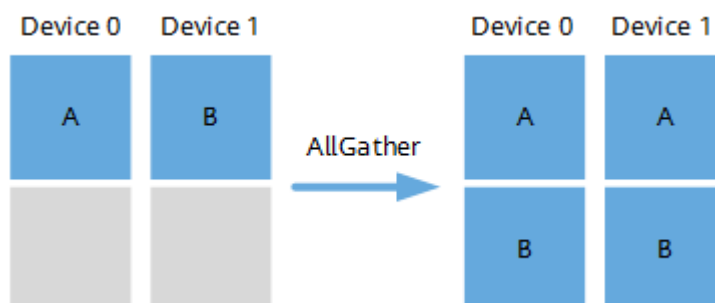
```
#-----AllReduce test (two devices)-----
from npu_bridge.npu_init import *
tensor = tf.random_uniform((1, 3), minval=1, maxval=10, dtype=tf.float32)
allreduce_test = hccl_ops.allreduce(tensor, "sum")
```



AllGather

allgather: Each device receives the aggregation of tensor data from all ranks in the order of the ranks.

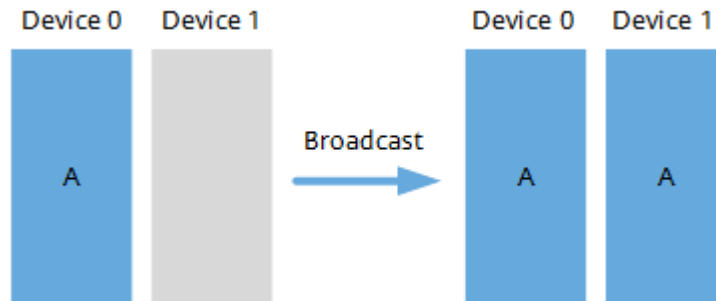
```
#-----AllGather test (two devices)-----
from npu_bridge.npu_init import *
cCon = tf.constant([1.0, 2.0, 3.0])
allgather_test = hccl_ops.allgather(cCon, 2)
#----- rank 0/1 allgather_test = [1.0, 2.0, 3.0, 1.0, 2.0, 3.0] -----
```



Broadcast

broadcast: copies an N-element buffer on the root rank to all ranks.

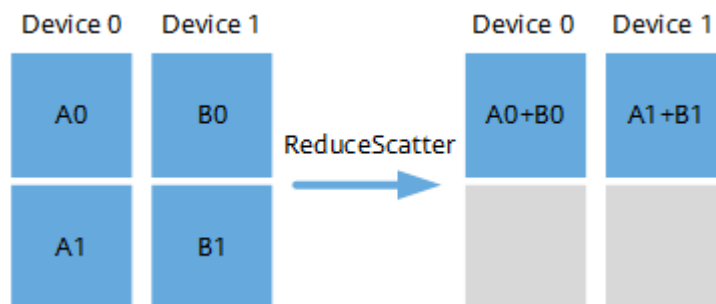
```
#-----Broadcast test (two devices)-----
from npu_bridge.npu_init import *
cCon = tf.Variable([1.0,2.0,3.0])
input = [cCon]
broadcast_test = hccl_ops.broadcast(input, 0)
#----- rank 0/1 broadcast_test = [1.0, 2.0, 3.0] -----
```



ReduceScatter

reduce_scatter: performs the same operation as the Reduce operation, except that each rank receives a subpart of the result. The Reduce operation is specified by the **reduction** parameter.

```
#-----ReduceScatter test (two devices)-----
from npu_bridge.npu_init import *
cCon = tf.constant([1.0,2.0,3.0,4.0])
reducescatter_test = hccl_ops.reduce_scatter(cCon, "sum", 2)
#-----rank 0 reducescatter_test = [2.0, 4.0] -----
#-----rank 1 reducescatter_test = [6.0, 8.0] -----
```



Send

send: sends data to a rank within a collective communication group.

```
#-----Send test-----
from npu_bridge.npu_init import *
sr_tag = 0
dest_rank = 1
hccl_ops.send(tensor, sr_tag, dest_rank)
```

Receive

receive: receives data from a rank within a collective communication group.

```
#-----Receive test (two devices)-----
from npu_bridge.npu_init import *
sr_tag = 0
src_rank = 0
tensor = hccl_ops.receive(tensor.shape, tensor.dtype, sr_tag, src_rank)
```


10 Reference

[10.1 Migration Examples](#)

[10.2 API Reference](#)

[10.3 Environment Variables](#)

[10.4 Device Resource Configuration File Templates](#)

10.1 Migration Examples

10.1.1 ResNet-50 Model Training Using the ImageNet Dataset

10.1.1.1 Preparations

Obtaining the Dataset

This sample uses the ImageNet dataset as an example. Download the dataset from <http://www.image-net.org/>.

Model Overview

ResNet-50 is a deep residual network that can be used to classify 1000 classes of the CIFAR-10 and ImageNet datasets.

Obtaining the Original Model

The original ResNet network script is available at https://github.com/tensorflow/models/tree/r2.1_model_reference/official.

Directory Structure

The directory is organized as follows. (Only some involved files are listed. For more files, see the original ResNet script.)

```
|— r1 // Original model directory.  
|— resnet // ResNet main directory.
```

```

├── __init__.py
├── imagenet_main.py    // Script for training the network based on the ImageNet dataset.
├── imagenet_preprocessing.py // ImageNet preprocessing module.
├── resnet_model.py    // ResNet model file.
├── resnet_run_loop.py // Data input processing and run loop (training, validation, and test).
├── README.md         // Project description file.
├── utils
│   ├── export.py    // Data receive functions, which define the parameter format that the exported
│   │               // model can respond to.
│   └── utils
│       ├── flags
│       ├── core.py    // Public APIs including the parameter definition.
│       ├── logs
│       ├── hooks_helper.py // Tool used for custom model tests and training, such as the function for
│       │               // calculating the number of steps per second, and the function of capturing CPU/GPU analysis information.
│       ├── logger.py // Log tool.
│       └── misc
│           ├── distribution_utils.py // Auxiliary functions used for running models in distributed mode.
│           └── model_helpers.py    // Functions that can be called by models, such as functions that controls
│                                   // whether models stop.

```

Migration Description

The following describes the migration workflow by manually editing the training script instead of using the migration tool, in order to help you understand all key migration points in detail.

10.1.1.2 Training Workflow

About Estimator

Estimator is a high-level API of TensorFlow and is introduced in TensorFlow 1.10 released in 2018. It greatly streamlines the programming process of machine learning. **Estimator** has many advantages, for example, good support for distribution, simplified model creation, and code sharing between model developers.

To use the **Estimator** API to develop a training script, perform the following steps.

Table 10-1 Training flow

Step	Description
Preprocess the data.	Create the input function input_fn .
Build a model.	Construct the model function model_fn .
Set run configuration.	Instantiate Estimator and pass an object of the Runconfig class as the run parameter.
Start training.	Call the training method Estimator.train() in Estimator to train the model with a fixed number of steps using the specified input.

10.1.1.3 Training Code Directory Structure

Directory Structure

The directory is organized as follows. (Only some involved files are listed. For more files, see the original ResNet script.)

```

r1
├── resnet      // ResNet main directory.
│   ├── imagenet_main.py    // Script for training the network based on the ImageNet dataset.
│   ├── imagenet_preprocessing.py  // ImageNet preprocessing module.
│   ├── resnet_model.py    // ResNet model file.
│   └── resnet_run_loop.py  // Data input processing and run loop (training, validation, and test).
└── utils
    ├── flags
    └── _base.py  // Defines the common parameters and set the default value.

```

Directory Files

Table 10-2 PY file description

File Name	Description
imagenet_main.py	Includes APIs related to ImageNet preprocessing, model construction definition, and model runtime. The get_filenames() , parse_record() , input_fn() , get_synth_input_fn() , and _parse_example_proto() functions are used for data preprocessing. The ImagenetModel class, imagenet_model_fn() , run_cifar() , and define_cifar_flags() functions are used for model operations.
imagenet_preprocessing.py	Includes ImageNet image data preprocessing APIs for sampling training images with the provided bounding box, cropping images based on the bounding box, randomly flipping images, and adjusting images to the target output size (the aspect ratio is not retained). Image resizing (aspect ratio retained) and central cropping are used during the evaluation process.
resnet_model.py	Implements the ResNet model, including the auxiliary functions for ResNet model construction and ResNet block definition functions.

File Name	Description
resnet_run_loop.py	Model runtime file, including input processing and run loop. Input processing includes decoding the input data, converting the format, outputting image and label, as well as setting data randomization, batch, and pre-reading based on whether it is the training scenario. Run loop includes constructing Estimator , and performing training and validation. In general, the model is run in specific environments to implement data and error flow, so that model parameters can be updated using gradient descent.

10.1.1.4 Data Preprocessing

The data preprocessing process is the same as that of the original model. Some of the code is modified to adapt to the Ascend 910 AI Processor for higher compute capability. The displayed code shows the modifications.

Defining the Input Function `input_fn`

Data preprocessing of the ImageNet dataset is used as an example. The modified .py files and functions for adapting to the Ascend 910 AI Processor are as follows.

Table 10-3 Data preprocessing APIs

Function	Description	Location
<code>input_fn()</code>	Input function that processes the dataset for Estimator training and outputs real data.	<code>/official/r1/resnet/imagenet_main.py</code>
<code>resnet_main()</code>	Main API that contains data input, run configuration, training, and verification.	<code>/official/r1/resnet/resnet_run_loop.py</code>

1. Import the following header files to the **official/r1/resnet/imagenet_main.py** file:

```
from hccl.manage.api import get_rank_size
from hccl.manage.api import get_rank_id
```

2. Obtain the number of devices and device IDs to support parallel data processing.

Tweak: **`input_fn()`** in **official/r1/resnet/imagenet_main.py** (The changes are in bold.)

```
def input_fn(is_training, data_dir, batch_size, num_epochs=1, dtype=tf.float32,
            datasets_num_private_threads=None, parse_record_fn=parse_record,
            input_context=None, drop_remainder=False, tf_data_experimental_slack=False):
    """Function that provides training and verification batches.
```

Parameter description:

- is_training**: boolean value indicating whether the input is used for training.
- data_dir**: file path that contains the input dataset.
- batch_size**: size of each batch.
- num_epochs**: number of epochs.
- dtype**: data type of an image or feature.
- datasets_num_private_threads**: number of threads dedicated to **tf.data**.
- parse_record_fn**: entry function for parsing TFRecords.
- input_context**: **tf.distribute.InputContext** object passed by **tf.distribute.Strategy**
- drop_remainder**: specifies whether to retain or discard the last batch if the data volume of the last batch is smaller than the value of **batch_size**. If set to **True**, the batch dimension is fixed.
- tf_data_experimental_slack**: specifies whether to enable the **experimental_slack** option of **tf.data**.

Returns:
A dataset that can be used for iteration.

```

"""
# Obtain the file path.
filenames = get_filenames(is_training, data_dir)
# Split the file based on the first dimension.
dataset = tf.data.Dataset.from_tensor_slices(filenames)
if input_context:
    # Obtain the number of devices and device IDs to support parallel data processing.
    ##### npu modify begin #####
    dataset = dataset.shard(get_rank_size(), get_rank_id())
    ##### npu modify end #####

    # tf.compat.v1.logging.info(
    #     'Sharding the dataset: input_pipeline_id=%d num_input_pipelines=%d' % (
    #         input_context.input_pipeline_id, input_context.num_input_pipelines))
    # dataset = dataset.shard(input_context.num_input_pipelines,
    #                         input_context.input_pipeline_id)

if is_training:
    # Disorder the files.
    dataset = dataset.shuffle(buffer_size=_NUM_TRAIN_FILES)

# cycle_length = 10 Read and deserialize 10 files in parallel. You can increase the value if the CPU
resources are sufficient.
dataset = dataset.interleave(
    tf.data.TFRecordDataset,
    cycle_length=10,
    num_parallel_calls=tf.data.experimental.AUTOTUNE)

return resnet_run_loop.process_record_dataset(
    dataset=dataset,
    is_training=is_training,
    batch_size=batch_size,
    shuffle_buffer=_SHUFFLE_BUFFER,
    parse_record_fn=parse_record_fn,
    num_epochs=num_epochs,
    dtype=dtype,
    datasets_num_private_threads=datasets_num_private_threads,
    drop_remainder=drop_remainder,
    tf_data_experimental_slack=tf_data_experimental_slack,
)

```

3. In **input_fn()** in the training or testing scenario, **drop_remainder** must be set to **True**.

Tweak: **resnet_main()** in **official/r1/resnet/resnet_run_loop.py** (The **input_fn_train()** and **input_fn_eval()** child functions are tweaked.)

```

def input_fn_train(num_epochs, input_context=None):
    ##### npu modify begin #####
    # Use dtype=tf.float16 to improve data transfer performance.
    # In the current version, drop_remainder can only be set to True.
    # batch_size indicates the batch size of a single device instead of the global batch size.
    return input_function(
        is_training=True,
        data_dir=flags_obj.data_dir,

```

```
batch_size=flags_obj.batch_size,
num_epochs=num_epochs,
dtype=tf.float16,
input_context=input_context,
drop_remainder=True)

def input_fn_eval():
    # Use dtype=tf.float16 to improve data transfer performance.
    # In the current version, drop_remainder can only be set to True.
    # batch_size indicates the batch size of a single device instead of the global batch size.
    return input_function(
        is_training=False,
        data_dir=flags_obj.data_dir,
        batch_size=flags_obj.batch_size,
        num_epochs=1,
        dtype=tf.float16,
        input_context=True,
        drop_remainder=True)
##### npu modify end #####

# input_fn() for training and verification in the code are as follows.
# def input_fn_train(num_epochs, input_context=None):
#     return input_function(
#         is_training=True,
#         data_dir=flags_obj.data_dir,
#         batch_size=distribution_utils.per_replica_batch_size(
#             flags_obj.batch_size, flags_core.get_num_gpus(flags_obj)),
#         num_epochs=num_epochs,
#         dtype=flags_core.get_tf_dtype(flags_obj),
#         datasets_num_private_threads=flags_obj.datasets_num_private_threads,
#         input_context=input_context)
#
# def input_fn_eval():
#     return input_function(
#         is_training=False,
#         data_dir=flags_obj.data_dir,
#         batch_size=distribution_utils.per_replica_batch_size(
#             flags_obj.batch_size, flags_core.get_num_gpus(flags_obj)),
#         num_epochs=1,
#         dtype=flags_core.get_tf_dtype(flags_obj))
```

10.1.1.5 Model Building

Build a model the same as the original model. Some code is modified for adaption to improve compute performance. The sample code in this section shows the modifications.

Defining Model Functions

The following uses the model function constructed based on ImageNet as an example. The related APIs are as follows.

Table 10-4 Model APIs

Class or API	Description	Location
imagenet_model_fn()	Model function constructed based on ImageNet.	/official/r1/resnet/ imagenet_main.py

Class or API	Description	Location
learning_rate_with_decay()	Learning rate function. When the number of global steps is less than the configured value, the learning rate increases linearly. When the number of global steps is greater than the configured value, the learning rate decreases by phase.	/official/r1/resnet/resnet_run_loop.py
resnet_model_fn()	Constructs the EstimatorSpec class, which defines the model that is run using Estimator .	/official/r1/resnet/resnet_run_loop.py
ImagenetModel()	Inherited from Model in the resnet_model module. It specifies the network scale, version, number of classes, convolution parameters, and pooling parameters of the ResNet model that is based on ImageNet.	/official/r1/resnet/imagenet_main.py
__call__()	Adds more operations to classify input images, including: 1. performing NHWC to NCHW conversion to accelerate GPU computing; 2. performing the first convolution operation; 3. determining whether to perform batch normalization based on the ResNet version; 4. performing the first pooling; 5. performing block stacking; 6. computing the mean values of the input images; 7. adding fully-connected layers.	/official/r1/resnet/resnet_model.py

Performance Improvement

1. Import the following header file to the `/official/r1/resnet/resnet_run_loop.py` file:

```
from npu_bridge.hcl import hccl_ops
```

2. Check the data type of the input features or images.

Tweak: `resnet_model_fn()` in `official/r1/resnet/resnet_run_loop.py` (The changes are in bold.)

```
##### npu modify begin #####
# Check whether the data type of input features or images is consistent with the data type used
for compute.
if features.dtype != dtype:
    # Change the data type of the features to dtype.
    features = tf.cast(features, dtype)
##### npu modify end #####

# The source code is as follows.
# assert features.dtype == dtype
```

3. Use the float32 type for `labels` to improve accuracy.

Tweak: `resnet_model_fn()` in `official/r1/resnet/resnet_run_loop.py` (The changes are in bold.)

```
##### npu modify begin #####
# Use the float32 type for labels to improve accuracy.
accuracy = tf.compat.v1.metrics.accuracy(tf.cast(labels, tf.float32), predictions['classes'])
##### npu modify end #####

# The accuracy computation code is as follows.
# accuracy = tf.compat.v1.metrics.accuracy(labels, predictions['classes'])

accuracy_top_5 = tf.compat.v1.metrics.mean(
    tf.nn.in_top_k(predictions=logits, targets=labels, k=5, name='top_5_op'))

##### npu modify begin #####
# Calculate accuracy during distributed training.
rank_size = int(os.getenv('RANK_SIZE'))
newaccuracy = (hccl_ops.allreduce(accuracy[0], "sum") / rank_size, accuracy[1])
newaccuracy_top_5 = (hccl_ops.allreduce(accuracy_top_5[0], "sum") / rank_size,
accuracy_top_5[1])
metrics = {'accuracy': newaccuracy,
'accuracy_top_5': newaccuracy_top_5}
##### npu modify end #####

# The metrics source code is as follows.
# metrics = {'accuracy': accuracy,
#           'accuracy_top_5': accuracy_top_5}
```

4. Replace the `max_pooling2d` operator with `max_pool_with_argmax` for better compute performance.

Tweak: `__call__()` in `official/r1/resnet/resnet_model.py` (The changes are in bold.)

```
# Determine whether to perform the first pooling.
if self.first_pool_size:
    ##### npu modify begin #####
    # Replace max_pooling2d with max_pool_with_argmax for better performance.
    inputs, argmax = tf.compat.v1.nn.max_pool_with_argmax(
        input=inputs, ksize=(1, self.first_pool_size, self.first_pool_size, 1),
        strides=(1, self.first_pool_stride, self.first_pool_stride, 1), padding='SAME',
        data_format='NCHW' if self.data_format == 'channels_first' else 'NHWC')
    ##### npu modify end #####

    # The code uses the max_pooling2d() API for pooling.
    # inputs = tf.compat.v1.layers.max_pooling2d(
    #     inputs=inputs, pool_size=self.first_pool_size,
    #     strides=self.first_pool_stride, padding='SAME',
```



```
# data_format=self.data_format)

inputs = tf.identity(inputs, 'initial_max_pool')
```

Configuring Distributed Training

1. Import the following header file to the **official/r1/resnet/resnet_run_loop.py** file:
from npu_bridge.estimator.npu.npu_optimizer import **NPUDistributedOptimizer**
2. Add the distributed training optimizer **NPUDistributedOptimizer**.
Tweak: **resnet_model_fn()** in **official/r1/resnet/resnet_run_loop.py** (The changes are in bold.)

```
if flags.FLAGS.enable_lars:
    optimizer = tf.contrib.opt.LARSOptimizer(
        learning_rate,
        momentum=momentum,
        weight_decay=weight_decay,
        skip_list=['batch_normalization', 'bias'])
else:
    optimizer = tf.compat.v1.train.MomentumOptimizer(
        learning_rate=learning_rate,
        momentum=momentum
    )

##### npu modify begin #####
# Use the distributed training optimizer to encapsulate the single-server optimizer to support
distributed training.
# Add the following content to the source code.
optimizer = NPUDistributedOptimizer(optimizer)
##### npu modify end #####

fp16_implementation = getattr(flags.FLAGS, 'fp16_implementation', None)
if fp16_implementation == 'graph_rewrite':
    optimizer = (
        tf.compat.v1.train.experimental.enable_mixed_precision_graph_rewrite(
            optimizer, loss_scale=loss_scale))
```

10.1.1.6 Run Configuration

Run Configuration

Set run configuration using the **resnet_main()** function.

Table 10-5 Model configuration function

Function	Description	Location
resnet_main()	Main function for run configuration, training, and validation.	/official/r1/resnet/resnet_run_loop.py

1. Import the following header files to the **/official/r1/resnet/resnet_run_loop.py** file:
from npu_bridge.estimator.npu.npu_config import **NPURunConfig**
from npu_bridge.estimator.npu.npu_estimator import **NPUEstimator**
2. Replace **Runconfig** with **NPURunconfig** to configure run parameters.
Tweak: **resnet_main()** in **official/r1/resnet/resnet_run_loop.py**

```
##### npu modify begin #####
# Replace Runconfig with NPURunconfig to adapt to the Ascend AI Processor. Save the checkpoint
every 115200 steps and summary every 10000 times,
# Preprocess data and enable the mixed precision mode to improve the training speed.
run_config = NPURunConfig(
    model_dir=flags_obj.model_dir,
    session_config=session_config,
    save_checkpoints_steps=115200,
    enable_data_pre_proc=True,
    iterations_per_loop=100,
    # enable_auto_mix_precision=True,
    # Set the mixed precision mode.
    precision_mode='allow_mix_precision',
    hcom_parallel=True
)
##### npu modify end #####

# The run configuration in the code is as follows.
# run_config = tf.estimator.RunConfig(
#     train_distribute=distribution_strategy,
#     session_config=session_config,
#     save_checkpoints_secs=60 * 60 * 24,
#     save_checkpoints_steps=None)
```

NOTE

For details about how to set the mixed precision mode
(`precision_mode='allow_mix_precision'`), see [6.2.4 Mixed Precision](#).

3. # Create **NPUEstimator** to replace **tf.estimator.Estimator**.

Tweak: **resnet_main()** in **/official/r1/resnet/resnet_run_loop.py**. The modifications are as follows.

```
# Replace tf.estimator.Estimator with NPUEstimator.
classifier = NPUEstimator(
    model_fn=model_function, model_dir=flags_obj.model_dir, config=run_config,
    params={
        'resnet_size': int(flags_obj.resnet_size),
        'data_format': flags_obj.data_format,
        'batch_size': flags_obj.batch_size,
        'resnet_version': int(flags_obj.resnet_version),
        'loss_scale': flags_core.get_loss_scale(flags_obj,
                                                default_for_fp16=128),
        'dtype': flags_core.get_tf_dtype(flags_obj),
        'fine_tune': flags_obj.fine_tune,
        'num_workers': num_workers,
        'num_gpus': flags_core.get_num_gpus(flags_obj),
    })
# The creation of Estimator in the code is as follows.
# classifier = tf.estimator.Estimator(
#     model_fn=model_function, model_dir=flags_obj.model_dir, config=run_config,
#     warm_start_from=warm_start_settings, params={
#         'resnet_size': int(flags_obj.resnet_size),
#         'data_format': flags_obj.data_format,
#         'batch_size': flags_obj.batch_size,
#         'resnet_version': int(flags_obj.resnet_version),
#         'loss_scale': flags_core.get_loss_scale(flags_obj,
#                                                 default_for_fp16=128),
#         'dtype': flags_core.get_tf_dtype(flags_obj),
#         'fine_tune': flags_obj.fine_tune,
#         'num_workers': num_workers,
#     })
```

10.1.1.7 Training

Training Module

The following table describes the APIs of the training module.

Table 10-6 Training module API

Function	Overview	Location
main()	Main function, for collective communication initialization and model training.	/official/r1/resnet/ imagenet_main.py
run_imagenet()	Model training entry, for input function selection and training result returns.	/official/r1/resnet/ imagenet_main.py
resnet_main()	Main function for run configuration, training, and validation.	/official/r1/resnet/ resnet_run_loop.py

Distributed Training

1. Import the following header files to the **official/r1/resnet/resnet_run_loop.py** file:

```
from npu_bridge.estimator import npu_ops
from tensorflow.core.protobuf import rewriter_config_pb2
```

2. Initialize collective communication before training.

Tweak: **main()** in **official/r1/resnet/imagenet_main.py**. The modifications are as follows.

```
def main():
    ##### npu modify begin #####
    # Call the HCCL API to initialize the NPU.
    # Add the following content to the code.
    init_sess, npu_init = resnet_run_loop.init_npu()
    init_sess.run(npu_init)
    ##### npu modify end #####

    with logger.benchmark_context(flags.FLAGS):
        run_imagenet(flags.FLAGS)
```

3. Define the collective communication initialization function.

Tweak: **init_npu()** in **official/r1/resnet/resnet_run_loop.py**.

```
def resnet_main(flags_obj, model_function, input_function, dataset_name, shape=None):...
##### npu modify begin #####
#Add the following code.
def init_npu():
    """This API is used to manually initialize the NPU.
    Returns:
        `init_sess` npu init session config.
        `npu_init` npu init ops.
    """
    npu_init = npu_ops.initialize_system()
    config = tf.ConfigProto()

    config.graph_options.rewrite_options.remapping = rewriter_config_pb2.RewriterConfig.OFF
    custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
    custom_op.name = "NpuOptimizer"
    # custom_op.parameter_map["precision_mode"].b = True
    custom_op.parameter_map["precision_mode"].s = tf.compat.as_bytes("allow_mix_precision")
    custom_op.parameter_map["use_off_line"].b = True

    init_sess = tf.Session(config=config)
    return init_sess, npu_init
##### npu modify end #####
```

4. Destroy the device allocations after a single training or verification process is complete.

Tweak: **resnet_main()** in **official/r1/resnet/resnet_run_loop.py** (The changes are in bold.)

```
for cycle_index, num_train_epochs in enumerate(schedule):
    tf.compat.v1.logging.info('Starting cycle: %d/%d', cycle_index,
                              int(n_loops))

    if num_train_epochs:
        # Since we are calling classifier.train immediately in each loop, the
        # value of num_train_epochs in the lambda function will not be changed
        # before it is used. So it is safe to ignore the pylint error here
        # pylint: disable=cell-var-from-loop
        classifier.train(
            input_fn=lambda input_context=None: input_fn_train(
                num_train_epochs, input_context=input_context),
            hooks=train_hooks,
            max_steps=flags_obj.max_train_steps)

        ##### npu modify begin #####
        # When a single training process is complete, destroy the NPU allocations. You need to reinitialize
        the NPU before starting a new training process so that the HCCL API is available in the new training
        process:
        # Add the following content to the code.
        init_sess, npu_init = init_npu()
        npu_shutdown = npu_ops.shutdown_system()
        init_sess.run(npu_shutdown)
        init_sess.run(npu_init)
        ##### npu modify end #####

        tf.compat.v1.logging.info('Starting to evaluate.')
        eval_results = classifier.evaluate(input_fn=input_fn_eval,
                                          steps=flags_obj.max_train_steps)

        benchmark_logger.log_evaluation_result(eval_results)

        if model_helpers.past_stop_threshold(
            flags_obj.stop_threshold, eval_results['accuracy']):
            break

        ##### npu modify begin #####
        # When a single training process is complete, destroy the NPU allocations. You need to reinitialize
        the NPU before starting a new training process so that the HCCL API is available in the new training
        process:
        # Add the following content to the code.
        init_sess, npu_init = init_npu()
        npu_shutdown = npu_ops.shutdown_system()
        init_sess.run(npu_shutdown)
        init_sess.run(npu_init)
        ##### npu modify end #####
```

5. Destroy device allocations after training or validation is complete.

After training or validation is complete, call **npu_ops.shutdown_system** to clean up the NPU allocations.

Tweak: **resnet_main()** in **official/r1/resnet/resnet_run_loop.py**. The modifications are as follows.

```
if flags_obj.export_dir is not None:
    # Exports a saved model for the given classifier.
    export_dtype = flags_core.get_tf_dtype(flags_obj)
    if flags_obj.image_bytes_as_serving_input:
        input_receiver_fn = functools.partial(
            image_bytes_serving_input_fn, shape, dtype=export_dtype)
    else:
        input_receiver_fn = export.build_tensor_serving_input_receiver_fn(
            shape, batch_size=flags_obj.batch_size, dtype=export_dtype)
    classifier.export_savedmodel(flags_obj.export_dir, input_receiver_fn,
```

```
strip_default_attrs=True)

##### npu modify begin #####
# After training or validation is complete, destroy the NPU allocations through the
npu_ops.shutdown_system API. Add the following content to the code.
npu_shutdown = npu_ops.shutdown_system()
init_sess.run(npu_shutdown)
##### npu modify end #####

stats = {}
stats['eval_results'] = eval_results
stats['train_hooks'] = train_hooks

return stats
```

Loss Scaling Settings

Set the default value of **loss_scale**.

Tweak: **define_imagenet_flags()** in **official/r1/resnet/imagenet_main.py**. The modifications are as follows.

```
def define_imagenet_flags():
    resnet_run_loop.define_resnet_flags(
        resnet_size_choices=['18', '34', '50', '101', '152', '200'],
        dynamic_loss_scale=True,
        fp16_implementation=True)
    flags.adapt_module_key_flags(resnet_run_loop)
    flags_core.set_defaults(train_epochs=90)

    ##### npu modify begin #####
    # The Ascend AI Processor supports mixed precision training by default. If the value of loss_scale is too
    large, the gradient may explode. If the value is too small, the gradient may vanish.
    # Set loss_scale as follows to avoid the preceding issues.
    flags_core.set_defaults(loss_scale='512')
    ##### npu modify end #####
```

10.1.1.8 Script Execution

Preparing a Dataset

Prepare a dataset and upload it to a directory in the operating environment, for example, **/home/data/resnet50/imagenet**.

Preparing ranktable File

For details about the **ranktable** file example and file description, see [10.4 Device Resource Configuration File Templates](#).

Configuring Environment Variables

For details about the environment variable configuration, see [5.2 Training in a Bare Metal Environment](#).

Running Command

```
python3 /home/official/r1/resnet/imagenet_main.py --batch_size=32 --
hooks=ExamplesPerSecondHook --data_dir=/home/data/resnet50/imagenet
```

10.1.2 More Migration Cases

For more migration cases, visit <https://gitee.com/ascend/modelzoo>.

- [Accuracy Troubleshooting in Ascend 910 ALBERT-Base Migration](#)
- [Dynamic Shape Tuning in Open-Source Network Migration \(Transformer\)](#)
- [Dynamic Shape Tuning in Open-Source Network Migration \(YOLOv4\)](#)
- [Dynamic Shape Tuning in Open-Source Network Migration \(WideDeep\)](#)
- [\[Ascend Model Training Challenge\] Ascend 910 BERT-Base Migration Workflow](#)

10.2 API Reference

10.2.1 Available TensorFlow APIs

The training APIs of the are used with the TensorFlow 1.15 version. This section lists the support of TensorFlow Python APIs.

Supported Python APIs

The following table lists part of the supported Python APIs.

Module	Supported Python API
tf	tf.Dtype
tf	tf.FIFOQueue
tf	tf.FixedLenFeature
tf	tf.FixedLenSequenceFeature
tf	tf.Graph
tf	tf.IndexedSlices
tf	tf.NotDifferentiable
tf	tf.Operation
tf	tf.PaddingFIFOQueue
tf	tf.RandomShuffleQueue
tf	tf.ReaderBase
tf	tf.RegisterGradient
tf	tf.Session
tf	tf.TFRecordReader
tf	tf.Tensor
tf	tf.TensorArray

Module	Supported Python API
tf	tf.TensorShape
tf	tf.VarLenFeature
tf	tf.Variable
tf	tf.VariableAggregation
tf	tf.VariableScope
tf	tf.WholeFileReader
tf	tf.abs
tf	tf.acos
tf	tf.add
tf	tf.add_n
tf	tf.argmax
tf	tf.as_dtype
tf	tf.assert_greater
tf	tf.assert_less_equal
tf	tf.cast
tf	tf.ceil
tf	tf.confusion_matrix
tf	tf.constant_initializer
tf	tf.cos
tf	tf.count_nonzero
tf	tf.cumsum
tf	tf.decode_csv
tf	tf.decode_raw
tf	tf.depth_to_space
tf	tf.diag_part
tf	tf.divide
tf	tf.dynamic_stitch
tf	tf.einsum
tf	tf.equal
tf	tf.exp

Module	Supported Python API
tf	tf.fixed_size_partitioner
tf	tf.flags
tf	tf.floor
tf	tf.floordiv
tf	tf.get_default_graph
tf	tf.get_default_session
tf	tf.get_seed
tf	tf.global_norm
tf	tf.glorot_uniform_initializer
tf	tf.gradients
tf	tf.greater
tf	tf.greater_equal
tf	tf.less
tf	tf.less_equal
tf	tf.log
tf	tf.logical_and
tf	tf.logical_not
tf	tf.logical_or
tf	tf.matmul (int64 is not supported.)
tf	tf.matrix_band_part
tf	tf.maximum
tf	tf.meshgrid
tf	tf.minimum
tf	tf.mod
tf	tf.multinomial
tf	tf.multiply
tf	tf.name_scope
tf	tf.not_equal
tf	tf.one_hot
tf	tf.ones_initializer

Module	Supported Python API
tf	tf.orthogonal_initializer
tf	tf.parse_example
tf	tf.parse_single_example
tf	tf.parse_single_sequence_example
tf	tf.pow
tf	tf.read_file
tf	tf.reduce_all
tf	tf.reduce_any
tf	tf.reduce_logsumexp
tf	tf.reduce_max
tf	tf.reduce_mean
tf	tf.reduce_min
tf	tf.reduce_prod
tf	tf.reduce_sum
tf	tf.reverse_v2
tf	tf rint
tf	tf.round
tf	tf.rsqrt
tf	tf.scalar_mul
tf	tf.sequence_mask
tf	tf.set_random_seed
tf	tf.sigmoid
tf	tf.sign
tf	tf.sin
tf	tf.space_to_depth
tf	tf.sqrt
tf	tf.square
tf	tf.squared_difference
tf	tf.string_join
tf	tf.string_to_hash_bucket_fast

Module	Supported Python API
tf	tf.string_to_number
tf	tf.subtract
tf	tf.tanh
tf	tf.tensordot
tf	tf.tile
tf	tf.timestamp
tf	tf.to_float
tf	tf.to_int32
tf	tf.to_int64
tf	tf.transpose
tf	tf.truediv
tf	tf.truncated_normal
tf	tf.unique
tf	tf.unsorted_segment_min
tf	tf.unsorted_segment_sum
tf	tf.unstack
tf	tf.variance_scaling_initializer
tf	tf.where
tf	tf.while_loop
tf	tf.zeros
tf	tf.zeros_initializer
tf	tf.zeros_like
tf	tf.add_to_collection
tf	tf.AggregationMethod
tf	tf.assign
tf	tf.assign_add
tf	tf.assign_sub
tf	tf.batch_to_space_nd
tf	tf.boolean_mask
tf	tf.case

Module	Supported Python API
tf	tf.clip_by_global_norm
tf	tf.clip_by_norm
tf	tf.clip_by_value
tf	tf.concat
tf	tf.cond
tf	tf.constant
tf	tf.container
tf	tf.control_dependencies
tf	tf.convert_to_tensor
tf	tf.Dimension
tf	tf.div
tf	tf.enable_resource_variables
tf	tf.expand_dims
tf	tf.eye
tf	tf.fill
tf	tf.gather
tf	tf.gather_nd
tf	tf.get_collection
tf	tf.get_collection_ref
tf	tf.get_local_variable
tf	tf.get_variable
tf	tf.get_variable_scope
tf	tf.global_variables
tf	tf.GraphDef
tf	tf.GraphKeys
tf	tf.group
tf	tf.identity
tf	tf.lin_space
tf	tf.local_variables
tf	tf.model_variables

Module	Supported Python API
tf	tf.moving_average_variables
tf	tf.NodeDef
tf	tf.norm
tf	tf.no_op
tf	tf.ones
tf	tf.ones_like
tf	tf.pad
tf	tf.parallel_stack
tf	tf.placeholder
tf	tf.placeholder_with_default
tf	tf.print
tf	tf.range
tf	tf.rank
tf	tf.report_uninitialized_variables
tf	tf.reshape
tf	tf.reverse
tf	tf.scatter_nd
tf	tf.shape
tf	tf.size
tf	tf.slice
tf	tf.split
tf	tf.squeeze
tf	tf.stack
tf	tf.stop_gradient
tf	tf.string_split
tf	tf.ConditionalAccumulator
tf	tf.QueueBase
tf	tf.VariableSynchronization
tf	tf.assert_rank
tf	tf.batch_gather

Module	Supported Python API
tf	tf.broadcast_to
tf	tf.custom_gradient
tf	tf.ensure_shape
tf	tf.import_graph_def
tf	tf.is_finite
tf	tf.is_inf
tf	tf.log1p
tf	tf.print
tf	tf.scatter_sub
tf	tf.variable_creator_scope
tf	tf.make_template
tf	tf.make_tensor_proto
tf	tf.erf
tf	tf.check_numerics
tf	tf.is_nan
tf	tf.linspace
tf	tf.tables_initializer
tf	tf.GradientTape
tf	tf.floormod
tf	tf.realdiv
tf	tf.global_variables_initializer
tf	tf.histogram_fixed_width
tf	tf.truncated_normal_initializer
tf	tf.unique_with_counts
tf	tf.local_variables_initializer
tf	tf.get_logger
tf	tf.TensorSpec
tf	tf.argsort
tf	tf.switch_case
tf	tf.tensor_scatter_nd_update

Module	Supported Python API
tf	tf.sort
tf.app	tf.app.flags
tf.app	tf.app.run
tf.bitwise	tf.bitwise.bitwise_and
tf.bitwise	tf.bitwise.bitwise_or
tf.bitwise	tf.bitwise.bitwise_xor
tf.bitwise	tf.bitwise.invert
tf.bitwise	tf.bitwise.left_shift
tf.bitwise	tf.bitwise.right_shift
tf.compat	tf.compat.as_bytes
tf.compat	tf.compat.as_text
tf.compat	tf.compat.as_str_any
tf.config	tf.ConfigProto
tf.config	tf.config.optimizer
tf.data	tf.data.Dataset
tf.data	tf.data.FixedLengthRecordDataset
tf.data	tf.data.Iterator
tf.data	tf.data.TFRecordDataset
tf.data	tf.data.TextLineDataset
tf.data	tf.data.Options
tf.data	tf.data.experimental.RandomDataset
tf.data	tf.data.experimental.Reducer
tf.data	tf.data.experimental.SqlDataset
tf.data	tf.data.experimental.StatsAggregator
tf.data	tf.data.experimental.dense_to_sparse_batch
tf.data	tf.data.experimental.get_single_element
tf.data	tf.data.experimental.group_by_reducer
tf.data	tf.data.experimental.ignore_errors
tf.data	tf.data.experimental.latency_stats
tf.data	tf.data.experimental.parse_example_dataset

Module	Supported Python API
tf.data	tf.data.experimental.prefetch_to_device
tf.data	tf.data.experimental.set_stats_aggregator
tf.data	tf.data.experimental.shuffle_and_repeat
tf.data	tf.data.experimental.unbatch
tf.distributions	tf.distributions.Categorical
tf.distributions	tf.distributions.Normal
tf.distributions	tf.distributions.Uniform
tf.dtypes	tf.dtypes.as_string
tf.dtypes	tf.dtypes
tf.dtypes	tf.dtypes.complex
tf.dtypes	tf.dtypes.saturate_cast
tf.errors	tf.errors.AbortedError
tf.errors	tf.errors.CancelledError
tf.errors	tf.errors.DeadlineExceededError
tf.errors	tf.errors.InternalError
tf.errors	tf.errors.InvalidArgumentError
tf.errors	tf.errors.NotFoundError
tf.errors	tf.errors.OutOfRangeError
tf.errors	tf.errors.UnavailableError
tf.errors	tf.errors.AlreadyExistsError
tf.errors	tf.errors.DataLossError
tf.errors	tf.errors.FailedPreconditionError
tf.errors	tf.errors.OpError
tf.errors	tf.errors.PermissionDeniedError
tf.errors	tf.errors.ResourceExhaustedError
tf.errors	tf.errors.UnauthenticatedError
tf.errors	tf.errors.UnimplementedError
tf.errors	tf.errors.UnknownError
tf.errors	tf.errors.error_code_from_exception_type
tf.errors	tf.errors.exception_type_from_error_code

Module	Supported Python API
tf.estimator	tf.estimator.Estimator
tf.estimator	tf.estimator.EstimatorSpec
tf.estimator	tf.estimator.ModeKeys
tf.estimator	tf.estimator.WarmStartSettings
tf.estimator	tf.estimator.TrainSpec
tf.estimator	tf.estimator.EvalSpec
tf.estimator	tf.estimator.inputs.numpy_input_fn
tf.estimator	tf.estimator.DNNClassifier
tf.estimator	tf.estimator.DNNRegressor
tf.estimator	tf.estimator.DNNLinearCombinedClassifier
tf.estimator	tf.estimator.LinearClassifier
tf.estimator	tf.estimator.RunConfig
tf.estimator	tf.estimator.export.build_parsing_serving_input_receiver_fn
tf.estimator	tf.estimator.export.build_raw_serving_input_receiver_fn
tf.estimator	tf.estimator.export.PredictOutput
tf.estimator	tf.estimator.export.ServingInputReceiver
tf.estimator	tf.estimator.export.TensorServingInputReceiver
tf.estimator	tf.estimator.FinalExporter
tf.estimator	tf.estimator.LatestExporter
tf.feature_column	tf.feature_column
tf.feature_column	tf.feature_column.bucketized_column
tf.feature_column	tf.feature_column.categorical_column_with_hash_bucket
tf.feature_column	tf.feature_column.categorical_column_with_vocabulary_list
tf.feature_column	tf.feature_column.crossed_column
tf.feature_column	tf.feature_column.embedding_column
tf.feature_column	tf.feature_column.indicator_column
tf.feature_column	tf.feature_column.input_layer
tf.feature_column	tf.feature_column.make_parse_example_spec

Module	Supported Python API
tf.feature_column	tf.feature_column.numeric_column
tf.feature_column	tf.feature_column.categorical_column_with_identity
tf.gfile	tf.gfile.Copy
tf.gfile	tf.gfile.DeleteRecursively
tf.gfile	tf.gfile.Exists
tf.gfile	tf.gfile.GFile
tf.gfile	tf.gfile.Glob
tf.gfile	tf.gfile.IsDirectory
tf.gfile	tf.gfile.ListDirectory
tf.gfile	tf.gfile.MakeDirs
tf.gfile	tf.gfile.Mkdir
tf.gfile	tf.gfile.Open
tf.gfile	tf.gfile.Remove
tf.gfile	tf.gfile.Rename
tf.gfile	tf.gfile.Stat
tf.gfile	tf.gfile.Walk
tf.gfile	tf.gfile.FastGFile
tf.image	tf.image.ResizeMethod
tf.image	tf.image.adjust_brightness
tf.image	tf.image.adjust_contrast
tf.image	tf.image.adjust_hue
tf.image	tf.image.adjust_saturation
tf.image	tf.image.central_crop
tf.image	tf.image.convert_image_dtype
tf.image	tf.image.crop_and_resize
tf.image	tf.image.crop_to_bounding_box
tf.image	tf.image.decode_and_crop_jpeg
tf.image	tf.image.decode_image
tf.image	tf.image.decode_jpeg

Module	Supported Python API
tf.image	tf.image.decode_png
tf.image	tf.image.draw_bounding_boxes
tf.image	tf.image.encode_jpeg
tf.image	tf.image.encode_png
tf.image	tf.image.extract_jpeg_shape
tf.image	tf.image.flip_left_right
tf.image	tf.image.flip_up_down
tf.image	tf.image.grayscale_to_rgb
tf.image	tf.image.non_max_suppression
tf.image	tf.image.non_max_suppression_padded
tf.image	tf.image.pad_to_bounding_box
tf.image	tf.image.per_image_standardization
tf.image	tf.image.random_brightness
tf.image	tf.image.random_contrast
tf.image	tf.image.random_flip_left_right
tf.image	tf.image.random_flip_up_down
tf.image	tf.image.random_hue
tf.image	tf.image.random_saturation
tf.image	tf.image.resize_area
tf.image	tf.image.resize_bicubic
tf.image	tf.image.resize_bilinear
tf.image	tf.image.resize_image_with_crop_or_pad
tf.image	tf.image.resize_images
tf.image	tf.image.resize_nearest_neighbor
tf.image	tf.image.rgb_to_grayscale
tf.image	tf.image.rot90
tf.image	tf.image.sample_distorted_bounding_box
tf.image	tf.image.decode_bmp
tf.image	tf.image.decode_gif
tf.initializers	tf.initializers.he_uniform

Module	Supported Python API
tf.initializers	tf.initializers.variables
tf.keras	tf.keras.layers.Lambda
tf.keras	tf.keras.utils.get_file
tf.keras	tf.keras.estimator
tf.keras	tf.keras.estimator.model_to_estimator
tf.layers	tf.layers.Dense
tf.layers	tf.layers.Layer
tf.layers	tf.layers.average_pooling2d
tf.layers	tf.layers.batch_normalization
tf.layers	tf.layers.conv2d
tf.layers	tf.layers.dense
tf.layers	tf.layers.dropout
tf.layers	tf.layers.flatten
tf.layers	tf.layers.max_pooling2d
tf.layers	tf.layers.separable_conv2d
tf.layers	tf.layers.InputSpec
tf.logging	tf.logging.debug
tf.logging	tf.logging.error
tf.logging	tf.logging.fatal
tf.logging	tf.logging.get_verbosity
tf.logging	tf.logging.info
tf.logging	tf.logging.log
tf.logging	tf.logging.log_every_n
tf.logging	tf.logging.set_verbosity
tf.logging	tf.logging.warn
tf.logging	tf.logging.warning
tf.losses	tf.losses.Reduction
tf.losses	tf.losses.add_loss
tf.losses	tf.losses.compute_weighted_loss
tf.losses	tf.losses.get_losses

Module	Supported Python API
tf.losses	tf.losses.get_total_loss
tf.losses	tf.losses.huber_loss
tf.losses	tf.losses.log_loss
tf.losses	tf.losses.mean_squared_error
tf.losses	tf.losses.sigmoid_cross_entropy
tf.losses	tf.losses.softmax_cross_entropy
tf.losses	tf.losses.sparse_softmax_cross_entropy
tf.losses	tf.losses.get_regularization_loss
tf.manip	tf.manip.roll
tf.manip	tf.manip.space_to_batch_nd
tf.math	tf.math.exp
tf.math	tf.math.acosh
tf.math	tf.math.asinh
tf.math	tf.math.atan
tf.math	tf.math.atan2
tf.math	tf.math.atanh
tf.math	tf.math.cosh
tf.math	tf.math.expm1
tf.math	tf.math.log1p
tf.math	tf.math.reciprocal
tf.math	tf.math.segment_max
tf.math	tf.math.sinh
tf.math	tf.math.tan
tf.math	tf.math.unsorted_segment_mean
tf.math	tf.math.unsorted_segment_prod
tf.math	tf.math.unsorted_segment_sqrt_n
tf.math	tf.math.xdivy
tf.math	tf.math.xlogy
tf.metrics	tf.metrics.accuracy
tf.metrics	tf.metrics.auc

Module	Supported Python API
tf.metrics	tf.metrics.mean
tf.metrics	tf.metrics.mean_iou
tf.metrics	tf.metrics.recall_at_k
tf.metrics	tf.metrics.root_mean_squared_error
tf.metrics	tf.metrics
tf.metrics	tf.metrics.false_negatives
tf.metrics	tf.metrics.false_negatives_at_thresholds
tf.metrics	tf.metrics.false_positives
tf.metrics	tf.metrics.false_positives_at_thresholds
tf.metrics	tf.metrics.mean_absolute_error
tf.metrics	tf.metrics.mean_cosine_distance
tf.metrics	tf.metrics.mean_per_class_accuracy
tf.metrics	tf.metrics.mean_relative_error
tf.metrics	tf.metrics.mean_tensor
tf.metrics	tf.metrics.percentage_below
tf.metrics	tf.metrics.precision
tf.metrics	tf.metrics.precision_at_thresholds
tf.metrics	tf.metrics.precision_at_top_k
tf.metrics	tf.metrics.recall
tf.metrics	tf.metrics.recall_at_thresholds
tf.metrics	tf.metrics.sensitivity_at_specificity
tf.metrics	tf.metrics.sparse_average_precision_at_k
tf.metrics	tf.metrics.sparse_precision_at_k
tf.metrics	tf.metrics.true_negatives
tf.metrics	tf.metrics.true_negatives_at_thresholds
tf.metrics	tf.metrics.true_positives
tf.metrics	tf.metrics.true_positives_at_thresholds
tf.nest	tf.nest.flatten
tf.nest	tf.nest.map_structure
tf.nest	tf.nest.pack_sequence_as

Module	Supported Python API
tf.nn	tf.nn.avg_pool
tf.nn	tf.nn.batch_normalization
tf.nn	tf.nn.bias_add
tf.nn	tf.nn.bidirectional_dynamic_rnn
tf.nn	tf.nn.conv2d
tf.nn	tf.nn.convolution
tf.nn	tf.nn.depthwise_conv2d
tf.nn	tf.nn.dropout
tf.nn	tf.nn.dynamic_rnn
tf.nn	tf.nn.embedding_lookup
tf.nn	tf.nn.fused_batch_norm
tf.nn	tf.nn.in_top_k
tf.nn	tf.nn.l2_loss
tf.nn	tf.nn.l2_normalize
tf.nn	tf.nn.leaky_relu
tf.nn	tf.nn.log_softmax
tf.nn	tf.nn.lrn
tf.nn	tf.nn.max_pool
tf.nn	tf.nn.moments
tf.nn	tf.nn.relu
tf.nn	tf.nn.relu6
tf.nn	tf.nn.rnn_cell.BasicLSTMCell
tf.nn	tf.nn.rnn_cell.GRUCell
tf.nn	tf.nn.rnn_cell.LSTMCell
tf.nn	tf.nn.rnn_cell.LSTMStateTuple
tf.nn	tf.nn.rnn_cell.RNNCell
tf.nn	tf.nn.sampled_softmax_loss
tf.nn	tf.nn.separable_conv2d
tf.nn	tf.nn.sigmoid
tf.nn	tf.nn.sigmoid_cross_entropy_with_logits

Module	Supported Python API
tf.nn	tf.nn.softmax
tf.nn	tf.nn.softmax_cross_entropy_with_logits
tf.nn	tf.nn.softmax_cross_entropy_with_logits_v2
tf.nn	tf.nn.softplus
tf.nn	tf.nn.softsign
tf.nn	tf.nn.sparse_softmax_cross_entropy_with_logits
tf.nn	tf.nn.static_rnn
tf.nn	tf.nn.tanh
tf.nn	tf.nn.top_k
tf.nn	tf.nn.xw_plus_b
tf.nn	tf.nn.zero_fraction
tf.nn	tf.nn.crelu
tf.nn	tf.nn.elu
tf.nn	tf.nn.max_pool_with_argmax
tf.nn	tf.nn.normalize_moments
tf.nn	tf.nn.rnn_cell
tf.nn	tf.nn.rnn_cell.BasicRNNCell
tf.nn	tf.nn.rnn_cell.MultiRNNCell
tf.nn	tf.nn.selu
tf.nn	tf.nn.static_bidirectional_rnn
tf.nn	tf.nn.sufficient_statistics
tf.python_io	tf.python_io.TFRecordCompressionType
tf.python_io	tf.python_io.TFRecordWriter
tf.python_io	tf.python_io.tf_record_iterator
tf.python_io	tf.python_io.TFRecordOptions
tf.random	tf.random_crop
tf.random	tf.random_normal
tf.random	tf.random_normal_initializer
tf.random	tf.random_shuffle
tf.random	tf.random_uniform

Module	Supported Python API
tf.random	tf.random_uniform_initializer
tf.resource_loader	tf.resource_loader.get_data_files_path
tf.saved_model	tf.saved_model
tf.saved_model	tf.saved_model.builder.SavedModelBuilder
tf.saved_model	tf.saved_model.signature_constants
tf.saved_model	tf.saved_model.tag_constants
tf.saved_model	tf.saved_model.utils
tf.saved_model	tf.saved_model.save
tf.saved_model	tf.saved_model.Builder
tf.saved_model	tf.saved_model.loader.load
tf.saved_model	tf.saved_model.signature_def_utils.build_signature_def
tf.saved_model	tf.saved_model.utils.build_tensor_info
tf.sparse	tf.sparse.to_dense
tf.sparse	tf.sparse_tensor_to_dense
tf.sparse	tf.SparseConditionalAccumulator
tf.spectral	tf.spectral.dct
tf.spectral	tf.spectral.idct
tf.strings	tf.strings.substr
tf.strings	tf.strings.to_number
tf.summary	tf.Summary
tf.summary	tf.Summary.Image
tf.summary	tf.Summary.Value
tf.summary	tf.summary.FileWriter
tf.summary	tf.summary.FileWriterCache
tf.summary	tf.summary.histogram
tf.summary	tf.summary.image
tf.summary	tf.summary.merge
tf.summary	tf.summary.merge_all
tf.summary	tf.summary.scalar

Module	Supported Python API
tf.summary	tf.summary.text
tf.sysconfig	tf.sysconfig.get_compile_flags
tf.sysconfig	tf.sysconfig.get_link_flags
tf.test	tf.test.TestCase
tf.test	tf.test.get_temp_dir
tf.test	tf.test.main
tf.train	tf.train.Scaffold
tf.train	tf.trainable_variables
tf.train	tf.train.cosine_decay
tf.train	tf.train.exponential_decay
tf.train	tf.train.piecewise_constant
tf.train	tf.train.polynomial_decay
tf.train	tf.train.create_global_step
tf.train	tf.train.get_global_step
tf.train	tf.train.get_or_create_global_step
tf.train	tf.train.global_step
tf.train	tf.train.Saver
tf.train	tf.train.Checkpoint
tf.train	tf.train.CheckpointSaverHook
tf.train	tf.train.checkpoint_exists
tf.train	tf.train.NewCheckpointReader
tf.train	tf.train.get_checkpoint_state
tf.train	tf.train.init_from_checkpoint
tf.train	tf.train.latest_checkpoint
tf.train	tf.train.load_checkpoint
tf.train	tf.train.import_meta_graph
tf.train	tf.train.list_variables
tf.train	tf.train.BytesList
tf.train	tf.train.FloatList
tf.train	tf.train.Int64List

Module	Supported Python API
tf.train	tf.train.Feature
tf.train	tf.train.Features
tf.train	tf.train.FeatureList
tf.train	tf.train.FeatureLists
tf.train	tf.train.Example
tf.train	tf.train.SequenceExample
tf.train	tf.train.Optimizer
tf.train	tf.train.AdadeltaOptimizer
tf.train	tf.train.AdamOptimizer
tf.train	tf.train.GradientDescentOptimizer
tf.train	tf.train.MomentumOptimizer
tf.train	tf.train.RMSPropOptimizer
tf.train	tf.train.MonitoredSession
tf.train	tf.train.Coordinator
tf.train	tf.train.SessionRunHook
tf.train	tf.train.SessionRunArgs
tf.train	tf.train.LoggingTensorHook
tf.train	tf.train.ProfilerHook
tf.train	tf.train.SummarySaverHook
tf.train	tf.train.write_graph
tf.train	tf.train.SecondOrStepTimer
tf.train	tf.train.AdagradOptimizer
tf.train	tf.train.ChiefSessionCreator
tf.train	tf.train.ExponentialMovingAverage
tf.train	tf.train.FtrlOptimizer
tf.train	tf.train.MonitoredTrainingSession
tf.train	tf.train.SyncReplicasOptimizer
tf.train	tf.train.cosine_decay_restarts
tf.train	tf.train.generate_checkpoint_state_proto
tf.train	tf.train.linear_cosine_decay

Module	Supported Python API
tf.train	tf.train.load_variable
tf.train	tf.train.start_queue_runners
tf.train	tf.train.summary_iterator
tf.train	tf.train.assert_global_step
tf.train	tf.train.queue_runner
tf.train	tf.train.SessionRunHook
tf.train	tf.train.CheckpointManager
tf.train	tf.train.CheckpointSaverHook
tf.train	tf.train.checkpoints_iterator
tf.train	tf.train.piecewise_constant_decay

Unsupported Python APIs

The following table lists part of the unsupported Python APIs.

Module	Unsupported Python API
tf	tf.py_func
tf.metrics	tf.metrics.specificity_at_sensitivity
tf.nn	tf.nn.ctc_loss
tf	tf.device
tf	tf.SparseTensor
tf	tf.map_fn
tf	tf.reset_default_graph
tf	tf.GPUOptions
tf	tf.GPUOptions.Experimental
tf	tf.GPUOptions.Experimental.VirtualDevices
tf	tf.RunOptions.Experimental
tf	tf.executing_eagerly
tf	tf.enable_eager_execution
tf	tf.autograph
tf	tf.distribute

Module	Unsupported Python API
tf	tf.disable_v2_tensorshape
tf	tf.enable_control_flow_v2
tf	tf.enable_tensor_equality
tf	tf.enable_v2_behavior
tf	tf.enable_v2_tensorshape
tf	tf.CriticalSection
tf	tf.IndexedSlicesSpec
tf	tf.Module
tf	tf.OptionalSpec
tf	tf.RaggedTensor
tf	tf.function
tf	tf.disable_control_flow_v2
tf	tf.disable_eager_execution
tf	tf.disable_tensor_equality
tf	tf.disable_v2_behavior
tf.config	tf.ConfigProto.Experimental
tf.config	tf.ConfigProto.Experimental
tf.config	tf.config.get_soft_device_placement
tf.config	tf.config.optimizer.get_experimental_options
tf.config	tf.config.optimizer.get_jit
tf.config	tf.config.optimizer.set_experimental_options
tf.config	tf.config.optimizer.set_jit
tf.config	tf.config.set_soft_device_placement
tf.estimator	tf.estimator.train_and_evaluate
tf.estimator	tf.estimator.tpu.TPUConfig
tf.estimator	tf.estimator.tpu.RunConfig
tf.estimator	tf.estimator.tpu.TPUEstimatorSpec
tf.estimator	tf.estimator.tpu.InputPipelineConfig
tf.estimator	tf.estimator.tpu.TPUEstimator
tf.keras	tf.keras.models.Sequential

Module	Unsupported Python API
tf.keras	tf.keras.utils.multi_gpu_model
tf.keras	tf.keras.preprocessing.image.DirectoryIterator
tf.keras	tf.keras.preprocessing.text.hashing_trick
tf.keras	tf.keras.preprocessing.text.one_hot
tf.keras	tf.keras.preprocessing.text.text_to_word_sequence
tf.keras	tf.keras.utils.model_to_dot
tf.keras	tf.keras.preprocessing.sequence.pad_sequences
tf.keras	tf.keras.preprocessing.sequence.skipgrams
tf.keras	tf.keras.preprocessing.text.Tokenizer
tf.keras	tf.keras.preprocessing.image.ImageDataGenerator
tf.keras	tf.keras.preprocessing.image.Iterator
tf.keras	tf.keras.preprocessing.image.NumpyArrayIterator
tf.keras	tf.keras.preprocessing.image.apply_affine_transform
tf.keras	tf.keras.preprocessing.image.apply_brightness_shift
tf.keras	tf.keras.preprocessing.image.apply_channel_shift
tf.keras	tf.keras.preprocessing.image.array_to_img
tf.keras	tf.keras.preprocessing.image.img_to_array
tf.keras	tf.keras.preprocessing.image.load_img
tf.keras	tf.keras.preprocessing.image.random_brightness
tf.keras	tf.keras.preprocessing.image.random_zoom
tf.keras	tf.keras.preprocessing.image.save_img
tf.keras	tf.keras.preprocessing.sequence.TimeseriesGenerator
tf.keras	tf.keras.preprocessing.sequence.make_sampling_table
tf.keras	tf.keras.preprocessing.image.random_rotation
tf.keras	tf.keras.preprocessing.image.random_shift
tf.keras	tf.keras.preprocessing.image.random_shear
tf.keras	tf.keras.preprocessing.image.random_channel_shift
tf.math	tf.math.segment_mean

Module	Unsupported Python API
tf.math	tf.math.segment_min
tf.math	tf.math.segment_prod
tf.math	tf.math.segment_sum
tf.profiler	tf.profiler.ProfileOptionBuilder
tf.profiler	tf.profiler.profile
tf.profiler	tf.profiler.Profiler
tf.profiler	tf.profiler.AdviceProto
tf.profiler	tf.profiler.AdviceProto.Checker
tf.profiler	tf.profiler.AdviceProto.CheckersEntry
tf.profiler	tf.profiler.GraphNodeProto
tf.profiler	tf.profiler.GraphNodeProto.InputShapesEntry
tf.profiler	tf.profiler.MultiGraphNodeProto
tf.profiler	tf.profiler.OpLogProto
tf.profiler	tf.profiler.OpLogProto.IdToStringEntry
tf.profiler	tf.profiler.advise
tf.profiler	tf.profiler.write_op_log
tf.summary	tf.summary.all_v2_summary_ops
tf.train	tf.train.ClusterSpec
tf.train	tf.train.replica_device_setter
tf.train	tf.train.Server
tf.train	tf.train.ClusterDef
tf.train	tf.train.JobDef
tf.train	tf.train.JobDef.TasksEntry

Deprecated Python APIs

The following table lists the deprecated TensorFlow Python APIs. You are not advised to use them.

Module	Deprecated Python API
tf	tf.op_scope
tf	tf.all_variables

Module	Deprecated Python API
tf	tf.colocate_with
tf	tf.initialize_all_variables
tf	tf.initialize_variables
tf	tf.Print
tf	tf.sparse_to_dense
tf	tf.initialize_all_tables
tf	tf.initialize_local_variables
tf	tf.disable_resource_variables
tf.contrib	tf.contrib
tf.contrib	tf.contrib.cluster_resolver.TPUClusterResolver
tf.contrib	tf.contrib.cudnn_rnn.CudnnCompatibleLSTMCell
tf.contrib	tf.contrib.data.batch_and_drop_remainder
tf.contrib	tf.contrib.data.bucket_by_sequence_length
tf.contrib	tf.contrib.data.group_by_window
tf.contrib	tf.contrib.data.map_and_batch
tf.contrib	tf.contrib.data.parallel_interleave
tf.contrib	tf.contrib.distribute.MirroredStrategy
tf.contrib	tf.contrib.distribute.OneDeviceStrategy
tf.contrib	tf.contrib.framework.add_arg_scope
tf.contrib	tf.contrib.framework.arg_scope
tf.contrib	tf.contrib.framework.deprecated
tf.contrib	tf.contrib.framework.filter_variables
tf.contrib	tf.contrib.framework.get_variables
tf.contrib	tf.contrib.framework.is_tensor
tf.contrib	tf.contrib.framework.model_variable
tf.contrib	tf.contrib.image
tf.contrib	tf.contrib.layers
tf.contrib	tf.contrib.layers.flatten
tf.contrib	tf.contrib.layers.fully_connected
tf.contrib	tf.contrib.layers.l2_regularizer

Module	Deprecated Python API
tf.contrib	tf.contrib.layers.optimize_loss
tf.contrib	tf.contrib.layers.summarize_collection
tf.contrib	tf.contrib.layers.variance_scaling_initializer
tf.contrib	tf.contrib.layers.xavier_initializer
tf.contrib	tf.contrib.learn.Experiment
tf.contrib	tf.contrib.learn.ModeKeys
tf.contrib	tf.contrib.learn.utils
tf.contrib	tf.contrib.lookup.HashTable
tf.contrib	tf.contrib.lookup.KeyValueTensorInitializer
tf.contrib	tf.contrib.opt.LazyAdamOptimizer
tf.contrib	tf.contrib.opt.MovingAverageOptimizer
tf.contrib	tf.contrib.quantize
tf.contrib	tf.contrib.quantize.create_eval_graph
tf.contrib	tf.contrib.quantize.create_training_graph
tf.contrib	tf.contrib.rnn.DropoutWrapper
tf.contrib	tf.contrib.rnn.LSTMCell
tf.contrib	tf.contrib.slim
tf.contrib	tf.contrib.tfprof
tf.contrib	tf.contrib.tpu
tf.contrib	tf.contrib.tpu.CrossShardOptimizer
tf.contrib	tf.contrib.tpu.RunConfig
tf.contrib	tf.contrib.tpu.TPUConfig
tf.contrib	tf.contrib.tpu.TPUEstimator
tf.contrib	tf.contrib.tpu.TPUEstimatorSpec
tf.contrib	tf.contrib.tpu.bfloat16_scope
tf.contrib	tf.contrib.tpu.initialize_system
tf.contrib	tf.contrib.tpu.rewrite
tf.contrib	tf.contrib.tpu.shutdown_system
tf.contrib	tf.contrib.training.GreedyLoadBalancingStrategy
tf.contrib	tf.contrib.training.HParams

Module	Deprecated Python API
tf.contrib	tf.contrib.training.StopAfterNEvalsHook
tf.contrib	tf.contrib.training.SummaryAtEndHook
tf.contrib	tf.contrib.training.checkpoints_iterator
tf.contrib	tf.contrib.training.evaluate_repeatedly
tf.contrib	tf.contrib.all_reduce
tf.contrib	tf.contrib.data.padded_batch_and_drop_remainder
tf.contrib	tf.contrib.data.unbatch
tf.contrib	tf.contrib.estimator.clip_gradients_by_norm
tf.contrib	tf.contrib.framework.argsort
tf.contrib	tf.contrib.framework.assign_from_checkpoint_fn
tf.contrib	tf.contrib.framework.get_variables_to_restore
tf.contrib	tf.contrib.framework.local_variable
tf.contrib	tf.contrib.framework.smart_cond
tf.contrib	tf.contrib.layers.apply_regularization
tf.contrib	tf.contrib.layers.batch_norm
tf.contrib	tf.contrib.layers.bow_encoder
tf.contrib	tf.contrib.layers.conv2d
tf.contrib	tf.contrib.layers.layer_norm
tf.contrib	tf.contrib.learn
tf.contrib	tf.contrib.learn.read_batch_features
tf.contrib	tf.contrib.lookup.index_table_from_file
tf.contrib	tf.contrib.lookup.index_table_from_tensor
tf.contrib	tf.contrib.lookup.index_to_string_table_from_file
tf.contrib	tf.contrib.metrics.aggregate_metric_map
tf.contrib	tf.contrib.metrics.streaming_pearson_correlation
tf.contrib	tf.contrib.opt.LARSOptimizer
tf.contrib	tf.contrib.rnn.DeviceWrapper
tf.contrib	tf.contrib.rnn.LayerNormBasicLSTMCell
tf.contrib	tf.contrib.rnn.MultiRNNCell
tf.contrib	tf.contrib.rnn.NASCell

Module	Deprecated Python API
tf.contrib	tf.contrib.rnn.ResidualWrapper
tf.contrib	tf.contrib.saved_model
tf.contrib	tf.contrib.seq2seq.AttentionWrapper
tf.contrib	tf.contrib.seq2seq.AttentionWrapperState
tf.contrib	tf.contrib.seq2seq.BahdanauAttention
tf.contrib	tf.contrib.seq2seq.BasicDecoder
tf.contrib	tf.contrib.seq2seq.BeamSearchDecoder
tf.contrib	tf.contrib.seq2seq.GreedyEmbeddingHelper
tf.contrib	tf.contrib.seq2seq.Helper
tf.contrib	tf.contrib.seq2seq.LuongAttention
tf.contrib	tf.contrib.seq2seq.SampleEmbeddingHelper
tf.contrib	tf.contrib.seq2seq.TrainingHelper
tf.contrib	tf.contrib.seq2seq.dynamic_decode
tf.contrib	tf.contrib.seq2seq.tile_batch
tf.contrib	tf.contrib.signal.inverse_stft
tf.contrib	tf.contrib.signal.stft
tf.contrib	tf.contrib.staging.StagingArea
tf.contrib	tf.contrib.summary
tf.contrib	tf.contrib.tensorboard
tf.contrib	tf.contrib.training.byte_size_load_fn
tf.graph_util	tf.graph_util.convert_variables_to_constants
tf.graph_util	tf.graph_util.extract_sub_graph
tf.graph_util	tf.graph_util.must_run_on_cpu
tf.graph_util	tf.graph_util.remove_training_nodes
tf.graph_util	tf.graph_util.tensor_shape_from_node_def_name
tf.saved_model	tf.saved_model.get_tensor_from_tensor_info
tf.saved_model	tf.saved_model.main_op.main_op_with_restore
tf.saved_model	tf.saved_model.main_op_with_restore
tf.saved_model	tf.saved_model.simple_save
tf.saved_model	tf.saved_model.utils.get_tensor_from_tensor_info

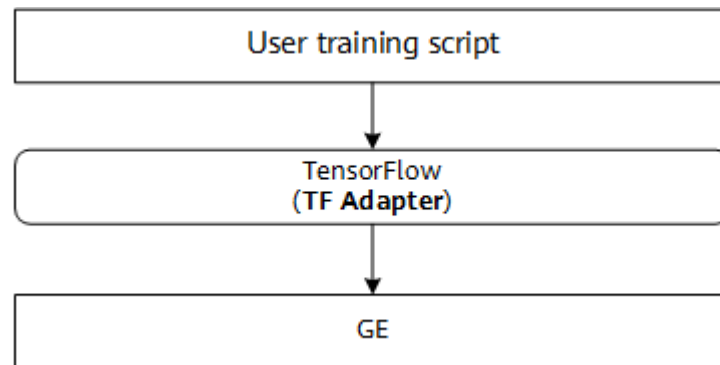
Module	Deprecated Python API
tf.train	tf.train.QueueRunner
tf.train	tf.train.add_queue_runner
tf.train	tf.train.batch
tf.train	tf.train.batch_join
tf.train	tf.train.range_input_producer
tf.train	tf.train.slice_input_producer
tf.train	tf.train.Supervisor
tf.train	tf.train.shuffle_batch
tf.train	tf.train.string_input_producer
tf.train	tf.train.input_producer
tf.train	tf.train.do_quantize_training_on_graphdef
tf.train	tf.train.limit_epochs
tf.train	tf.train.maybe_batch
tf.train	tf.train.maybe_batch_join
tf.train	tf.train.maybe_shuffle_batch
tf.train	tf.train.maybe_shuffle_batch_join
tf.train	tf.train.shuffle_batch_join
tf.train	tf.train.queue_runner.QueueRunner
tf.train	tf.train.queue_runner.add_queue_runner

10.2.2 TF Adapter APIs

10.2.2.1 Overview

Since TF Adapter provides APIs adapted to the TensorFlow framework, you can develop TensorFlow-based training scripts.

Figure 10-1 TF Adapter



To use TF Adapter, the TFPlugin software package must be installed.

- If the **--pylocal** mode is used for the installation of TFPlugin, the corresponding .whl is installed in **/tfplugin/python/site-packages/** in the TFPlugin installation path.
- If the **--pylocal** mode is not used to for the installation of TFPlugin, the corresponding .whl is installed in the local Python path.

10.2.2.2 npu_bridge.estimator.npu.npu_config

10.2.2.2.1 NPURunConfig Constructor

Prototype

```
def __init__(self,  
iterations_per_loop=1,  
profiling_config=None,  
model_dir=None,  
tf_random_seed=None,  
save_summary_steps=0,  
save_checkpoints_steps=None,  
save_checkpoints_secs=None,  
session_config=None,  
keep_checkpoint_max=5,  
keep_checkpoint_every_n_hours=10000,  
log_step_count_steps=100,  
distribute=None,  
enable_data_pre_proc=True,  
precision_mode=None,  
enable_reduce_precision=False,
```

```
variable_format_optimize=True,  
mix_compile_mode=False,  
hcom_parallel=False,  
graph_memory_max_size=None,  
variable_memory_max_size=None,  
auto_tune_mode=None,  
dump_config=None,  
stream_max_parallel_num=None,  
is_tailing_optimization=False,  
horovod_mode = False,  
graph_run_mode = 1,  
op_debug_level = 0,  
enable_scope_fusion_passes = None,  
enable_exception_dump = 0,  
op_select_implmode=None,  
optypelist_for_implmode=None,  
dynamic_input_config=None,  
mstune_mode=None,  
work_path=None,  
buffer_optimize="l2_optimize",  
enable_small_channel=0,  
fusion_switch_file=None,  
enable_compress_weight=False,  
compress_weight_conf=None,  
op_compiler_cache_mode=None,  
op_compiler_cache_dir=None,  
debug_dir=None  
)
```

Description

Constructor of the **NPURunConfig** class, which inherits the **RunConfig** class and can call the native APIs of the base class.

Restrictions

In multi-device training scenarios, the **save_checkpoints_secs** parameter cannot be used to save files by time.

Parameters

Parameter	Description
NPURunConfig parameters inherited from RunConfig :	
model_dir	Path of the model. Defaults to None . If model_dir set in NPURunConfig and NPUEstimator are different, an error is reported. If either NPURunConfig or NPUEstimator is configured with model_dir , the configured path applies. If neither NPURunConfig nor NPUEstimator is configured with model_dir , a model_dir_XXXXXXXXXX directory is created in the current script execution path to save the model file.
tf_random_seed	Seed of the initialization variable. Defaults to None .
save_summary_steps	Interval (in steps) for saving the summary file. Defaults to 0 . Applies only to the scenario where iterations_per_loop = 1 . If iterations_per_loop > 1 , the configured value may not be saved. For details about how to save information, see "Log and Summary Operators".

Parameter	Description
save_checkpoints_steps	Interval (in steps) for saving the checkpoint file. Defaults to None. - This parameter and save_checkpoints_secs are mutually exclusive. - If save_checkpoints_steps and save_checkpoints_secs are set to None, the checkpoint file is saved every 100 steps. - If the value of iterations_per_loop is greater than 1, the value of save_checkpoints_steps must be a positive integer multiple of iterations_per_loop; otherwise, checkpoint data is not saved as defined by save_checkpoints_steps. To save the checkpoint data on only a specific device, modify the training script as follows: Original TensorFlow code: <pre>self_classifier=tf.estimator.Estimator(model_fn=cnn_model_fn, model_dir=self_model_dir, config=tf.estimator.RunConfig(save_checkpoints_steps=50 if hvd.rank() == 0 else None, keep_checkpoint_max=1))</pre> Code after migration: <pre>self_classifier=NPUEstimator(model_fn=cnn_model_fn, model_dir=self_model_dir, config=tf.estimator.NPURunConfig(save_checkpoints_steps=50 if rank_id == 0 else 0, keep_checkpoint_max=1))</pre>
save_checkpoints_secs	Interval (in seconds) for saving the checkpoint file. Defaults to None. This parameter and save_checkpoints_steps are mutually exclusive.
session_config	ConfigProto format object for session configuration. Defaults to None.
keep_checkpoint_max	Maximum number of checkpoint files that can be stored. Defaults to 5.
keep_checkpoint_every_n_hours	Number of hours to retain the checkpoint file. Defaults to 10000. This function can be disabled. To use this function, set keep_checkpoint_max to a large value.

Parameter	Description
log_step_count_steps	Interval (in steps) for recording the global step and loss values. Defaults to 100 . Applies only to the scenario where iterations_per_loop = 1 . If iterations_per_loop > 1 , the configured value may not be saved. For details about how to save information, see "Log and Summary Operators".
NPURunConfig does not support the following parameters:	
train_distribute	Indicates whether to perform distributed training. The distributed configuration is specified by experimental_distribute . TF Adapter encapsulates the single-server training optimizer using the NPUDistributedOptimizer class, to construct an NPU distributed training optimizer.
device_fn	Obtains the function of the Device field of each operation.
protocol	Protocol used to start the server. This parameter is optional. If the parameter is empty, the gRPC is used by default.
eval_distribute	Indicates whether to perform distributed evaluation. The distributed configuration is specified by experimental_distribute .
experimental_distribute	Distributed configuration
The following NPURunConfig parameters are added:	

Parameter	Description
iterations_per_loop	Number of iterations per training loop performed on the per sess.run() call. Defaults to 1. The total number of training iterations per loop must be an integer multiple of the value of iterations_per_loop. Training is performed according to the specified number of iterations per loop (iterations_per_loop) on the and then the result is returned to the host. This parameter can save unnecessary interactions between the host and device and reduce the training time consumption. In mixed computing mode (mix_compile_mode is set to True), this parameter must be set to 1. Note: When iterations_per_loop is set to a value greater than 1, the total number of training iterations set by the user may be different from the actual total number of iterations due to issues such as loop offloading and loss scale overflow. Example: config = NPURunConfig(iterations_per_loop=1000)
profiling_config	Profiling switch. Before creating NPURunConfig, you can instantiate a ProfilingConfig class to configure profiling. For details about the constructor of the ProfilingConfig class, see 10.2.2.2.2 ProfilingConfig Constructor. Example: config = NPURunConfig(profiling_config=profiling_config)
dump_config	Dump switch. Before creating NPURunConfig, you can instantiate a DumpConfig class for dump configuration. For details about the constructor of the DumpConfig class, see 10.2.2.2.3 DumpConfig Constructor. Example: config = NPURunConfig(dump_config=dump_config)
enable_data_pre_proc	Indicates whether to offload the data preprocessing workload to the device side. • True (default): enabled • False: disabled Example: config = NPURunConfig(enable_data_pre_proc=True)

Parameter	Description
precision_mode	A string for the operator precision mode. • allow_fp32_to_fp16 or None: If an operator does not support the float32 data type, the precision is reduced to float16. By default, this mode is selected. • force_fp16: If an operator supports both float16 and float32 data types, float16 is forcibly selected. • must_keep_origin_dtype: The original precision is retained. • allow_mix_precision: Mixed precision is enabled. For operators of the float32 data type on a network, the precision of some float32 operators can be automatically reduced to float16 based on the built-in optimization policy. In this way, the system performance is improved and the memory usage is reduced with little accuracy loss. After this function is enabled, you can enable loss scaling to compensate for the accuracy drop caused by precision reduction. For details about the related APIs, see 10.2.2.6.1 NPULossScaleOptimizer Constructor. Example: <pre>config = NPURunConfig(precision_mode="allow_mix_precision")</pre>
enable_reduce_precision	This parameter is not supported in the current version.
variable_format_optimize	Variable format optimization enable • True (default): enabled • False: disabled To improve training efficiency, during the initialization of network variables, the variables are converted into a data format that is more suitable for running on the . However, this function should be disabled in special scenarios. Example: <pre>config = NPURunConfig(variable_format_optimize=True)</pre>

Parameter	Description
<code>mix_compile_mode</code>	Mixed computing enable • True: enabled. • False (default): disabled (fully offloaded mode) In fully offloaded mode, all compute operators are offloaded to the device side. As a supplement to the fully offloaded mode, mixed computing allows some operators to be executed online in the frontend framework, improving the flexibility for adapting to TensorFlow. Example: <code>config = NPURunConfig(mix_compile_mode=True)</code>
<code>hcom_parallel</code>	Whether to enable the AllReduce gradient update and forward and backward parallel execution. • True: enabled • False (default): disabled Example: <code>config = NPURunConfig(hcom_parallel=True)</code>
<code>graph_memory_max_size</code>	Network static memory and maximum dynamic memory, which can be specified based on the network size. The value ranges from 0 to 256 x 1024 x 1024 x 1024 or 0 to 274877906944, in bytes. Due to Ascend AI Processor hardware restrictions, the sum of graph_memory_max_size and variable_memory_max_size cannot exceed 31 GB. If not set, 26 GB is used by default. Example: <code>config = NPURunConfig(graph_memory_max_size=str(26*1024 * 1024 * 1024))</code>
<code>variable_memory_max_size</code>	Variable memory, which can be specified based on the network size. The value ranges from 0 to 256 x 1024 x 1024 x 1024 or 0 to 274877906944, in bytes. Due to Ascend AI Processor hardware restrictions, the sum of graph_memory_max_size and variable_memory_max_size cannot exceed 31 GB. If not set, 5 GB is used by default. Example: <code>config = NPURunConfig(variable_memory_max_size=str(5*1024 * 1024 * 1024))</code>

Parameter	Description
auto_tune_mode	You can determine whether to use the Auto Tune tool to tune the TBE operators during build to find the best performance configuration on the . - Genetic Algorithm (GA): configures the tuning performance of the Cube operator. - Reinforcement Learning (RL): sets vector operator tuning performance. You can specify both modes. Enclose the whole argument in double quotation marks (""), and separate the modes by commas (,). If not specified, Auto Tune is disabled. For details about the usage of the Auto Tune tool, see "Auto Tune Tool Instructions" in <i>CANN Auxiliary Development Tool Guide</i>. Example: <pre>config = NPURunConfig(auto_tune_mode="RL,GA")</pre>
stream_max_parallel_num	AI CPU and AI Core engine parallelism for parallel execution of AI CPU and AI Core operators. Example: <pre>config = NPURunConfig(stream_max_parallel_num="DNN_VM_TF: 10,DNN_V100:1")</pre> DNN_VM_TF is the name of the AI CPU engine. In this example, the number of concurrent tasks of the AI CPU engine is 10. DNN_V100 is the name of the AI Core engine. In this example, the number of concurrent tasks of the AI Core engine is 1. The value range is [1, 13]. Defaults to 1.

Parameter	Description
is_tailing_optimization	Communication hangover optimization enable in distributed training scenarios to improve performance. By changing a computation dependency relationship, a computation operation that does not depend on the last AR (gradient aggregation fragment) is scheduled to be performed in parallel with the last AR, to optimize communication hangover. • True: enabled • False (default): disabled. This parameter must be used in pair with 10.2.2.5.2 NPUOptimizer Constructor and the value must be the same as that of is_tailing_optimization in 10.2.2.5.2 NPUOptimizer Constructor. Example: <pre>config = NPURunConfig(is_tailing_optimization=True)</pre>
horovod_mode	This parameter is not supported in the current version.
graph_run_mode	Graph run mode • 0: online inference • 1 (default): training Example: <pre>config = NPURunConfig(graph_run_mode=1)</pre>

Parameter	Description
op_debug_level	Operator debug enable. • 0 (default): disables operator debug. • 1: enables operator debug and generates a TBE instruction mapping file. In this case, an operator CCE file (.cce), a Python-CCE mapping file (_loc.json), and operator .o and .json files are generated in the kernel_meta folder in the training script execution directory. You can locate the AI Core error by using the line numbers in the CCE code and TBE code of the error operator. • 2: enables operator debug and generates a TBE instruction mapping file. In this case, an operator CCE file (.cce), a Python-CCE mapping file (_loc.json), and operator .o and .json files are generated in the kernel_meta folder in the training script execution directory, and the build optimization is disabled by enabling the CCE compiler -O0-g. You can locate the AI Core error by using the line numbers in the CCE code and TBE code of the error operator. • 3: disables operator debug and retains the operator .o and .json files in the kernel_meta folder in the training script execution directory. NOTICE You are advised to set this parameter to 0 or 3 for training. If you need to locate AI Core errors, set this parameter to 1 or 2, which might compromise the network performance. Example: config = NPURunConfig(op_debug_level=1)

Parameter	Description
enable_scope_fusion_passes	Fusion pattern (or fusion patterns separated by commas) to take effect during build. Both built-in and custom scope fusion pattern files are classified into the following two types: • General: common scope fusion patterns applicable to all networks. It takes effect by default and cannot be manually invalidated. • Non-General: scope fusion patterns applicable to specific networks. By default, it does not take effect. You can use enable_scope_fusion_passes to specify the fusion pattern list that takes effect. Example: <pre>config = NPURunConfig(enable_scope_fusion_passes="ScopeLayerNormPass,ScopeClipBoxesPass")</pre>
distribute	For distributed training in the PS-Worker architecture, pass the ParameterServerStrategy object. Example: <pre>config = NPURunConfig(distribute=strategy)</pre>
enable_exception_dump	Whether to dump the inputs and outputs of error operators. • 0 (default): disabled • 1: enabled. Keeping this switch enabled does not compromise the performance. Example: <pre>config = NPURunConfig(enable_exception_dump=1)</pre>

Parameter	Description
op_select_implmode	Operator implementation mode select. Some operators built in the Ascend AI Processor can be implemented in either high-precision or high-performance mode. • high_precision: high precision implementation mode. In high-precision mode, Newton's Method or Taylor's Formula is used to improve operator precision with fp16 input. • high_performance (default): high performance implementation mode. The high-performance implementation mode refers to the optimal performance implementation without affecting the network precision with fp16 input. Example: <pre>config = NPURunConfig(op_select_implmode="high_precision")</pre>
optypelist_for_implmode	List of operator types separated by commas (,). The operators in the list use the mode specified by the op_select_implmode parameter. Currently, Pooling, SoftmaxV2, LRN, and ROIAlign operators are supported. This parameter is used in pair with the op_select_implmode parameter, for example: <pre>config = NPURunConfig(op_select_implmode="high_precision", optypelist_for_implmode="Pooling,SoftmaxV2")</pre>
dynamic_input_config	Dynamic dimension configuration. Before creating NPURunConfig, you can instantiate a DynamicInputConfig class to set related attributes. For details about the constructor of the DynamicInputConfig class, see DynamicInputConfig Constructor. Example: <pre>config = NPURunConfig(dynamic_input_config=dynamic_input_config)</pre>
mstune_mode	This parameter is not supported in the current version.
work_path	This parameter is not supported in the current version.
buffer_optimize	This parameter is not supported in the current version.
enable_small_channel	This parameter is not supported in the current version.

Parameter	Description
fusion_switch_file	Directory of the fusion switch configuration file, including the file name. The value can contain letters, digits, underscores (_), hyphens (-), periods (.), and Chinese characters. The system is built in with a range of graph fusion and UB fusion patterns, which are enabled by default. You can disable any rule as required. Example: <pre>config = NPURunConfig(fusion_switch_file="/home/test/fusion_switch.cfg")</pre> The following is a template of the fusion_switch.cfg fusion pattern configuration file. on indicates that a fusion pattern is enabled, and off indicates that a fusion pattern is disabled. <pre>{ "Switch":{ "GraphFusion":{ "ConvToFullyConnectionFusionPass":"off", "SoftmaxFusionPass":"on", "ConvConcatFusionPass":"on", "MatMulBiasAddFusionPass":"on", "PoolingFusionPass":"on", "ZConcatv2dFusionPass":"on", "ZConcatExt2FusionPass":"on", "TfMergeSubFusionPass":"on" }, "UBFusion":{ "ThePool2dQuantFusionPass":"on" } } }</pre>
enable_compress_weight	This parameter is not supported in the current version.
compress_weight_conf	This parameter is not supported in the current version.

Parameter	Description
op_compiler_cache_mode	Disk cache enable for operator build. • enable: enabled. If enabled, operators with the same build configurations and operator configurations will not be built repeatedly, thus accelerating the build speed. • disable (default): disabled. • force: forcibly flushes the cache. That is, the existing cache is cleared up before adding new operator builds to the cache. This is especially useful when the Python libraries or dependent libraries are changed. Restrictions: • It needs to be used in pair with op_compiler_cache_dir. The cache function is enabled only when op_debug_level is set to 0 or 3. • If set to force, the existing cache will be cleared. Therefore, it is not recommended for parallel program compilation. Otherwise, the cache used by other models may be cleared, causing build failures. • disable and force are recommended for publishing the final model. Example: <pre>config = NPURunConfig(op_compiler_cache_mode="enable")</pre>
op_compiler_cache_dir	Disk cache directory for operator build. The value can contain letters, digits, underscores (_), hyphens (-), periods (.), and Chinese characters. If the specified directory exists and is valid, a kernel_cache subdirectory is automatically created. If the specified directory does not exist but is valid, the system automatically creates this directory and the kernel_cache subdirectory. Defaults to \$HOME/atc_data. Example: <pre>config = NPURunConfig(op_compiler_cache_dir="/home/test/kernel_cache")</pre>

Parameter	Description
debug_dir	Directory of the debug files generated during the operator build process, including the .o, .json, and .cce files. Defaults to the execution path of the current script. Example: config = NPURunConfig(debug_dir="/home/test")

Returns

An object of the **NPURunConfig** class is returned as the initialization parameter of **NPUEstimator**.

Example

The following describes how to construct a config instance in fully offloaded mode and setting the number of iterations per loop to **1000** as an example:

```
from npu_bridge.estimator.npu.npu_config import NPURunConfig
session_config=tf.ConfigProto()
config = NPURunConfig(session_config=session_config, mix_compile_mode=False, iterations_per_loop=1000)
```

10.2.2.2 ProfilingConfig Constructor

Prototype

```
def __init__(self,
enable_profiling=False,
profiling_options=None
)
```

Description

Constructs an object of class **ProfilingConfig** as the profiling configuration.

Parameters

Parameter	Input/Output	Description
enable_profiling	Input	Profiling enable. • True: enabled. The profiling option is determined by enable_options. • False (default): disabled.

Parameter	Input/ Output	Description
profiling_options	Input	Profiling options. output: path for storing profiling result files. The specified path must be created in advance in the environment (either in a container or on the host) where training is performed. The running user configured during installation must have the read and write permissions on this path. The path can be an absolute path or a relative path relative to the path where the command is executed. - An absolute path starts with a slash (/), for example, /home/HwHiAiUser/output. - A relative path starts with a directory name, for example, output. training_trace: iteration tracing. Collects software profile data of a training job and the AI Software Stack to profile the training job. Focuses on data augmentation, forward and backward propagation, and gradient aggregation and update. Either on or off. A value other than on or off is equivalent to off. task_trace: task tracing. Collects the HWTS hardware information of the and the start and end of each task. Either on or off. A value other than on or off is equivalent to off. aicpu: AI CPU data augmentation tracing. Either on or off. A value other than on or off is equivalent to off. fp_point: required when training_trace is on. Specifies the start point of the forward propagated operator in iteration traces, to record the start timestamp of forward propagation. Set the value to the name of the top operator in forward propagation. You can save the graph as a .ptxt file by using tf.io.write_graph in the training script to obtain this name. bp_point: required when training_trace is on. Specifies the end point of the backward propagated operator in iteration traces, to record the end timestamp of backward propagation. BP_POINT and FP_POINT are used to compute the time used by forward and backward propagation. Set the value to

Parameter	Input/Output	Description
		the name of the bottom operator in backward propagation. You can save the graph as a .pbtxt file by using tf.io.write_graph in the training script to obtain this name. • aic_metrics: AI Core metric to profile. ArithmeticUtilization: percentages of arithmetic utilization. PipeUtilization (default): percentages of time taken by the compute units and MTEs. Memory: percentages of external memory read/write instructions. MemoryL0: percentages of internal memory read/write instructions. ResourceConflictRatio: percentages of pipeline queue Instructions. Example: <pre>profiling_options = '{"output":"/tmp/ profiling","training_trace":"on","fp_point":"resnet_model/ conv2d/Conv2Dresnet_model/batch_normalization/ FusedBatchNormV3_Reduce","bp_point":"gradients/AddN_70"}'</pre>

Returns

An object of the **ProfilingConfig** class, as an argument passed to the **NPURunConfig** call.

10.2.2.2.3 DumpConfig Constructor

Prototype

```
def __init__(self,
enable_dump=False,
dump_path=None,
dump_step=None,
dump_mode="output",
enable_dump_debug=False,
dump_debug_mode="all")
```

Description

Constructs an object of class **ProfilingConfig** as the dump configuration.

Restrictions

enable_dump and **enable_dump_debug** cannot be both enabled.

Parameters

Parameter	Input/ Output	Description
enable_dump	Input	Data dump enable • True: enabled. The dump file path is read from dump_path. • False (default): disabled
dump_path	Input	Dump path. This parameter is required when enable_dump or enable_dump_debug is set to True . The specified path must be created in advance in the environment (either in a container or on the host) where training is performed. The running user configured during installation must have the read and write permissions on this path. The path can be an absolute path or a relative path relative to the path where the command is executed. • An absolute path starts with a slash (/), for example, /home/HwHiAiUser/output. • A relative path starts with a directory name, for example, output.
dump_step	Input	Iterations to dump. Defaults to None , indicating that all iterations are dumped. Separate multiple iterations using vertical bars (), for example, 0 5 10. You can also use hyphens (-) to specify the iteration range, for example, 0 3-5 10.
dump_mode	Input	Dump mode. • input: dumps only operator inputs. • output (default): dumps only operator outputs. • all: dumps both operator inputs and outputs.
enable_dump_debug	Input	Overflow detection enable. • True: enabled. The dump file path is read from dump_path. If dump_path is set to None, an exception occurs. • False (default): disabled.

Parameter	Input/ Output	Description
dump_debug_mode	Input	Overflow detection mode. • aicore_overflow: detects AI Core operator overflow, outputting the abnormal maximum values (such as float16 65500, 38400, and 51200) when the inputs of the operator are normal. Once such fault is detected, analyze the cause of the overflow and modify the operator implementation based on the network requirements and operator logic. • atomic_overflow: detects Atomic Add overflow, for checking modules involved in floating-point computing (such as SDMA) in addition to AI Core. • all (default): detects both AI Core operator overflow and Atomic Add overflow.

Returns

An object of the **DumpConfig** class, as an argument passed to the **NPURunConfig** call.

10.2.2.3 npu_bridge.estimator.npu.npu_estimator

10.2.2.3.1 NPUEstimator Constructor

Prototype

```
def __init__(self,
model_fn=None,
model_dir=None,
config=None,
params=None,
job_start_file="
)
```

Description

Constructor of the **NPUEstimator** class, which inherits the **Estimator** class and can call the native APIs of the base class.

Parameters

Parameter	Input/Output	Description
model_fn	Input	Model function definition. This function returns the NPUEstimatorSpec class object. For details about the constructor of the NPUEstimatorSpec class, see 10.2.2.3.2 NPUEstimatorSpec Constructor .
model_dir	Input	Model storage path, which is used to save or restore model files. Defaults to None . If model_dir set in NPURunConfig and NPUEstimator are different, an error is reported. If either NPURunConfig or NPUEstimator is configured with model_dir , the configured path applies. If neither NPURunConfig nor NPUEstimator is configured with model_dir , a model_dir_XXXXXXXXXX directory is created in the current script execution path to save the model file.
config	Input	NPURunConfig class object For details about the constructor of the NPURunConfig class, see 10.2.2.2.1 NPURunConfig Constructor .
params	Input	Argument of model_fn , which is of the dictionary type. The key is the name of the argument, and the value is the basic Python type value.
job_start_file	Input	Path of the CSA job startup file. This parameter does not need to be set in the cloud scenario.

Returns

An object of the **NPUEstimator** class.

10.2.2.3.2 NPUEstimatorSpec Constructor

Prototype

```
def __new__(cls,
mode,
predictions=None,
loss=None,
train_op=None,
```



```
eval_metric_ops=None,  
export_outputs=None,  
training_chief_hooks=None,  
training_hooks=None,  
scaffold=None,  
evaluation_hooks=None,  
prediction_hooks=None,  
host_call=None)
```

Description

Constructor of the **NPUEstimatorSpec** class, which inherits the **EstimatorSpec** class and can call the native APIs of the base class.

EstimatorSpec is the return data structure of **model_fn**, including the **mode**, **predictions**, **loss**, **train_op**, and **export_outputs** fields. If **EstimatorSpec** cannot meet the training requirements, define **NPUEstimatorSpec** to replace **EstimatorSpec**.

Parameters

Parameter	Input/Output	Description
NPUEstimatorSpec inherits the following EstimatorSpec parameters:		
mode	Input	Mode, indicating whether training, verification, or inference is being performed. • ModeKeys.TRAIN: training • ModeKeys.EVAL: validation • ModeKeys.PREDICT: inference
predictions	Input	Inference output tensor, required when mode is set to ModeKeys.PREDICT
loss	Input	Training loss
train_op	Input	Training operator
eval_metric_ops	Input	Dictionary of the measurement result (based on the tensor name). The dictionary value can be one of the following: • Metric instance • Result of calling the metric function, that is, the (metric_tensor, update_op) tuple.

Parameter	Input/Output	Description
export_outputs	Input	Used to save a model, describing the output format of the exported model
training_chief_hooks	Input	SessionRunHooks set of the master node during training
training_hooks	Input	SessionRunHooks set during training
scaffold	Input	Defines scaffold (providing the capability of customizing saver , init_op , summary_op , and global_step).
evaluation_hooks	Input	SessionRunHooks set during evaluation
prediction_hook	Input	SessionRunHooks set during inference
NPUEstimatorSpec has the following parameters added:		
host_call	Input	Captures the summary information and sends the information of each step back to the host side. host_call is a tuple consisting of a function and a list or dictionary of tensors. It is used to return a list of tensors. host_call applies to train() and evaluate() .

Returns

An object of the **NPUEstimatorSpec** class

10.2.2.4 npu_bridge.estimator.npu.npu_hook

10.2.2.4.1 NPUCheckpointSaverHook Constructor

Prototype

```
def __init__(self,  
    checkpoint_dir,  
    save_secs=None,  
    save_steps=None,  
    saver=None,  
    checkpoint_basename="model.ckpt",  
    scaffold=None,
```

listeners=None)

Description

Constructor of the **NPUCheckpointSaverHook** class, which is used to save the checkpoint file. The **NPUCheckpointSaverHook** class inherits the **CheckpointSaverHook** class and can call native APIs of the base class.

Restrictions

When **NPUEstimator** is used and **iteration_per_loop** is set to a value greater than 1, the hook may not take effect.

Parameters

Parameter	Input/Output	Description
checkpoint_dir	Input	Checkpoint file directory
save_secs	Input	Interval (in seconds) for saving the checkpoint file
save_steps	Input	Interval (in steps) for saving the checkpoint file
saver	Input	Saver object
checkpoint_basename	Input	Basename of the checkpoint file
scaffold	Input	Scaffold of the saver object
listeners	Input	Example of the CheckpointSaverListener subclass, for saving the checkpoint file

Returns

An object of the **NPUCheckpointSaverHook** class

Example

```
from npu_bridge.estimator.npu.npu_hook import NPUCheckpointSaverHook
checkpoint_hook = NPUCheckpointSaverHook(checkpoint_dir='./ckpt', save_steps=2000)
...
mnist_classifier.train(
    input_fn=train_input_fn,
    steps=2000,
    hooks=[checkpoint_hook])
```

10.2.2.4.2 NPUOutputTensorHook Constructor

Prototype

```
def __init__(self, tensors,
dependencies=None,
```

```
output_fn=None,  
output_every_n_steps=0  
)
```

Description

Constructor of the **NPUOutputTensorHook** class. **NPUOutputTensorHook** is the hook in the train, evaluate, and predict processes of **NPUEstimator**, and it is used to call the user-defined **output_fn** every *N* steps or at the end to print the output tensors. The **NPUOutputTensorHook** class inherits the **LoggingTensorHook** class and can call native APIs of the base class.

Restrictions

When **Iterations_per_loop** > 1, **output_fn** cannot be called as specified by **output_every_n_steps**.

Parameters

Parameter	Input/Output	Description
tensors	Input	Name set of the input tensors, in dictionary or list format
dependencies	Input	Dependencies corresponding to tensors
output_fn	Input	Print function of tensor output
output_every_n_steps	Input	The user-defined output_fn , which is called when the session is executed for <i>N</i> times and the training script is executed.

Returns

An object of the **NPUOutputTensorHook** class

Example

```
from npu_bridge.estimator.npu.npu_hook import NPUOutputTensorHook  
  
# Define output_fn.  
def output_fn(inputs):  
    device_id = os.environ["ASCEND_DEVICE_ID"]  
    output_file = os.path.join("/code", device_id, "test_npu_output_tensor.txt")  
    for item in inputs:  
        content = "step:{},loss:{}".format(str(item['global_step']), str(item['loss']))  
        with open(output_file, 'a') as f:  
            f.write(content)  
            f.write("\n")  
  
# Define output_hook for calling the user-defined output_fn.  
tensors = {'global_step': global_step, 'loss': loss}  
output_hook = NPUOutputTensorHook(  

```

```
tensors,  
dependencies=train_op_list,  
output_fn=output_fn,  
output_every_n_steps=10)  
train_hook.append(output_hook)  
  
# Pass the hook to EstimatorSpec.  
return tf.estimator.EstimatorSpec(  
    mode=mode,  
    predictions=predictions,  
    loss=loss,  
    train_op=train_op,  
    training_chief_hooks=train_hook,  
    eval_metric_ops=metrics)
```

10.2.2.5 npu_bridge.estimator.npu.npu_optimizer

10.2.2.5.1 NPUDistributedOptimizer Constructor

Prototype

```
def __init__(self, optimizer, name=None)
```

Description

Constructor of the **NPUDistributedOptimizer** class, which is used to package the single-server training optimizer provided by the user and construct the NPU distributed training optimizer.

In single-server single-device, single-server multi-device, and multi-server multi-device networking modes, gradient aggregation can be performed among devices after gradient calculation.

Parameters

Parameter	Input/Output	Description
optimizer	Input	Standalone training optimizer for gradient calculation and weight update
name	Input	Name of the optimizer

Returns

An object of the **NPUDistributedOptimizer** class

Example

After defining a standalone optimizer, you can use it for packaging. The following is an example.

```
import tensorflow as tf  
from npu_bridge.estimator.npu.npu_optimizer import NPUDistributedOptimizer
```

```
optimizer = tf.train.GradientDescentOptimizer(learning_rate=learning_rate)  
optimizer = NPUDistributedOptimizer(optimizer)
```

10.2.2.5.2 NPUOptimizer Constructor

Prototype

```
def __init__(self,  
    opt,  
    loss_scale_manager=None,  
    is_distributed=False,  
    is_loss_scale=False,  
    is_tailing_optimization=False,  
    name=None)
```

Description

Constructor of the **NPUOptimizer** class, which combines the **NPUDistributedOptimizer** and **NPULossScaleOptimizer** optimizers.

It provides the following functions:

- Loss scaling: Loss scaling can be enabled during mixed precision training to solve the underflow problem caused by a small float16 representation range.
- Distributed training: The single-server training optimizer of the user is packaged and an NPU distributed training optimizer is constructed. In single-server single-device, single-server multi-device, and multi-server multi-device networking modes, gradient aggregation is performed after gradient calculation on devices.
- Communication hangover optimization: By changing a computation dependency relationship, a computation operation that does not depend on the last AR (gradient aggregation fragment) is scheduled to be performed in parallel with the last AR, to optimize communication hangover.

Parameters

Parameter	Input / Output	Description
opt	Input	Standalone training optimizer for gradient calculation and weight update

Parameter	Input / Output	Description
loss_scale_manager	Input	This parameter needs to be configured only when is_loss_scale is set to True and the loss scaling function is enabled. This parameter determines the update mode of loss scale, including static update and dynamic update. - Before creating NPUOptimizer, you can instantiate a FixedLossScaleManager class to statically configure loss scale. (The loss scale value is fixed, which needs to be specified.) For details about the constructor of the FixedLossScaleManager class, see 10.2.2.7.1 FixedLossScaleManager Constructor. - Before creating NPUOptimizer, you can instantiate an ExponentialUpdateLossScaleManager class to dynamically configure loss scale. For details about the constructor of the ExponentialUpdateLossScaleManager class, see 10.2.2.7.2 ExponentialUpdateLossScaleManager Constructor.
is_distributed	Input	Distributed training enable True: executes AllReduce. False (default)
is_loss_scale	Input	Loss scaling enable True: enabled (recommended if mixed precision training is enabled). In this case, the value of loss_scale_manager cannot be None. False (default): disabled
is_tailing_optimization	Input	Communication hangover optimization enable, for improving training performance. This function takes effect only when is_distributed is set to True. True: enabled False (default): disabled Argument of this parameter must be the same as that set in 10.2.2.2.1 NPURunConfig Constructor.
name	Input	Name of the optimizer

Returns

An object of the **NPUOptimizer** class

Example

```
import tensorflow as tf
from npu_bridge.estimator.npu.npu_optimizer import NPUOptimizer
from npu_bridge.estimator.npu.npu_loss_scale_manager import FixedLossScaleManager
from npu_bridge.estimator.npu.npu_loss_scale_manager import ExponentialUpdateLossScaleManager

# Define a single-server optimizer.
optimizer = LAMBOptimizer(
    learning_rate=learning_rate,
    weight_decay_rate=0.01,
    beta_1=0.9,
    beta_2=0.999,
    epsilon=1e-6,
    exclude_from_weight_decay=["LayerNorm", "layer_norm", "bias"])

# Enable loss scaling.
if tf.flags.FLAGS.npu_bert_loss_scale not in [None, -1]:
    if tf.flags.FLAGS.npu_bert_loss_scale == 0:
        loss_scale_manager =
        ExponentialUpdateLossScaleManager(init_loss_scale=tf.flags.FLAGS.init_loss_scale_value,
        incr_every_n_steps=1000, decr_every_n_nan_or_inf=2, decr_ratio=0.5)
    elif tf.flags.FLAGS.npu_bert_loss_scale >= 1:
        loss_scale_manager = FixedLossScaleManager(loss_scale=tf.flags.FLAGS.npu_bert_loss_scale)
    else:
        raise ValueError("Invalid loss scale: %d" % tf.flags.FLAGS.npu_bert_loss_scale)
    optimizer = NPUOptimizer(optimizer, loss_scale_manager, is_distributed=tf.flags.FLAGS.distributed,
    is_loss_scale=True, is_tailing_optimization=True)

# Disable loss scaling.
else:
    optimizer = NPUOptimizer(optimizer, is_distributed=tf.flags.FLAGS.distributed)
```

10.2.2.6 npu_bridge.estimator.npu.npu_loss_scale_optimizer

10.2.2.6.1 NPULossScaleOptimizer Constructor

Prototype

```
def __init__(self, opt, loss_scale_manager, is_distributed=False)
```

Description

Constructor of the **NPULossScaleOptimizer** class, which is used to enable loss scaling during mixed precision training. Loss scaling solves the underflow problem caused by the small float16 representation range. The **NPULossScaleOptimizer** class inherits the **LossScaleOptimizer** class and can call native APIs of the base class.

Parameters

Parameter	Input/Output	Description
opt	Input	Standalone training optimizer for gradient calculation and weight update
loss_scale_manager	Input	Loss scale update mode, including static update and dynamic update - Before creating NPULossScaleOptimizer, you can instantiate a FixedLossScaleManager class to statically configure loss scale. (The loss scale value is fixed, which needs to be specified.) For details about the constructor of the FixedLossScaleManager class, see 10.2.2.7.1 FixedLossScaleManager Constructor. - Before creating NPULossScaleOptimizer, you can instantiate an ExponentialUpdateLossScaleManager class to dynamically configure the loss scaling factor. For details about the constructor of the ExponentialUpdateLossScaleManager class, see 10.2.2.7.2 ExponentialUpdateLossScaleManager Constructor.
is_distributed	Input	Used to support the loss scaling function in the distributed training scenario. True: Set this parameter to True for distributed training. False

Returns

An object of the **NPULossScaleOptimizer** class

Example

```
from npu_bridge.estimator.npu.npu_loss_scale_optimizer import NPULossScaleOptimizer
from npu_bridge.estimator.npu.npu_loss_scale_manager import FixedLossScaleManager
from npu_bridge.estimator.npu.npu_loss_scale_manager import ExponentialUpdateLossScaleManager

if FLAGS.use_fp16 and (FLAGS.npu_bert_loss_scale not in [None, -1]):
    opt_tmp = opt
    if FLAGS.npu_bert_loss_scale == 0:
        loss_scale_manager = ExponentialUpdateLossScaleManager(init_loss_scale=2**32,
            incr_every_n_steps=1000, decr_every_n_nan_or_inf=2, decr_ratio=0.5)
    elif FLAGS.npu_bert_loss_scale >= 1:
        loss_scale_manager = FixedLossScaleManager(loss_scale=FLAGS.npu_bert_loss_scale)
    else:
        raise ValueError("Invalid loss scale: %d" % FLAGS.npu_bert_loss_scale)
    if ops_adapter.size() > 1:
```

```
opt = NPULossScaleOptimizer(opt_tmp, loss_scale_manager, is_distributed=True)
else:
    opt = NPULossScaleOptimizer(opt_tmp, loss_scale_manager)
```

10.2.2.7 npu_bridge.estimator.npu.npu_loss_scale_manager

10.2.2.7.1 FixedLossScaleManager Constructor

Prototype

```
def __init__(self, loss_scale)
```

Description

Constructor of the **FixedLossScaleManager** class, which is used to define static loss scale parameters.

Parameters

Parameter	Input/Output	Description
loss_scale	Input	Loss scale value. The value is of the float type and cannot be less than 1. If the value of loss scale is too small, model convergence may be affected. If the value of loss scale is too large, overflow may occur during training. The value can be the same as that of GPU.

Returns

An object of the **FixedLossScaleManager** class

10.2.2.7.2 ExponentialUpdateLossScaleManager Constructor

Prototype

```
def __init__(self,
    init_loss_scale,
    incr_every_n_steps,
    decr_every_n_nan_or_inf=2,
    incr_ratio=2,
    decr_ratio=0.8)
```

Description

Constructor of the **ExponentialUpdateLossScaleManager** class, which is used to define dynamic loss scale parameters.

Parameters

Parameter	Input/Output	Description
init_loss_scale	Input	Initial loss scale value. A float.
incr_every_n_steps	Input	If no overflow occurs for N iterations, increase the value of loss scale.
decr_every_n_nan_or_inf	Input	If an overflow occurs for N iterations, decrease the value of loss scale. Defaults to 2.
incr_ratio	Input	Percentage increase of loss scale. Defaults to 2.
decr_ratio	Input	Percentage decrease of loss scale. Defaults to 0.8.

Returns

An object of the **ExponentialUpdateLossScaleManager** class

10.2.2.8 npu_bridge.estimator.npu_ops

10.2.2.8.1 dropout

Prototype

```
def dropout(x, keep_prob, noise_shape=None, seed=None, name=None)
```

Description

The function works the same as **tf.nn.dropout**. Scales the input tensor by $1/\text{keep_prob}$, and the reservation probability of the input tensor is **keep_prob**. Otherwise, **0** is output, and the shape of the output tensor is the same as that of the input tensor.

Parameters

Parameter	Input / Output	Description
x	Input	Input tensor of type float.
keep_prob	Input	Scalar tensor of type float, which indicates the retention probability of each element.
noise_shape	Input	1D tensor of type int32, which indicates the shape of the randomly generated keep/drop flag.
seed	Input	Random seed
name	Input	Name of the network layer.

Returns

Tensor: output tensor after the dropout operation is performed on input **x**.

Example

```
from npu_bridge.estimator import npu_ops  
layers = npu_ops.dropout()
```

10.2.2.8.2 LARSV2

Prototype

```
def LARSV2(input_weight,  
input_grad,  
weight_decay,  
learning_rate,  
hyperpara=0.001,  
epsilon=0.00001,  
use_clip=False,  
name=None)
```

Description

This operator scales gradients based on the norm of weight and the norm of gradient at different levels using different learning rates. It is used to improve the training precision in large batch size scenarios and is used for large-scale cluster training to reduce the training time.

Parameters

Parameter	Input/ Output	Description
input_weight	Input	Weight tensor of type float.
input_grad	Input	Weight gradient tensor of type float.
weight_decay	Input	Scalar tensor of type float.
learning_rate	Input	Scalar tensor of type float, indicating the learning rate.
hyperpara	Input	Scalar of type float, for the hyperparameter of the operator. Generally set to 0.001 .
epsilon	Input	A scalar, added to avoid dividing by zero. Generally set to 1e-5 .
use_clip	Input	A bool. Defaults to False . If this parameter is set to True , the scaling coefficient must be within a specified range.
name	Input	Name of the network layer.

Returns

Tensor: output gradient tensor after the input gradient is updated.

Example

```
from npu_bridge.estimator import npu_ops  
layers = npu_ops.LARSV2(input_weight, input_grad, weight_decay, learning_rate)
```

10.2.2.8.3 initialize_system

Prototype

```
def initialize_system(name = None)
```

Description

Generally, this API is not required for training. If you want to exclude the GE initialization time in the training time statistics, you can call this API. Before using the collective communication API, you need to call this API to initialize the collective communication.

Restrictions

If the **initialize_system** API needs to be called and the following functions need to be enabled during training, the configuration must be performed when a session is started in **initialize_system**.

Table 10-7 Session configuration options in initialize_system

Item	Description
profiling_mode	Profiling enable. • True: enabled. The profiling option is determined by enable_options. • False (default): disabled.
profiling_options	Option (or options separated by colons) to be traced in profiling. • training_trace: iteration tracing. Collects software profile data of a training job and the AI Software Stack to profile the training job. Focuses on data augmentation, forward and backward propagation, and gradient aggregation and update. • task_trace: task tracing. Collects the HWTS and AI Core hardware information of the Ascend AI Processor and the start and end of each task. • op_trace: single-operator tracing. To do so, you need to construct single-operator networks and train the network using a training script. This variable is exclusive with training_trace and task_trace. NOTE • If training_trace is selected, fp_point and bp_point need to be configured. • If set to task_trace, training_trace is automatically enabled.
fp_point	Required if training_trace is selected. Specifies the start point of the forward propagated operator in iteration traces, to record the start timestamp of forward propagation. Set the value to the name of the top operator in forward propagation. You can save the graph as a .pbtxt file by using tf.io.write_graph in the training script to obtain this name.

Item	Description
bp_point	Required if training_trace is selected. Specifies the end point of the backward propagated operator in iteration traces, to record the end timestamp of backward propagation. BP_POINT and FP_POINT are used to compute the time used by forward and backward propagation. Set the value to the name of the bottom operator in backward propagation. You can save the graph as a .ptxt file by using tf.io.write_graph in the training script to obtain this name.
enable_dump	Data dump enable. • True: enabled. The dump file path is read from dump_path. If dump_path is set to None, an exception occurs. • False (default): disabled.
dump_path	Dump path. This parameter is required when enable_dump or enable_dump_debug is set to True. The specified path must be created in advance in the environment (either in a container or on the host) where training is performed. The running user configured during installation must have the read and write permissions on this path. The path can be an absolute path or a relative path relative to the path where the command is executed. • An absolute path starts with a slash (/), for example, /home/HwHiAiUser/output. • A relative path starts with a directory name, for example, output.
dump_step	Iterations to dump. Defaults to None, indicating that all iterations are dumped. Separate multiple iterations using vertical bars (), for example, 0 5 10. You can also use hyphens (-) to specify the iteration range, for example, 0 3-5 10.
dump_mode	Dump mode. • input: dumps only operator inputs. • output (default): dumps only operator outputs. • all: dumps both operator inputs and outputs.
enable_dump_debug	Overflow detection enable. • True: enabled. The dump file path is read from dump_path. If dump_path is set to None, an exception occurs. • False (default): disabled

Item	Description
dump_debug_mode	Overflow detection mode. • aicore_overflow: detects AI Core operator overflow, outputting the abnormal maximum values (such as float16 65500, 38400, and 51200) when the inputs of the operator are normal. Once such fault is detected, analyze the cause of the overflow and modify the operator implementation based on the network requirements and operator logic. • atomic_overflow: detects Atomic Add overflow, for checking modules involved in floating-point computing (such as SDMA) in addition to AI Core. • all: detects both AI Core operator overflow and Atomic Add overflow.
precision_mode	A string for the operator precision mode. • allow_fp32_to_fp16 or None: If an operator does not support the float32 data type, the precision is reduced to float16. By default, this mode is selected. • force_fp16: If an operator supports both float16 and float32 data types, float16 is forcibly selected. • must_keep_origin_dtype: The original precision is retained. • allow_mix_precision: Mixed precision is enabled. For operators of the float32 data type on a network, the precision of some float32 operators can be automatically reduced to float16 based on the built-in optimization policy. In this way, the system performance is improved and the memory usage is reduced with little accuracy loss. After this function is enabled, you can enable loss scaling to compensate for the accuracy drop caused by precision reduction. For details about the related APIs, see 10.2.2.6.1 NPULossScaleOptimizer Constructor.
auto_tune_mode	You can determine whether to use the Auto Tune tool to tune the TBE operators during build to find the best performance configuration on the . Example: auto_tune_mode = "RL,GA"; If this parameter is not specified, auto tuning is disabled. For details about the usage of the Auto Tune tool, see "Auto Tune Tool Instructions" in <i>CANN Auxiliary Development Tool Guide</i>.

Item	Description
graph_run_mode	Graph run mode. • 0: online inference • 1 (default): training
op_debug_level	Operator debug enable. • 0 (default): disables operator debug. • 1: enables operator debug and generates a TBE instruction mapping file. In this case, an operator CCE file (.cce), a Python-CCE mapping file (_loc.json), and operator .o and .json files are generated in the kernel_meta folder in the training script execution directory. You can locate the AI Core error by using the line numbers in the CCE code and TBE code of the error operator. • 2: enables operator debug and generates a TBE instruction mapping file. In this case, an operator CCE file (.cce), a Python-CCE mapping file (_loc.json), and operator .o and .json files are generated in the kernel_meta folder in the training script execution directory, and the build optimization is disabled by enabling the CCE compiler -O0-g. You can locate the AI Core error by using the line numbers in the CCE code and TBE code of the error operator. • 3: disables operator debug and retains the operator .o and .json files in the kernel_meta folder in the training script execution directory. NOTICE You are advised to set this parameter to 0 or 3 for training. If you need to locate AI Core errors, set this parameter to 1 or 2, which might compromise the network performance.
enable_exception_dump	Whether to dump the inputs and outputs of error operators. • 0 (default): disabled • 1: enabled. Keeping this switch enabled does not compromise the performance.

Item	Description
op_select_implmode	Operator implementation mode select. Some operators built in the Ascend AI Processor can be implemented in either high-precision or high-performance mode. • high_precision: high precision implementation mode. In high-precision mode, Newton's Method or Taylor's Formula is used to improve operator precision with fp16 input. • high_performance (default): high performance implementation mode. The high-performance implementation mode refers to the optimal performance implementation without affecting the network precision with fp16 input.
optypelist_for_implmode	List of operator types. The operators in the list use the mode specified by the OP_SELECT_IMPL_MODE parameter. Currently, only Pooling operator is supported. This parameter is used in pair with the OP_SELECT_IMPL_MODE parameter, for example: Set op_select_implmode to high_precision. Set optypelist_for_implmode to Pooling.

Parameters

Parameter	Input/Output	Description
name	Input	Operator name

Returns

An operator for the user to initialize GE by using **sess.run(op)**

Example

If you call an HCCL API such as **get_local_rank_id**, **get_rank_size**, or **get_rank_id** before calling **sess.run()** or **estimator.train()**, you need to start another session and execute **initialize_system** to initialize collective communication. After the training is complete, execute **shutdown_system** and close the session.

```
import tensorflow as tf
from npu_bridge.estimator import npu_ops
from tensorflow.core.protobuf.rewriter_config_pb2 import RewriterConfig

npu_int = npu_ops.initialize_system()
npu_shutdown = npu_ops.shutdown_system()

config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
```

```
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

init_sess = tf.Session(config=config)
init_sess.run(npu_init)

# Call an HCCL API...
# Perform training...

init_sess.run(npu_shutdown)
init_sess.close()
```

Or:

```
import tensorflow as tf
from npu_bridge.estimator import npu_ops
from tensorflow.core.protobuf.rewriter_config_pb2 import RewriterConfig

npu_init = npu_ops.initialize_system()
npu_shutdown = npu_ops.shutdown_system()

config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

with tf.Session(config=config) as sess:
    sess.run(npu_init)
    # Call an HCCL API...
    # Perform training...
    sess.run(npu_shutdown)
```

10.2.2.8.4 shutdown_system

Prototype

```
def shutdown_system(name = None)
```

Description

Disables all devices. This API is used in conjunction with [10.2.2.8.3 initialize_system](#).

Parameters

Parameter	Input/Output	Description
name	Input	Operator name.

Returns

An operator is returned for the user to close the device by using **sess.run(op)**.

10.2.2.9 npu_bridge.estimator.npu.npu_rnn

10.2.2.9.1 npu_dynamic_rnn

Prototype

```
def npu_dynamic_rnn(cell,  
    inputs,  
    initial_state=None,  
    dtype=None,  
    sequence_length=None,  
    scope=None)
```

Description

Creates a high-performance neural network specified by RNNCell.

Restrictions

This API applies to the neural machine translation (NMT) network training script in the **while_loop** expansion scenario.

Parameters

Parameter	Input / Output	Description
cell	Input	RNNCell instance, which is the memory unit of long short-term memory (LSTM) and gated recurrent unit (GRU).
inputs	Input	An input list whose length is T . Each input is a tuple whose shape is $[max_time, batch_size, input_size]$, or a nested tuple of this shape.
initial_state	Input	(Optional) Initial state of the recurrent neural network (RNN). If cell.state_size is an integer, it must be a tensor whose shape is $[batch_size, cell.state_size]$. If cell.state_size is a tuple, it must be a tuple of the $[batch_size, s]$ tensor in cell.state_size .
dtype	Input	(Optional) Data type of the initial state and expected output. This parameter is required if initial_state is empty, or there is a heterogeneous data type in the RNN state.
sequence_length	Input	Specifies the length of each sequence for an input. This parameter is a int32 or int64 vector (tensor) whose size is $[batch_size]$. The value range is $[0, T)$.

Parameter	Input / Output	Description
scope	Input	Creates VariableScope of the subgraph. Defaults to rnn .

Returns

Tensor: output tensor of the RNN

State: final state

Example

```
from npu_bridge.estimator.npu.npu_rnn import npu_dynamic_rnn
code:
inputs = npu_unstack(self.encoder_emb_inp, axis=0)
encoder_outputs, encoder_state = static_rnn(
    cell,
    inputs,
    dtype= dtype,
    sequence_length = sequence_length
)
encoder_outputs = npu_stack( encoder_outputs, axis=0 )
Modified code:
encoder_outputs, encoder_state = npu_dynamic_rnn(
    cell,
    inputs=self.encoder_emb_inp,
    dtype= dtype,
    sequence_length= sequence_length)
```

10.2.2.10 npu_bridge.estimator.npu.npu_scope

10.2.2.10.1 without_npu_compile_scope

Prototype

```
def without_npu_compile_scope()
```

Description

Configures the operator built on the host.

Restrictions

This API is valid only in mixed computing mode.

Parameters

None

Returns

None

10.2.2.10.2 keep_dtype_scope

Prototype

```
def keep_dtype_scope()
```

Description

Specifies the operators that retain the original precision.

Restrictions

None

Parameters

None

Returns

None

Example

```
with npu_scope.keep_dtype_scope():  
    X = tf.conv2d(a)
```

10.2.2.11 npu_bridge.estimator.npu.util

10.2.2.11.1 set_iteration_per_loop

Prototype

```
def set_iteration_per_loop(sess, train_op, iterations_per_loop=1)
```

Description

Sets the number of iterations per training loop in **sess.run** mode, that is, the number of training iterations executed on the device side in each **sess.run()** call. This API can save unnecessary interactions between the host and device and reduce the training time consumption.

Restrictions

The preceding API involves graph modification. If a graph cannot be modified (for example, the graph is frozen or a session is created using **tf.train.Supervisor**), you cannot use the **set_iteration_per_loop** API to set the loops and iterations per loop. In this case, use [10.2.2.11.2 create_iteration_per_loop_var](#) and [10.2.2.11.3 load_iteration_per_loop_var](#).

Parameters

Parameter	Input/Output	Description
sess	Input	Created TensorFlow session
train_op	Input	Operation that updates a variable or gradient
iterations_per_loop	Input	Number of iterations per training loop per sess.run() call on the device side. Defaults to 1 . The total number of iterations per training loop must be an integer multiple of iterations_per_loop . In mixed computing mode (mix_compile_mode is set to True), this parameter must be set to 1 .

Returns

An operator for the user to call by using **sess.run(op)**

10.2.2.11.2 create_iteration_per_loop_var

Prototype

```
def create_iteration_per_loop_var(self, train_op)
```

Description

This API is used in conjunction with [10.2.2.11.3 load_iteration_per_loop_var](#) to set the number of iterations per training loop every **sess.run()** call on the device side. This API is used to modify a graph and set the number of iterations per loop using [10.2.2.11.3 load_iteration_per_loop_var](#).

Parameters

Parameter	Input / Output	Description
train_op	Input	Operation that updates a variable or gradient

Returns

An operator for the user to call by using **sess.run(op)**

Example

```
# Train a model.
with tf.Session(config=config) as sess:
    sess.run(init)
    # Set the number of iterations per loop to 10 in sess.run mode.
    iteration = util.IterationPerLoop()
    train_op = iteration.create_iteration_per_loop_var(optimizer) # Modify the graph.
    tf.train.Supervisor(logdir="/home/xxx",init_op=init) # Freeze the graph.
    iteration.load_iteration_per_loop_var(sess, 10) # Set the number of iterations per loop.

    for epoch in range(training_epochs):
        avg_cost = 0
        total_batch = int(mnist.train.num_examples / batch_size)

        for i in range(total_batch):
            batch_xs, batch_ys = mnist.train.next_batch(batch_size)
            _, c = sess.run([train_op, cost], feed_dict={x: batch_xs, y: batch_ys})

            avg_cost += c / total_batch
```

10.2.2.11.3 load_iteration_per_loop_var

Prototype

```
def load_iteration_per_loop_var(self, sess, iterations_per_loop=1)
```

Description

This API is used in conjunction with [10.2.2.11.2 create_iteration_per_loop_var](#) to set the number of iterations per training loop every **sess.run()** call on the device side.

Restrictions

In mixed computing mode (**mix_compile_mode** is set to **True**), this parameter must be set to **1**.

Parameters

Parameter	Input/Output	Description
sess	Input	Created TensorFlow session
iterations_per_loop	Input	Number of iterations per training loop per sess.run() call on the device side. Defaults to 1 . The total number of iterations per training loop must be an integer multiple of iterations_per_loop .

Returns

None

Example

```
# Train a model.
with tf.Session(config=config) as sess:
    sess.run(init)
    # Set the number of iterations per loop to 10 in sess.run mode.
    iteration = util.IterationPerLoop()
    train_op = iteration.create_iteration_per_loop_var(optimizer) # Modify the graph.
    tf.train.Supervisor(logdir="/home/xxxx",init_op=init) # Freeze the graph.
    iteration.load_iteration_per_loop_var(sess, 10) # Set the number of iterations per loop.

    for epoch in range(training_epochs):
        avg_cost = 0
        total_batch = int(mnist.train.num_examples / batch_size)

        for i in range(total_batch):
            batch_xs, batch_ys = mnist.train.next_batch(batch_size)
            _, c = sess.run([train_op, cost], feed_dict={x: batch_xs, y: batch_ys})

            avg_cost += c / total_batch
```

10.2.2.12 npu_bridge.estimator.npu.keras_to_npu

10.2.2.12.1 model_to_npu_estimator

Prototype

```
def model_to_npu_estimator(keras_model=None,
                           keras_model_path=None,
                           custom_objects=None,
                           model_dir=None,
                           checkpoint_format='saver',
                           config=None,
                           job_start_file="")
```

Description

Converts the model constructed by using **Keras** to an **NPUEstimator** object.

Restrictions

Currently, only the function model and sequence model (Keras graph construction mode) can be converted into an NPUEstimator object using the **model_to_npu_estimator** API.

Parameters

Parameter	Description
keras_model	Built Keras model object. This parameter and keras_model_path are mutually exclusive.

Parameter	Description
keras_model_path	Path for storing the built Keras model on the drive. You can use the save() method of the Keras model to generate a Keras model in HDF5 format. This parameter and keras_model are mutually exclusive.
custom_objects	Dictionary of user-defined objects. If a user-defined layer or function is used during Keras construction, custom_objects must be used during model loading.
model_dir	Model storage path, which is used to save or restore model files. If this parameter is not set, the value of model_dir in the config file is used. If both parameters are set, the values of the two parameters must be the same. If both parameters are set to None , the temporary directory /tmp is used.
checkpoint_format	Format of the checkpoint file saved by NPUEstimator during training. • saver (default): Save the model using tf.train.Saver(). • checkpoint: Save the model using tf.train.Checkpoint(). Compared with tf.train.Saver, tf.train.Checkpoint supports delayed variable recovery in instant execution mode.
config	NPURunConfig class object, which is used to configure NPUEstimator running parameters. For details about the constructor of the NPURunConfig class, see 10.2.2.2.1 NPURunConfig Constructor .
job_start_file	Path of the configuration file used to start the training process in the CSA scenario. This parameter does not need to be set in the cloud scenario.

Returns

An **NPUEstimator** object is returned based on the input Keras model.

10.2.2.13 Session Configuration in sess.run Mode

When training or online inference is performed on the in **sess.run** mode, the following configuration options are supported.

NOTICE

If this parameter is not listed in the table, it is reserved or applicable to other chip versions. You can ignore this parameter.

Table 10-8 Session configuration items

Item	Description	Use Case
use_off_line	Whether the training is performed on the . • True (default): yes • False: not (training is performed on the host CPU.) Example: custom_op.parameter_map["use_off_line"].b = True	Training/Online inference
enable_data_pre_proc	Data preprocessing enable • True (default): enabled • False: disabled Example: custom_op.parameter_map["enable_data_pre_proc"].b = True	Training
iterations_per_loop	Number of iterations per loop set by using set_iteration_per_loop in sess.run mode, that is, the number of iterations per training loop every sess.run() call on the device side. The parameter value must be the same as the value of iterations_per_loop set by set_iteration_per_loop for function verification. Example: custom_op.parameter_map["iterations_per_loop"].i = 10	Training
profiling_mode	Profiling enable. • True: enabled. The profiling option is determined by enable_options. • False (default): disabled. Example: custom_op.parameter_map["profiling_mode"].b = True	Training/Online inference

Item	Description	Use Case
profiling_options	Profiling options. • output: path for storing profiling result files. The specified path must be created in advance in the environment (either in a container or on the host) where training is performed. The running user configured during installation must have the read and write permissions on this path. The path can be an absolute path or a relative path relative to the path where the command is executed. – An absolute path starts with a slash (/), for example, /home/HwHiAiUser/output. – A relative path starts with a directory name, for example, output. • training_trace: iteration tracing. Collects software profile data of a training job and the AI Software Stack to profile the training job. Focuses on data augmentation, forward and backward propagation, and gradient aggregation and update. Either on or off. A value other than on or off is equivalent to off. • task_trace: task tracing. Collects the HWTS hardware information of the and the start and end of each task. Either on or off. A value other than on or off is equivalent to off. • aicpu: AI CPU data augmentation tracing. Either on or off. A value other than on or off is equivalent to off. • fp_point: required when training_trace is on. Specifies the start point of the forward propagated operator in iteration traces, to record the start timestamp of forward propagation. Set the value to the name of the top operator in forward propagation. You can save the graph as a .pbtxt file by using tf.io.write_graph in the training script to obtain this name. • bp_point: required when training_trace is on. Specifies the end point of the backward propagated operator in iteration traces, to record the end	Training/Online inference

Item	Description	Use Case
	timestamp of backward propagation. BP_POINT and FP_POINT are used to compute the time used by forward and backward propagation. Set the value to the name of the bottom operator in backward propagation. You can save the graph as a .pbtxt file by using tf.io.write_graph in the training script to obtain this name. • aic_metrics: AI Core metric to profile. ArithmeticUtilization: percentages of arithmetic utilization. PipeUtilization (default): percentages of time taken by the compute units and MTEs. Memory: percentages of external memory read/write instructions. MemoryL0: percentages of internal memory read/write instructions. ResourceConflictRatio: percentages of pipeline queue Instructions. NOTE Online inference supports task_trace and aicpu but does not support training_trace. Example: <pre>custom_op.parameter_map["profiling_options"].s = tf.compat.as_bytes({'output':"/tmp/ profiling","training_trace":"on","fp_point":"resnet_model/ conv2d/Conv2Dresnet_model/batch_normalization/ FusedBatchNormV3_Reduce","bp_point":"gradients/ AddN_70"})</pre>	
enable_dump	Data dump enable • True: enabled. The dump file path is read from dump_path. If dump_path is set to None, an exception occurs. • False (default): disabled. Example: <pre>custom_op.parameter_map["enable_dump"].b = True</pre>	Training/Online inference

Item	Description	Use Case
dump_path	Dump path. This parameter is required when enable_dump or enable_dump_debug is set to True. The specified path must be created in advance in the environment (either in a container or on the host) where training is performed. The running user configured during installation must have the read and write permissions on this path. The path can be an absolute path or a relative path relative to the path where the command is executed. • An absolute path starts with a slash (/), for example, /home/HwHiAiUser/output. • A relative path starts with a directory name, for example, output. Example: <pre>custom_op.parameter_map["dump_path"].s = tf.compat.as_bytes("/home/HwHiAiUser/output")</pre>	Training/ Online inference
dump_step	Iterations to dump. Defaults to None, indicating that all iterations are dumped. Separate multiple iterations using vertical bars (), for example, 0 5 10. You can also use hyphens (-) to specify the iteration range, for example, 0 3-5 10. Example: <pre>custom_op.parameter_map["dump_step"].s = tf.compat.as_bytes("0 5 10")</pre>	Training
dump_mode	Dump mode. • input: dumps only operator inputs. • output (default): dumps only operator outputs. • all: dumps both operator inputs and outputs. Example: <pre>custom_op.parameter_map["dump_mode"].s = tf.compat.as_bytes("all")</pre>	Training/ Online inference

Item	Description	Use Case
enable_dump_debug	Overflow detection enable. • True: enabled. The dump file path is read from dump_path. If dump_path is set to None, an exception occurs. • False (default): disabled. Example: <code>custom_op.parameter_map["enable_dump_debug"].b = True</code>	Training
dump_debug_mode	Overflow detection mode. • aicore_overflow: detects AI Core operator overflow, outputting the abnormal maximum values (such as float16 65500, 38400, and 51200) when the inputs of the operator are normal. Once such fault is detected, analyze the cause of the overflow and modify the operator implementation based on the network requirements and operator logic. • atomic_overflow: detects Atomic Add overflow, for checking modules involved in floating-point computing (such as SDMA) in addition to AI Core. • all: detects both AI Core operator overflow and Atomic Add overflow. Example: <code>custom_op.parameter_map["dump_debug_mode"].s = tf.compat.as_bytes("all")</code>	Training

Item	Description	Use Case
precision_mode	A string for the operator precision mode. • allow_fp32_to_fp16 or None: If an operator does not support the float32 data type, the precision is reduced to float16. This mode is selected by default in the training scenario. • force_fp16: If an operator supports both float16 and float32 data types, float16 is forcibly selected. This mode is selected by default in the online inference scenario. • must_keep_origin_dtype: The original precision is retained. • allow_mix_precision: Mixed precision is enabled. For operators of the float32 data type on a network, the precision of some float32 operators can be automatically reduced to float16 based on the built-in optimization policy. In this way, the system performance is improved and the memory usage is reduced with little accuracy loss. After this function is enabled, you can enable loss scaling to compensate for the accuracy drop caused by precision reduction. Example: <pre>custom_op.parameter_map["precision_mode"].s = tf.compat.as_bytes("allow_mix_precision")</pre>	Training/Online inference
variable_format_optimize	Variable format optimization enable • True (default): enabled • False: disabled To improve training efficiency, during the initialization of network variables, the variables are converted into a data format that is more suitable for running on the (for example, NCHW to NC1HWC0 conversion). However, this function should be disabled in special scenarios. Example: <pre>custom_op.parameter_map["variable_format_optimize"].b = True</pre>	Training

Item	Description	Use Case
mix_compile_mode	Mixed computing enable • True: enabled • False (default): disabled In fully offloaded mode, all compute operators are offloaded to the device side. As a supplement to the fully offloaded mode, mixed computing allows some operators to be executed online in the frontend framework, improving the flexibility for adapting to TensorFlow. Example: <code>custom_op.parameter_map["mix_compile_mode"].b = True</code>	Training/Online inference
hcom_parallel	Whether to enable the AllReduce gradient update and forward and backward parallel execution. • True: enabled • False (default): disabled Example: <code>custom_op.parameter_map["hcom_parallel"].b = True</code>	Training
graph_memory_max_size	Network static memory and maximum dynamic memory, which can be specified based on the network size. The value ranges from 0 to 256 x 1024 x 1024 x 1024 or 0 to 274877906944, in bytes. Due to Ascend AI Processor hardware restrictions, the sum of graph_memory_max_size and variable_memory_max_size cannot exceed 31 GB. If not set, 26 GB is used by default. Example: <code>custom_op.parameter_map["graph_memory_max_size"].s = tf.compat.as_bytes(str(26*1024 * 1024 * 1024))</code>	Training/Online inference
variable_memory_max_size	Variable memory, which can be specified based on the network size. The value ranges from 0 to 256 x 1024 x 1024 x 1024 or 0 to 274877906944, in bytes. Due to Ascend AI Processor hardware restrictions, the sum of graph_memory_max_size and variable_memory_max_size cannot exceed 31 GB. If not set, 5 GB is used by default. Example: <code>custom_op.parameter_map["variable_memory_max_size"].s = tf.compat.as_bytes(str(5*1024 * 1024 * 1024))</code>	Training/Online inference

Item	Description	Use Case
auto_tune_mode	You can determine whether to use the Auto Tune tool to tune the TBE operators during build to find the best performance configuration on the . If not specified, automatic tuning is disabled. For details about the usage of the Auto Tune tool, see "Auto Tune Tool Instructions" in <i>CANN Auxiliary Development Tool Guide</i>. Example: <pre>custom_op.parameter_map["auto_tune_mode"].s = tf.compat.as_bytes("RL,GA")</pre>	Training
stream_max_parallel_num	AI CPU and AI Core engine parallelism for parallel execution of AI CPU and AI Core operators. DNN_VM_TF is the name of the AI CPU engine. In this example, the number of concurrent tasks of the AI CPU engine is 10. DNN_V100 is the name of the AI Core engine. In this example, the number of concurrent tasks of the AI Core engine is 1. The parallelism degree of the AI CPU engine and AI Core engine is 1 by default. The value range is [1, 13]. Example: <pre>custom_op.parameter_map["stream_max_parallel_num"]. s = tf.compat.as_bytes("DNN_VM_TF:10,DNN_V100:1")</pre>	Training/ Online inference

Item	Description	Use Case
is_tailing_optimization	Communication hangover optimization enable in distributed training scenarios to improve performance. By changing a computation dependency relationship, a computation operation that does not depend on the last AR (gradient aggregation fragment) is scheduled to be performed in parallel with the last AR, to optimize communication hangover. • True: enabled • False (default): disabled This parameter must be used in pair with NPUOptimizer and the value must be the same as that of is_tailing_optimization in NPUOptimizer. Example: <code>custom_op.parameter_map["is_tailing_optimization"].b = True</code>	Training
graph_run_mode	Graph run mode • 0: online inference • 1 (default): training Example: <code>custom_op.parameter_map["graph_run_mode"].i = 1</code>	Training/ Online inference

Item	Description	Use Case
op_debug_level	Operator debug enable. • 0 (default): disables operator debug. • 1: enables operator debug and generates a TBE instruction mapping file. In this case, an operator CCE file (.cce), a Python-CCE mapping file (_loc.json), and operator .o and .json files are generated in the kernel_meta folder in the training script execution directory. You can locate the AI Core error by using the line numbers in the CCE code and TBE code of the error operator. • 2: enables operator debug and generates a TBE instruction mapping file. In this case, an operator CCE file (.cce), a Python-CCE mapping file (_loc.json), and operator .o and .json files are generated in the kernel_meta folder in the training script execution directory, and the build optimization is disabled by enabling the CCE compiler - O0-g. You can locate the AI Core error by using the line numbers in the CCE code and TBE code of the error operator. • 3: disables operator debug and retains the operator .o and .json files in the kernel_meta folder in the training script execution directory. NOTICE You are advised to set this parameter to 0 or 3 for training. If you need to locate AI Core errors, set this parameter to 1 or 2, which might compromise the network performance. Example: <pre>custom_op.parameter_map["op_debug_level"].i = 0</pre>	Training/Online inference

Item	Description	Use Case
enable_scope_fusion_passes	Scope fusion pattern (or scope fusion patterns separated by commas) to take effect during build. Both built-in and custom scope fusion pattern files are classified into the following two types: • General: common scope fusion patterns applicable to all networks. It takes effect by default and cannot be manually invalidated. • Non-General: scope fusion patterns applicable to specific networks. By default, it does not take effect. You can use enable_scope_fusion_passes to specify the fusion pattern list that takes effect. Example: <pre>custom_op.parameter_map["enable_scope_fusion_passes"].s = tf.compat.as_bytes("ScopeLayerNormPass,ScopeClipBoxesPass")</pre>	Training/Online inference
enable_exception_dump	Whether to dump the inputs and outputs of error operators. • 0 (default): disabled • 1: enabled. Keeping this switch enabled does not compromise the performance. Example: <pre>custom_op.parameter_map["enable_exception_dump"].i = 1</pre>	Training

Item	Description	Use Case
op_select_implmode	Operator implementation mode select. Some operators built in the Ascend AI Processor can be implemented in either high-precision or high-performance mode. • high_precision: high precision implementation mode. In high-precision mode, Newton's Method or Taylor's Formula is used to improve operator precision with fp16 input. • high_performance (default): high performance implementation mode. The high-performance implementation mode refers to the optimal performance implementation without affecting the network precision with fp16 input. Example: <code>custom_op.parameter_map["op_select_implmode"].s = tf.compat.as_bytes("high_precision")</code>	Training/Online inference
optypelist_for_implmode	List of operator types. The operators in the list use the mode specified by the OP_SELECT_IMPL_MODE parameter. Currently, Pooling, SoftmaxV2, LRN, and ROIAlign operators are supported. This parameter is used in pair with the OP_SELECT_IMPL_MODE parameter, for example: Set op_select_implmode to high_precision. Set optypelist_for_implmode to Pooling. Example: <code>custom_op.parameter_map["optypelist_for_implmode"].s = tf.compat.as_bytes("Pooling")</code>	Training/Online inference

Item	Description	Use Case
input_shape	Input shape. Example: <pre>custom_op.parameter_map["input_shape"].s = tf.compat.as_bytes("data:1,1,40,-1;label:1,-1;mask:-1,-1")</pre> In this example, the network model has three inputs: data (1, 1, 40, -1), label (1, -1), and mask (-1, -1). Separate each input and its shape with colons (:). A -1 indicates a dynamic dimension, whose size choices are configured by using dynamic_dims. Precautions: • If a network has both dataset inputs and placeholder inputs, since dynamic inputs are described in only one of the modes, you only need to fill in the shapes of the dynamic inputs. • For scalar inputs, set the shape to 0.	Online inference
dynamic_dims	Input dimension size choices. Separate the dimension sizes by a semicolon (;). The dim values one-to-one correspond to the -1s in the input_shape argument, and the number of -1s equals the number of dimension sizes of each choice. Provide at least one dynamic dimension size choice. The argument must match the input_shape and dynamic_dims arguments. Otherwise, an error is reported and the system exits. Example: <pre>custom_op.parameter_map["dynamic_dims"].s = tf.compat.as_bytes("20,20,1,1;40,40,2,2;80,60,4,4")</pre> Based on the input_shape information in the preceding example, the supported input shape choices are as follows: • Choice 0: data(1, 1, 40, 20)+label(1, 20)+mask(1, 1) • Choice 1: data(1, 1, 40, 40)+label(1, 40)+mask(2, 2) • Choice 2: data(1, 1, 40, 80)+label(1, 60)+mask(4, 4)	Online inference

Item	Description	Use Case
dynamic_node_type	Type of the dynamic input node. • 0: dataset input • 1: placeholder input Dynamic inputs can be described only in one of the modes, dataset or placeholder. Example: <code>custom_op.parameter_map["dynamic_node_type"].i = 0</code>	Online inference
mstune_mode	This parameter is not supported in the current version.	Training
work_path	This parameter is not supported in the current version.	Training
buffer_optimize	Buffer optimization enable. • l2_optimize (default): enabled • off_optimize: disabled Example: <code>custom_op.parameter_map["buffer_optimize"].s = tf.compat.as_bytes("l2_optimize")</code>	Online inference
enable_small_channel	Small channel optimization enable. If this function is enabled, performance benefits are generated at the convolutional layers with channel ≤ 4. You are advised to enable this function in inference scenarios. • 0 (default): disabled • 1: enabled NOTE After this function is included, performance benefits can be obtained on the ResNet-50, ResNet-101, and ResNet-152 networks. For other networks, the performance may deteriorate. Example: <code>custom_op.parameter_map["enable_small_channel"].i = 1</code>	Online inference

Item	Description	Use Case
fusion_switch_file	Directory of the fusion switch configuration file, including the file name. The value can contain letters, digits, underscores (_), hyphens (-), periods (.), and Chinese characters. The system is built in with a range of graph fusion and UB fusion patterns, which are enabled by default. You can disable any rule as required. The following is a template of the fusion_switch.cfg configuration file. on indicates that a fusion pattern is enabled, and off indicates that a fusion pattern is disabled. <pre> { "Switch":{ "GraphFusion":{ "RequantFusionPass": "on", "ConvToFullyConnectionFusionPass": "off", "SoftmaxFusionPass": "on", "NotRequantFusionPass": "on", "SplitConvConcatFusionPass": "on", "ConvConcatFusionPass": "on", "MatMulBiasAddFusionPass": "on", "PoolingFusionPass": "on", "ZConcatv2dFusionPass": "on", "ZConcatExt2FusionPass": "on", "TfMergeSubFusionPass": "on" }, "UBFusion":{ "TbePool2dQuantFusionPass": "on" } } } </pre> Example: <pre> custom_op.parameter_map["fusion_switch_file"].s = tf.compat.as_bytes("/home/test/fusion_switch.cfg") </pre>	Training/Online inference

Item	Description	Use Case
op_compiler_cache_mode	Disk cache enable for operator build. • enable: enabled. If enabled, operators with the same build configurations and operator configurations will not be built repeatedly, thus accelerating the build speed. • disable (default): disabled. • force: forcibly flushes the cache. That is, the existing cache is cleared up before adding new operator builds to the cache. This is especially useful when the Python libraries or dependent libraries are changed. Restrictions: • It needs to be used in pair with op_compiler_cache_dir. The cache function is enabled only when op_debug_level is set to 0 or 3. • If set to force, the existing cache will be cleared. Therefore, it is not recommended for parallel program compilation. Otherwise, the cache used by other models may be cleared, causing build failures. • disable and force are recommended for publishing the final model. Example: <pre>custom_op.parameter_map["op_compiler_cache_mode"].s = tf.compat.as_bytes("enable")</pre>	Training/Online inference
op_compiler_cache_dir	Disk cache directory for operator build. The value can contain letters, digits, underscores (_), hyphens (-), periods (.), and Chinese characters. If the specified directory exists and is valid, a kernel_cache subdirectory is automatically created. If the specified directory does not exist but is valid, the system automatically creates this directory and the kernel_cache subdirectory. Defaults to \$HOME/atc_data. Example: <pre>custom_op.parameter_map["op_compiler_cache_dir"].s = tf.compat.as_bytes("/home/test/kernel_cache")</pre>	Training/Online inference

Item	Description	Use Case
debug_dir	Directory of the debug files generated during the operator build process, including the .o, .json, and .cce files. Defaults to the execution path of the current script. Example: custom_op.parameter_map["debug_dir"].s = tf.compat.as_bytes("/home/test")	Training/Online inference

Example

```
import tensorflow as tf
from npu_bridge.estimator import npu_ops
from tensorflow.core.protobuf.rewriter_config_pb2 import RewriterConfig

config = tf.ConfigProto()
custom_op = config.graph_options.rewrite_options.custom_optimizers.add()
custom_op.name = "NpuOptimizer"
custom_op.parameter_map["use_off_line"].b = True
config.graph_options.rewrite_options.remapping = RewriterConfig.OFF # Disable remapping.

with tf.Session(config=config) as sess:
    sess.run(cost)
```

10.2.3 Collective Communication APIs

10.2.3.1 Overview

The high-level API **NPUDistributedOptimizer** enables the user to automatically complete gradient aggregation without sensing AllReduce, implementing data parallel training. In addition, to meet the users' requirements for flexibility, collective communication provides common APIs such as rank management, gradient segmentation, and collective communication prototype.

To use the collective communication APIs, the FwkACCLib and TFPlugin software packages must be installed.

- If the **--pylocal** mode is used for the installation of FwkACCLib and TFPlugin, the corresponding .whl files are installed in the software installation paths, which are **/fwkacclib/python/site-packages** in the FwkACCLib installation path and **/tfplugin/python/site-packages/** in the TFPlugin installation path.
- If the **--pylocal** mode is not used to for the installation of FwkACCLib and TFPlugin, the corresponding .whl files are installed in the local Python path.

10.2.3.2 hccl.manage.api

10.2.3.2.1 create_group

Prototype

```
def create_group(group, rank_num, rank_ids)
```

Description

Creates a user-defined group for collective communication.

Parameters

Parameter	Input/ Output	Description
group	Input	A string containing a maximum of 128 bytes, including the end character. Group name, which is the ID of the collective communication group. The value cannot be hccl_world_group . hccl_world_group is the default group created by the ranktable file and cannot be created using this API. If hccl_world_group is passed to this parameter, group creation fails.
rank_num	Input	An int. Number of ranks in a group. The maximum value is 4096 .

Parameter	Input/Output	Description
rank_ids	Input	A list. List of world_rank_ids that form the group. Different types of boards have different restrictions. In the single-server scenario, rank_ids must meet the following requirement: The number of ranks must be 1, 2, 4, or 8. When the rank size is 2 or 4, the selected NPUs must reside in the same cluster. In the server cluster scenario, rank_ids must meet the following requirements: • The number of ranks selected for each server must be the same (the number must be 1, 2, 4, or 8). • The positions of ranks selected for each server must be the same. That is, the rank IDs must be the same based on the modulo operation of 8. • When the number of ranks selected for each server is 2 or 4, the selected Ascend AI Processors must belong to the same cluster. That is, the remainder of the rank ID divided by 8 is less than 4, or greater than or equal to 4. • Example: Assume that a group is created for three servers and the rank IDs of the three servers are {0,1,2,3,4,5,6,7}, {8,9,10,11,12,13,14,15}, and {16,17,18,19,20,21,22,23}. rank_ids that meets the requirement may be: rank_ids=[1, 9, 17]; rank_ids=[1,2,9,10,17,18]; rank_ids=[4,5,6,7,12,13,14,15,20,21,22,23]. In the Atlas 300T training card (model: 9000) scenario, the number of ranks must be 1, 2, 4, or 8.

Returns

None

Example

```
from hccl.manage.api import create_group
create_group(" myGroup", 4, [0, 1, 2, 3])
```

Restrictions

- This API must be called after the initialization of collective communication is complete.
- The caller rank must be within the range defined by **group** in the current API. Otherwise, the API fails to be called.
- If a group with the same name is created repeatedly, group creation fails.
- This API is available only when the number of ranks in the ranktable file is 1, 2, 4, or 8.

10.2.3.2.2 destroy_group

Prototype

```
def destroy_group(group)
```

Description

Destroy a user-defined group.

Parameters

Parameter	Input/Output	Description
group	Input	A string of up to 128 bytes, including the end character. Group name, which is the ID of the collective communication group.

Returns

None

Example

```
from npu_bridge.npu_init import *  
create_group(" myGroup", 4, [0, 1, 2, 3])  
destroy_group(" myGroup ")
```

Restrictions

- This API must be called after the initialization of collective communication is complete.
- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- Groups with the same name must be used in conjunction with [10.2.3.2.2 destroy_group](#) and [10.2.3.2.1 create_group](#) and called after **create_group** is complete.

- If the group transferred by the user is **hccl_world_group** (default group), the group fails to be destroyed.

10.2.3.2.3 get_rank_size

Prototype

```
def get_rank_size(group="hccl_world_group")
```

Description

Obtains the number of ranks (that is, the number of devices) in a group.

Parameters

Parameter	Input/Output	Description
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be user-defined. If no argument is passed, the default value hccl_world_group is used.

Returns

An int for the number of ranks in a group.

Example

```
from npu_bridge.npu_init import *  
create_group(" myGroup", 4, [0, 1, 2, 3])  
rankSize = get_rank_size("myGroup")  
#rankSize = 4
```

Restrictions

- This API must be called after the initialization of collective communication is complete.
- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- After [10.2.3.2.1 create_group](#) is complete, this API is called to obtain the number of ranks in the current group.
- If **hccl_world_group** is passed, the number of ranks in **world_group** is returned.

10.2.3.2.4 get_local_rank_size

Prototype

```
def get_local_rank_size(group="hccl_world_group")
```

Description

Obtains the number of local ranks on the server where the devices in the group are located.

Parameters

Parameter	Input/Output	Description
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be user-defined. If no argument is passed, the default value hccl_world_group is used.

Returns

An int for the number of local ranks on the server where the device is located.

Example

```
from npu_bridge.npu_init import *  
create_group(" myGroup", 4, [0, 1, 2, 3])  
localRankSize = get_local_rank_size("myGroup")  
#localRankSize = 1
```

Restrictions

- This API must be called after the initialization of collective communication is complete.
- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- After [10.2.3.2.1 create_group](#) is complete, this API is called to obtain the number of local ranks in the current group.
- If **hccl_world_group** is passed, the number of local ranks in **world_group** is returned.

10.2.3.2.5 get_rank_id

Prototype

```
def get_rank_id(group="hccl_world_group")
```

Description

Obtains the rank ID of a device in a group.

Parameters

Parameter	Input/Output	Description
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be user-defined. If no argument is passed, the default value hccl_world_group is used.

Returns

An int for the rank ID of the group to which the device belongs is returned.

Example

```
from npu_bridge.npu_init import *
create_group(" myGroup", 4, [0, 1, 2, 3])
rankId = get_rank_id(" myGroup ")
#rankId = 0/1/2/3
```

Restrictions

- This API must be called after the initialization of collective communication is complete.
- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- After [10.2.3.2.1 create_group](#) is complete, this API is called to obtain the rank ID of a process in a group.
- If **hccl_world_group** is passed, the rank ID of the process in **world_group** is returned.

10.2.3.2.6 get_local_rank_id

Prototype

```
def get_local_rank_id(group="hccl_world_group")
```

Description

Obtains the local rank ID of a device in a group.

Parameters

Parameter	Input/Output	Description
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be user-defined. If no argument is passed, the default value hccl_world_group is used.

Returns

An int for the ID of the local rank on the server where the device is located.

Example

```
from npu_bridge.npu_init import *  
create_group(" myGroup", 4, [0, 1, 2, 3])  
localRankId = get_local_rank_id("myGroup")  
#rankId = 0
```

Restrictions

- This API must be called after the initialization of collective communication is complete.
- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- After [10.2.3.2.1 create_group](#) is complete, this API is called to obtain the local rank ID of a process in a group.
- If **hccl_world_group** is passed, the local rank ID of the process in **world_group** is returned.

10.2.3.2.7 get_world_rank_from_group_rank

Prototype

```
def get_world_rank_from_group_rank(group, group_rank_id)
```

Description

Obtains the world rank ID of the process using the group rank ID.

Parameters

Parameter	Input/Output	Description
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group .
group_rank_id	Input	An int. Rank ID of a process in a group

Returns

An int for the rank ID of the process in **hccl_world_group**.

Example

```
from npu_bridge.npu_init import *
create_group(" myGroup", 4, [0, 1, 2, 3])
worldRankId = get_world_rank_from_group_rank (" myGroup", 1)
#worldRankId = 8
```

Restrictions

- This API must be called after the initialization of collective communication is complete.
- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- After [10.2.3.2.1 create_group](#) is complete, this API is called to convert the group rank ID to the world rank ID.

10.2.3.2.8 get_group_rank_from_world_rank

Prototype

```
def get_group_rank_from_world_rank(world_rank_id, group)
```

Description

Obtains the group rank ID of the process in the group using the world rank ID.

Parameters

Parameter	Input/Output	Description
world_rank_id	Input	An int. Rank ID of the process in hccl_world_group

Parameter	Input/ Output	Description
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group .

Returns

An int for the rank ID of a process in a group.

Example

```
from npu_bridge.npu_init import *  
create_group(" myGroup", 4, [0, 1, 2, 3])  
groupRankId = get_group_rank_from_world_rank (8, " myGroup ")  
#groupRankId = 1
```

Restrictions

- This API must be called after the initialization of collective communication is complete.
- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- After [10.2.3.2.1 create_group](#) is complete, this API is called to convert the world rank ID to the group rank ID.

10.2.3.3 hccl.split.api

10.2.3.3.1 set_split_strategy_by_idx

Prototype

```
def set_split_strategy_by_idx(idxList, group="hccl_world_group")
```

Description

Sets the backward gradient segmentation policy in the collective communication group based on the gradient index ID to implement the convergence of AllReduce.

Parameters

Parameter	Input/Output	Description
idxList	Input	A list. Index ID list of gradients. - The index ID list of the gradient must be a non-negative number in ascending order. - The gradient index ID must be set based on the total number of gradient parameters of the model. The index ID starts from 0. The maximum value can be obtained as follows: - Do not call the gradient split API to set the gradient segmentation policy for training. In this case, the script uses the default gradient segmentation mode in 10.2.3.3.2 set_split_strategy_by_size for training. - After the training is complete, search for the keyword segment result in the host training log at the info level to obtain the segmentation information of gradient segmentation, for example, segment index list: [0,107] [108,159]. The maximum number (for example, 159) in the segment sequence is the maximum value of the total gradient parameter index. NOTE During the training process, logs may be overwritten. In this case, you can modify LogAgentMaxFileNum in /var/log/npu/conf/slog/slog.conf to increase the number of log files that can be stored on the host side. Alternatively, you can also perform only one iteration. - A maximum of eight gradient segments are supported. - For example, if a model has 160 parameters that generate gradients and need to be divided into three segments: [0, 20], [21, 100], and [101, 159], set idxList to [20, 100, 159].
group	Input	A string. Group name, which can be a user-defined value or hccl_world_group. Defaults to hccl_world_group.

Returns

None

Example

```
from npu_bridge.npu_init import *  
set_split_strategy_by_idx ([20, 100, 159], "group")
```

Restrictions

- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- If you do not call the gradient segmentation API to set the segmentation policy, the default backward gradient segmentation policy is used.

Default segmentation policy: two segments with the first taking up 96.54% of the gradient data volume, and the second segment taking up 3.46% (In some cases, there is only one segment).

10.2.3.3.2 set_split_strategy_by_size

Prototype

```
def set_split_strategy_by_size(dataSizeList, group="hccl_world_group")
```

Function

Sets the backward gradient segmentation policy in the collective communication group based on the data volume percentage to implement the convergence of AllReduce.

Parameters

Parameter	Input/Output	Description
dataSizeList	Input	A list. List of gradient parameter data volume percentages. • The index ID list of the gradient must be non-negative, and the total percentage of the gradient data volume sequence must be 100. • A maximum of eight gradient segments are supported. • For example, if the model has 150 MB gradient data and needs to be divided into three segments: 90 MB, 30 MB, and 30 MB, set dataSizeList to [60, 20, 20].

Parameter	Input/ Output	Description
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group . Defaults to hccl_world_group .

Returns

None

Example

```
from npu_bridge.npu_init import *  
set_split_strategy_by_size ([60, 20, 20], "group")
```

Restrictions

- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- When the backward gradient segmentation policy is set based on both the gradient data volume percentage and the gradient index ID, the setting result based on the gradient data volume percentage is preferred.
- If you do not call the gradient segmentation API to set the segmentation policy, the default backward gradient segmentation policy is used.

Default segmentation policy: The optimal segmentation location of ResNet-50 is as follows: ResNet-50 is divided into two segments based on the gradient data volume. The data volume of the first segment is 96.54%, and that of the second segment is 3.46%.

10.2.3.4 npu_bridge.hccl.hccl_ops

10.2.3.4.1 allreduce

Prototype

```
def allreduce(tensor, reduction, fusion=1, fusion_id=-1, group =  
"hccl_world_group")
```

Description

Provides the AllReduce function for collective communication in a group to reduce tensors with the same name on all nodes.

Parameters

Parameter	Input/Output	Description
tensor	Input	TensorFlow tensor type Must be one of the following data types: int8, int32, float16, float32.
reduction	Input	A string. Operator type for reduction. The value can be max , min , prod , or sum .
fusion	Input	An int. Fusion flag of the AllReduce operator • 0: not fused. The AllReduce operator is not fused with other AllReduce operators. • 1 (default): The AllReduce operator is fused based on the gradient segmentation policy. • 2: AllReduce operators with the same fusion_id are fused.
fusion_id	Input	Fusion ID of the AllReduce operator AllReduce operators with the same fusion_id are fused.
group	Input	A string containing a maximum of 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group .

Returns

The result tensor

Example

```
from npu_bridge.hccl import hccl_ops  
result = hccl_ops.allreduce(tensor, "sum")
```

Restrictions

- The caller rank must be within the range defined by **group** in the current API. Otherwise, the API fails to be called.
- The upstream node of AllReduce must not be variable.
- The input tensor size must be less than or equal to 8 GB.
- The AllReduce operator can be fused only when the **reduction** is set to **sum**.

10.2.3.4.2 allgather

Prototype

```
def allgather(tensor, rank_size, group = "hccl_world_group")
```

Description

Each device receives the aggregation of tensor data from all ranks in the order of the ranks.

Parameters

Parameter	Input/Output	Description
tensor	Input	TensorFlow tensor type. Must be one of the following data types: int8, int32, float16, float32, int64, uint64
rank_size	Input	An int. Number of devices in a group. The maximum value is 4096 .
group	Input	A string containing a maximum of 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group .

Returns

The result tensor

Example

```
from npu_bridge.hccl import hccl_ops
rank_size = 2
result = hccl_ops.allgather (tensor, rank_size)
```

Restrictions

- The caller rank must be within the range defined by **group** in the current API. Otherwise, the API fails to be called.

10.2.3.4.3 broadcast

Prototype

```
def broadcast(tensor, root_rank, fusion=0,fusion_id=-1,group = "hccl_world_group")
```

Description

Copies an N-element buffer on the root rank to all ranks.

Parameters

Parameter	Input/Output	Description
tensor	Input	TensorFlow tensor type. A list. Must be one of the following data types: int8, int32, float16, float32, int64, uint64
root_rank	Input	An int. Rank ID of the root node. The ID is the rank ID in the group.
group	Input	A string containing a maximum of 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group .
fusion	Input	An int. Broadcast operator fusion enable. • 0 (default): disabled. The AllReduce operator is not fused with other AllReduce operators. • 2: enabled. AllReduce operators with the same fusion_id are fused. • Other values: invalid
fusion_id	Input	Fusion ID of the Broadcast operator. Broadcast operators with the same fusion_id are fused.

Returns

The result tensor

Example

```
from npu_bridge.hccl import hccl_ops
root = 0
inputs = [tensor]
result = hccl_ops.broadcast(inputs, root)
```

Restrictions

- The caller rank must be within the range defined by **group** in the current API. Otherwise, the API fails to be called.
- In **sess.run** mode, if you call this API to fuse broadcast operators, the **GradFusionOptimizer** cannot be used at the same time.

- In **Estimator** mode, this API cannot be called to fuse broadcast operators.

10.2.3.4.4 reduce_scatter

Prototype

```
def reduce_scatter(tensor, reduction, rank_size, group = "hccl_world_group")
```

Description

Performs the same operation as the Reduce operation, except that each rank receives a subpart of the result. The Reduce operation is specified by the **reduction** parameter.

Parameters

Parameter	Input/Output	Description
tensor	Input	TensorFlow tensor type Must be one of the following data types: int8, int32, float16, float32. The size of the first dimension of a tensor must be an integer multiple of the rank size.
reduction	Input	A string. Operator type for reduction. The value can be max , min , prod , or sum .
rank_size	Input	An int. Number of devices in a group. The maximum value is 4096 .
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group .

Returns

The result tensor. It is recommended that the result tensor size be 32-byte aligned. Otherwise, the performance deteriorates.

Example

```
from npu_bridge.npu_init import *  
rank_size = 2  
result = hccl_ops.reduce_scatter(tensor, "sum", rank_size)
```

Restrictions

- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- The input tensor size must be less than or equal to 8 GB.

10.2.3.4.5 send

Prototype

```
def send(tensor, sr_tag, dest_rank, group = "hccl_world_group")
```

Description

Sends data to a rank within a collective communication group.

Parameters

Parameter	Input/Output	Description
tensor	Input	TensorFlow tensor type Must be one of the following data types: int8, int32, float16, float32.
sr_tag	Input	An int. Message tag. The send/recv pairs with the same sr_tag can receive and send data.
dest_rank	Input	An int. Target node of data. rank indicates the rank ID in the group.
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group .

Returns

None

Example

```
from npu_bridge.npu_init import *  
sr_tag = 0  
dest_rank = 1  
hccl_ops.send(tensor, sr_tag, dest_rank)
```

Restrictions

- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.

- Only point-to-point data transmit and receive between ranks with the same ID from different servers are supported.
- The Send and Receive functions must be used in pairs and depend on other operators in the graph.

10.2.3.4.6 receive

Prototype

```
def receive(shape, data_type, sr_tag, src_rank, group = "hccl_world_group")
```

Description

Receives data from a rank within a collective communication group.

Parameters

Parameter	Input/Output	Description
shape	Input	Shape of the received tensor.
data_type	Input	Data type of the received data. Must be one of the following data types: int8, int32, float16, float32.
sr_tag	Input	An int. Message tag. The send/recv pairs with the same sr_tag can receive and send data.
src_rank	Input	An int. Source node of the received data. rank indicates the rank ID in the group.
group	Input	A string of up to 128 bytes, including the end character. Group name, which can be a user-defined value or hccl_world_group .

Returns

The result tensor

Example

```
from npu_bridge.npu_init import *  
sr_tag = 0  
src_rank = 0  
tensor = hccl_ops.receive(tensor.shape, tensor.dtype, sr_tag, src_rank)
```

Restrictions

- The caller rank must be within the range defined by the **group** argument passed to this API call. Otherwise, the API call fails.
- Only point-to-point data transmit and receive between ranks with the same ID from different servers are supported.
- The Send and Receive functions must be used in pairs and depend on other operators in the graph.

10.3 Environment Variables

10.3.1 JOB_ID

Description

Sets the training job ID, which is user-defined. Allows letters, digits, hyphens (-), and underscores (_) only. You are not advised to set **JOB_ID** to pure digits starting with 0.

Example

```
export JOB_ID=10087
```

10.3.2 ASCEND_DEVICE_ID

Description

Sets the logical ID of a processor.

The value range is $[0, N - 1]$ and the default value is **0**. N indicates the device count in the physical machine, VM, or container.

Example

```
export ASCEND_DEVICE_ID=0
```

10.3.3 RANK_TABLE_FILE

Description

Sets the processor resource information of distributed training, that is, the directory of the ranktable file, including the file name. For details, see [10.4 Device Resource Configuration File Templates](#).

Example

```
export RANK_TABLE_FILE=/home/test/ranktable.json
```

10.3.4 RANK_ID

Description

Sets the rank ID corresponding to the training process in the collective communication process group.

When ranktable template 1 is used, the value of this parameter is the same as that of **rank_id**.

When ranktable template 2 is used, the value of this parameter is the same as that of **pod_name**.

Example

```
export RANK_ID=0
```

10.3.5 RANK_SIZE

Description

Sets the cluster rank size corresponding to the current training process, that is, the number of devices in the cluster.

Example

```
export RANK_SIZE=2
```

10.3.6 GE_USE_STATIC_MEMORY

Description

Enables static memory allocation for network execution.

Set this parameter to **1** to enable static memory allocation. This is especially useful when executing a deep inference network, for example, the BERT24 network whose intermediate data volume in feature map computation could reach 25 GB. In this case, enabling static memory allocation can improve the collaboration efficiency between the communication DIMMs in multi-device scenarios. You are advised to retain the default value to use dynamic memory allocation for networks other than BERT24.

In static memory allocation mode, the default allocation is 31 GB, which is determined by the sum of **graph_memory_max_size** and **variable_memory_max_size**. In dynamic memory allocation mode, the allocation is within the sum of **graph_memory_max_size** and **variable_memory_max_size**.

Example

```
export GE_USE_STATIC_MEMORY=1
```

10.3.7 PROFILING_MODE

Description

Enables Profiling.

- **true**: enabled. The option to be traced is determined by **PROFILING_OPTIONS**.
- **false** or not configured: disabled.

Example

```
export PROFILING_MODE=true
```

10.3.8 PROFILING_OPTIONS

Description

Sets Profiling options.

- **output**: path for storing profiling result files. Create the specified path in advance in the environment (either in a container or on the host) where training is performed. The running user configured during installation must have the read and write permissions on this path. The path can be an absolute path or a relative path relative to the path where the training script is executed.
 - An absolute path starts with a slash (/), for example, **/home/HwHiAiUser/output**.
 - A relative path starts with a directory name, for example, **output**.
- **training_trace**: iteration tracing switch. Collects software profile data of a training job and the AI Software Stack to profile the training job. Focuses on data augmentation, forward and backward propagation, and gradient aggregation and update. Either **on** or **off**. A value other than **on** or **off** is equivalent to **off**.
- **task_trace**: task tracing switch. Collects the HWTS hardware information of the and the start and end of each task. Either **on** or **off**. A value other than **on** or **off** is equivalent to **off**.
- **aicpu**: AI CPU data augmentation profiling switch. Either **on** or **off**. A value other than **on** or **off** is equivalent to **off**.
- **fp_point**: required when **training_trace** is **on**. Specifies the start point of the forward propagated operator in iteration traces, to record the start timestamp of forward propagation. Set the value to the name of the top operator in forward propagation. You can save the graph as a .pbtxt file by using **tf.io.write_graph** in the training script to obtain this name.
- **bp_point**: required when **training_trace** is **on**. Specifies the end point of the backward propagated operator in iteration traces, to record the end timestamp of backward propagation. **BP_POINT** and **FP_POINT** are used to compute the time used by forward and backward propagation. Set the value to the name of the bottom operator in backward propagation. You can save the graph as a .pbtxt file by using **tf.io.write_graph** in the training script to obtain this name.

- **aic_metrics**: AI Core metrics to profile.
ArithmeticUtilization: percentages of arithmetic utilization.
PipeUtilization (default): percentages of time taken by the compute units and MTEs.
Memory: percentages of external memory read/write instructions.
MemoryL0: percentages of internal memory read/write instructions.
ResourceConflictRatio: percentages of pipeline queue Instructions.

 NOTE

Online inference supports **task_trace** and **aicpu** but does not support **training_trace**.

Example

```
export PROFILING_OPTIONS='{ "output": "/tmp/  
profiling", "training_trace": "on", "task_trace": "on", "aicpu": "on", "fp_point": "resnet_model/conv2d/  
Conv2Dresnet_model/batch_normalization/FusedBatchNormV3_Reduce", "bp_point": "gradients/  
AddN_70", "aic_metrics": "PipeUtilization" }'
```

10.3.9 SKT_ENABLE

Description

Enables superkernel. If enabled, operator tasks are fused into one for delivery to accelerate task scheduling and network execution.

- **1**: enabled.
- **0**: disabled. The default value is **false**.

Example

```
export SKT_ENABLE=1
```

10.3.10 OP_NO_REUSE_MEM

Description

Selects the operator to skip in memory reuse. (Memory reuse is enabled by default.)

The specified operator (or operators separated by commas) will use exclusively allocated memory.

Examples

- Configuration by node name:

```
export OP_NO_REUSE_MEM=gradients/logits/semantic/kernel/Regularizer/l2_regularizer_grad/  
Mul_1,resnet_v1_50/conv1_1/BatchNorm/AssignMovingAvg2
```
- Configuration by operator type:

```
export OP_NO_REUSE_MEM=FusedMulAddN,BatchNorm
```
- Configuration by operator name and operator type:

```
export OP_NO_REUSE_MEM=FusedMulAddN, resnet_v1_50/conv1_1/BatchNorm/AssignMovingAvg
```

10.3.11 ENABLE_NETWORK_ANALYSIS_DEBUG

Description

Ignores graph build failures. If this environment variable is set (to any value), GE always returns a success even if graph build fails. In this way, the adapter can still deliver the graph to GE.

Example

```
export ENABLE_NETWORK_ANALYSIS_DEBUG=1
```

10.3.12 DUMP_GE_GRAPH

Description

Sets the graph dump mode.

- 1: dumps all.
- 2: dumps without data such as weights.
- 3: dumps only node relationships.

Example

```
export DUMP_GE_GRAPH=1
```

10.3.13 DUMP_GRAPH_LEVEL

Description

Sets the graph to dump.

- 1: dumps all graphs.
- 2 (default): dumps all graphs except sub-graphs.
- 3: dumps the generated built graph.

This environment variable takes effect only when **DUMP_GE_GRAPH** is enabled.

Example

```
export DUMP_GRAPH_LEVEL=1
```

10.3.14 TE_PARALLEL_COMPILER

Description

Sets the maximum number of parallel operator build processes. Defaults to **8**. When the value is greater than **0**, parallel build is enabled.

Parallel build is especially useful when a large network is used. The maximum value is calculated as follows: Maximum value = 80% the number of CPU cores/ Number of Ascend AI Processors.

Example

```
export TE_PARALLEL_COMPILER=8
```

10.3.15 ASCEND_MAX_OP_CACHE_SIZE

Description

Sets the maximum size (MB) of the cache folder of a specified processor in the scenario where the operator build cache function is enabled. Defaults to **500** (MB).

Example

```
export ASCEND_MAX_OP_CACHE_SIZE=500
```

10.3.16 ASCEND_REMAIN_CACHE_SIZE_RATIO

Description

Specifies how much (in percentage) of the build cache space is retained when the build cache space of a specified processor reaches **MAX_OP_CACHE_SIZE** in the scenario where the operator build cache function is enabled. Defaults to **50** (%).

Example

```
export ASCEND_REMAIN_CACHE_SIZE_RATIO=50
```

10.3.17 HCCL_INTRA_ROCE_ENABLE

Description

Applies to Atlas 300T training card (model: 9000) only. Specifies whether to use the RoCE path for multi-device communication within the server.

Example

```
export HCCL_INTRA_ROCE_ENABLE=1
```

10.3.18 HCCL_INTRA_PCIE_ENABLE

Description

Applies to Atlas 300T training card (model: 9000) only. Specifies whether to use the PCIe path for multi-processor communication within the server. Use this environment variable in conjunction with **HCCL_INTRA_ROCE_ENABLE**.

HCCL_INTRA_PCIE_ENABLE and **HCCL_INTRA_ROCE_ENABLE** control only the communication mode between the Ascend AI Processors in a server in the Atlas 300T training card (model: 9000) scenario. The mode of communication between servers is fixed to RoCE path communication. The following describes the configuration combination of **HCCL_INTRA_PCIE_ENABLE** and **HCCL_INTRA_ROCE_ENABLE**.

- If neither **HCCL_INTRA_PCIE_ENABLE** or **HCCL_INTRA_ROCE_ENABLE** is configured or they are both set to **0**, the PCIe path is used for communication between the Ascend AI Processors in a server.
- If **HCCL_INTRA_PCIE_ENABLE** is set to **1** and **HCCL_INTRA_ROCE_ENABLE** is set to **0**, the PCIe path is used for communication between the Ascend AI Processors in a server.
- If **HCCL_INTRA_PCIE_ENABLE** is set to **0** and **HCCL_INTRA_ROCE_ENABLE** is set to **1**, the RoCE path is used for communication between the Ascend AI Processors in a server (**recommended**).

 NOTE

HCCL_INTRA_PCIE_ENABLE and **HCCL_INTRA_ROCE_ENABLE** cannot be both set to 1.

Example

```
export HCCL_INTRA_PCIE_ENABLE=1
```

10.3.19 ASCEND_SLOG_PRINT_TO_STDOUT

Description

Enables log printing.

- **0** or not configured: disabled.
- **1**: enabled.

Example

```
export ASCEND_SLOG_PRINT_TO_STDOUT=1
```

10.3.20 ASCEND_GLOBAL_LOG_LEVEL

Description

Sets the global log level of application logs.

- **0**: DEBUG
- **1**: INFO
- **2**: WARNING
- **3**: ERROR
- **4**: NULL (no log output)
- Other values: invalid

Example

```
export ASCEND_GLOBAL_LOG_LEVEL=1
```

10.3.21 ASCEND_GLOBAL_EVENT_ENABLE

Description

Enables event log level of application logs.

- 0: disabled
- 1: enabled
- Other values: invalid

If the environment variable is not set or set to an invalid value, the event log level is enabled by default.

Example

```
export ASCEND_GLOBAL_EVENT_ENABLE=0
```

10.3.22 ASCEND_LOG_DEVICE_FLUSH_TIMEOUT

Description

Sets the time for transferring application logs from the device to the host. Must be set to an integer within the range of 0–180000, in milliseconds.

If the environment variable is not set or set to an invalid value, the default timeout interval 3000 ms is used.

Example

```
export ASCEND_LOG_DEVICE_FLUSH_TIMEOUT=5000
```

10.4 Device Resource Configuration File Templates

Overview

Before training, you need to prepare a device resource configuration file (that is, a ranktable file) and upload it to the operating environment. A ranktable file describes the Ascend AI Processor resources used for training and its path is specified by the **RANK_TABLE_FILE** environment variable.

A ranktable file is in JSON format. For example, for training with two Ascend AI Processors (that is, two devices), the file can be named **rank_table_2p.json**.

Precautions

1. Configure the number of devices used in training in the ranktable file. Currently, two configuration templates are supported. Template 1 is recommended for new scenarios, and template 2 is compatible with some existing scenarios.
2. In single-server or cluster scenarios, the number of configured devices must be one, two, four, or eight times the number of servers participating in training. When the number of devices used for training is two or four times

the number of the training servers, devices 0–3 and devices 4–7 form a network respectively, and cross-network cluster creation is not supported.

3. For training with Atlas 300T training card (model: 9000), the number of configured devices must not be greater than the number of devices in the server, and **template 1** must be used for the configuration.

Template 1 (Recommended)

```
{
  "server_count": "1", // Number of servers. In this example, there is only one AI Server.
  "server_list": [
    {
      "device": [// List of devices in the server
        {
          "device_id": "0", // Device HDC channel ID
          "device_ip": "192.168.1.8", // Actual NIC IP address of the device
          "rank_id": "0" // Rank ID, indexed starting at 0
        },
        {
          "device_id": "4",
          "device_ip": "192.168.1.9",
          "rank_id": "1"
        }
      ],
      "server_id": "10.0.0.10" // Server ID, which is an IP address in dotted decimal notation
    }
  ],
  "status": "completed", // Ranktable availability flag. The value completed indicates that the ranktable is available.
  "version": "1.0" // Version of the ranktable template. Must be 1.0.
}
```

Table 10-9 Description of the ranktable file (template 1)

Item	Description	Required/Optional
server_count	Number of server instances involved in training	Required
status	Availability flag of the rank table ● completed: The rank table is available and the training can be performed. ● initializing: The rank table is unavailable and the training cannot be performed.	Required
version	Version of a ranktable template. Currently, only 1.0 is supported.	Required
server_list	List of server instances involved in training.	Required
server_id	Physical IP address of a server, which is a character string in dotted decimal notation, for example, 10.0.0.10 .	Required

Item	Description	Required/Optional
device_id	Physical ID of a device, that is, the serial number of the device on the server, corresponding to the HDC channel ID. Value range: 0–7	Required
device_ip	IP address of the on-device NIC, which is a globally unique character string in dotted decimal notation, for example, 192.168.1.8 . You can run the cat /etc/hccn.conf command on the current server to obtain the IP address of the NIC. NOTE - In a typical 8-network-port communication scenario, the devices field in the ranktable file records the IP address of the ETH port available for each device. The following is the information about the device whose device_id is 0 and device_ip is 192.168.100.101. "devices": [{"device_id": "0", "device_ip": "192.168.100.101"}] - In this scenario, the information about the available one, two, or four ETH ports of each server is obtained through ranktable verification. If the device_ip field in devices is an empty string "", the ETH port of the device is not used in collective communication. For example, the following shows the "devices" information when device_id is 0 and the ETH port is not used: "devices": [{"device_id": "0", "device_ip": ""}]	Required
rank_id	Unique rank ID, which starts from 0 and is globally unique. The value range is [0, Total number of devices – 1].	Required

Template 2 (Compatible with Some Existing Scenarios)

```
{
  "status": "completed", // Ranktable availability flag. The value completed indicates that the ranktable is
  available.
  "group_count": "1", // Number of groups. The recommended value is 1.
  "group_list": // List of groups
  [
    {
      "group_name": "hccl_world_group", // Group name. The recommended value is hccl_world_group.
      "instance_count": "2", // Number of instances, which can be considered as the number of containers
      in the container scenario.
      "device_count": "2", // Number of all devices in the group
      "instance_list": [
        {
          "pod_name": "tf-bae41", // Instance name, which is generally the container name.
          "server_id": "10.0.0.10", // Server ID, which is an IP address in dotted decimal notation
          "devices": [ // List of devices of the instance
            {
```

```
        "device_id": "0", // Device HDC channel ID
        "device_ip": "192.168.1.8", // Actual NIC IP address of the device
    },
    {
        "pod_name": "tf-tbdf1",
        "server_id": "10.0.0.10",
        "devices": [
            {
                "device_id": "1",
                "device_ip": "192.168.1.9"
            }
        ]
    }
]
```

Table 10-10 Description of the ranktable file (template 2)

Item	Description	Required/ Optional
status	Availability flag of the rank table completed: The rank table is available and the training can be performed. initializing: The rank table is unavailable and the training cannot be performed.	Required
group_count	Number of groups that a user applies for. The recommended value is 1 .	Required
group_list	Group list	Required
group_name	Group name. When group_count is set to 1 , you are advised to Set the value to hccl_world_group or leave it empty. In the current version, a group named hccl_world_group is created regardless of the value of this parameter. If multiple groups are created by using this configuration file, the system automatically combines the groups into a group named hccl_world_group .	Optional
instance_count	The value of this parameter must be the same as the number of pod names in the instance list. For example, in the container scenario, the value is the actual number of containers.	Required
device_count	Number of devices in a group.	Required
instance_list	-	-

Item	Description	Required/Optional
pod_name	User-defined. Must be globally unique in the instance_list .	Required
server_id	Physical IP address of a server, which is a character string in dotted decimal notation, for example, 10.0.0.10 .	Required
devices	-	-
device_id	Physical ID of a device, that is, the serial number of the device on the server, corresponding to the HDC channel ID. Value range: 0–7	Required
device_ip	IP address of the on-device NIC, which is a globally unique character string in dotted decimal notation, for example, 192.168.1.8. You can run the cat /etc/hccn.conf command on the current server to obtain the IP address of the NIC. NOTE - In a typical 8-network-port communication scenario, the devices field in the ranktable file records the IP address of the ETH port available for each device. The following is the information about the device whose device_id is 0 and device_ip is 192.168.100.101. <pre>"devices": [{"device_id": "0", "device_ip": "192.168.100.101"}]</pre> - In this scenario, the information about the available one, two, or four ETH ports of each server is obtained through ranktable verification. If the device_ip field in devices is an empty string "", the ETH port of the device is not used in collective communication. For example, the following shows the "devices" information when device_id is 0 and the ETH port is not used: <pre>"devices": [{"device_id": "0", "device_ip": ""}]</pre>	Required

11 FAQ

[11.1 What Do I Do If Network Size Reaches Threshold?](#)

[11.2 What Do I Do If Training Times Out Due to Too Many Dataset Shuffle Operations?](#)

[11.3 How Do I Determine fp_point and bp_point?](#)

[11.4 What Do I Do If uint8 and quint8 Addition/Subtraction Overflows?](#)

11.1 What Do I Do If Network Size Reaches Threshold?

Symptom

When the batch size or network size is too large, a message is displayed indicating that the memory usage exceeds the threshold.

```
[ERROR] GE(179560,python3.7):2020-10-31-11:06:40.656.258 [graphengine/ge/graph/manager/  
graph_var_manager.cc:285]182539 AssignVarMem: ErrorNo: 1343225857(Parameter's invalid!) Out of  
memory : current var size[5382237696] exceeds total var size[5368709120]  
[ERROR] GE(179560,python3.7):2020-10-31-11:06:40.656.374 [graphengine/ge/graph/manager/  
graph_var_manager.cc:504]182539 AssignVarMem: ErrorNo: 1343225860(Internal errors) AssignVarMem by  
offset failed.  
[ERROR] GE(179560,python3.7):2020-10-31-11:06:40.656.420 [graphengine/ge/graph/build/memory/  
var_mem_assign_util.cc:65]182539 AssignStaticMemory2Node: ErrorNo: -1(failed)  
[ERROR] GE(179560,python3.7):2020-10-31-11:06:40.669.315 [graphengine/ge/graph/build/memory/  
memory_assigner.cc:27]182539 AssignMemory: ErrorNo: -1(failed) Memory assigner failed  
[ERROR] GE(179560,python3.7):2020-10-31-11:06:40.669.401 [graphengine/ge/graph/build/  
model_builder.cc:722]182539 BuildModelForGetTask: ErrorNo: -1(failed) Assign Memory Failed!
```

Cause Analysis

By default, the framework isolates the weight memory and feature map memory for management convenience. By default, 5 GB is allocated for weight use, and 26 GB is allocated for feature map use.

Solution

You can tune **graph_memory_max_size** and **variable_memory_max_size** to adjust the memory limits. The prerequisite is that the total memory of the weight and feature map is within 31 GB.

```
graph_mem = 1024 * 1024 * 1024 * 16
var_mem = 1024 * 1024 * 1024 * 15
run_config = NPURunConfig(hcom_parallel=False,
    enable_data_pre_proc=True,
    session_config=session_config,
    model_dir = params['log_dir'],
    iterations_per_loop=1,
    graph_memory_max_size = graph_mem,
    variable_memory_max_size = var_mem)
```

11.2 What Do I Do If Training Times Out Due to Too Many Dataset Shuffle Operations?

Symptom

An error is reported during the training process.

```
2020-11-27 11:26:00.510219: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:145] Filling up shuffle
buffer (this may take a while): 2169 of 10000
2020-11-27 11:26:10.454132: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:145] Filling up shuffle
buffer (this may take a while): 3252 of 10000
2020-11-27 11:26:20.375176: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:145] Filling up shuffle
buffer (this may take a while): 3915 of 10000
2020-11-27 11:26:30.543144: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:145] Filling up shuffle
buffer (this may take a while): 4672 of 10000
2020-11-27 11:26:40.479843: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:145] Filling up shuffle
buffer (this may take a while): 5439 of 10000
2020-11-27 11:26:50.496244: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:145] Filling up shuffle
buffer (this may take a while): 6232 of 10000
2020-11-27 11:26:50.638388: W tensorflow/core/framework/op_kernel.cc:1639] Unavailable: Internal errors
2020-11-27 11:26:53.638664: F tf_adapter/kernels/geop_npu.cc:669] GeOp33_0GEOP::DoRunAsync Failed
[ERROR] RUNTIME(62299)model execute error, retCode=0x91, [the model stream execute failed].
[ERROR] RUNTIME(62299)model execute task failed, device_id=0, model stream_id=575, model task_id=1,
model_id=522, first_task_id=65535
Fatal Python error: Aborted
```

Cause Analysis

When data preprocessing is offloaded to the NPU, preprocessing is performed in parallel with forward and backward propagation. If too many data shuffle operations are involved during preprocessing, preprocessing output will be unavailable in a long time after the forward propagation task is delivered. As a result, the forward propagation task times out.

Assume that the shuffle buffer size is set to **10000**. According to the preceding log, when the forward propagation task times out, the buffer receives only 6232 out of 10000 pieces of data. As a result, the task timeout error is printed.

Solution

You can resolve this problem in either of the following ways:

Reduce the number of shuffle operations based on the actual capacity. In this example, only 6232 shuffle operations are complete. Therefore, we can set **buffer_size** to **5000**.

```
dataset = dataset.shuffle(buffer_size=5000)
```

Disable the data preprocessing offloading switch (**enable_data_pre_proc = False**) so that preprocessing and forward and backward propagation can be executed in serial. However, the performance may be compromised.

```
run_config = NPURunConfig(enable_data_pre_proc=False)
```

11.3 How Do I Determine fp_point and bp_point?

To trace training iterations to profile a training job, you need to input the dotting operators for forward and backward propagation to obtain the forward and backward compute durations.

Obtain **fp_point** and **bp_point** by taking the following steps:

You can save the graph as a .pbtxt file by using **tf.io.write_graph** in the training script to obtain this name.

Start the search from the top. The first node (excluding the data and storage nodes) is the **fp_point** operator. Data and storage nodes can be identified by the **name** or **op** field. Generally, operators with **op** set to Const, VariableV2, IteratorV2, Identity, Reshape, or Cast, or **name** containing step, Dataset, seed, or kernel should be excluded. The following figure shows the **fp_point** operator of ResNet-50.

```
3684 node {
3685   name: "fp32_vars/conv2d/Conv2D"
3686   op: "Conv2D"
3687   input: "Cast"
3688   input: "fp32_vars/conv2d/kernel/read"
3689   device: "/device:GPU:0"
3690   attr {
3691     key: "T"
3692     value {
3693       type: DT_FLOAT
3694     }
3695   }
3696   attr {
3697     key: "_output_shapes"
3698     value {
3699       list {
3700         shape {
3701           dim {
3702             size: 32
3703           }
3704           dim {
3705             size: 112
3706           }
3707           dim {
3708             size: 112
3709           }
3710           dim {
3711             size: 64
3712           }
3713         }
3714       }
3715     }
3716   }
```

As for the **bp_point** operator, start the search from the bottom. The first node with gradient is the **bp_point** operator. The following figure shows the **bp_point** operator of ResNet-50.

```
213004 node {
213005   name: "loss_scale/gradients/AddN_70"
213006   op: "AddN"
213007   input: "loss_scale/gradients/L2Loss_grad/mul"
213008   input: "loss_scale/gradients/fp32_vars/conv2d/Conv2D_grad/tuple/control_dependency_1"
213009   device: "/device:GPU:0"
213010   attr {
213011     key: "N"
213012     value {
213013       i: 2
213014     }
213015   }
213016   attr {
213017     key: "T"
213018     value {
213019       type: DT_FLOAT
213020     }
213021   }
213022   attr {
213023     key: "_class"
213024     value {
213025       list {
213026         s: "loc:loss_scale/gradients/L2Loss_grad/mul"
213027       }
213028     }
213029   }
}
```

The operator may have been fused or renamed. To solve this problem, look up the name of the operator in the **ge_proto_xxxx_Build.txt** graph generated by GE. If an exact match is found, the operator name can be used directly. If a fuzzy match is found (generally with a **_1** suffix), the operator name in GE should be used.

11.4 What Do I Do If uint8 and quint8 Addition/Subtraction Overflows?

Symptom

The cumulative addition and subtraction result of operators of type uint8 and quint8 types overflow, which are different from the theoretical results.

For example, the theoretical compute result with the uint8 data type is **257**. However, the actual compute result on the Ascend AI Processor is **255**.

Cause Analysis

Ascend AI Processor addresses value overflows by saturating at the upper limits. Because **257** exceeds the maximum value (**255**) of an uint8, **255** is output.

Solution

You can choose to scale the value range to avoid overflows.

12 Appendixes

12.1 How Do I Install GCC 7.3.0?

12.2 Change History

12.1 How Do I Install GCC 7.3.0?

Perform the following steps as the **root** user.

Step 1 Download **gcc-7.3.0.tar.gz** from <https://mirrors.tuna.tsinghua.edu.cn/gnu/gcc/gcc-7.3.0/gcc-7.3.0.tar.gz>.

Step 2 To install GCC, you need to reserve adequate temporary space. You can run the following command to clear the **/tmp** directory in advance:

```
sudo rm -rf /tmp/*
```

Step 3 Install the dependency.

For CentOS/BCLinux, run the following command:

```
yum install bzip2
```

For Ubuntu/Debian, run the following command:

```
apt-get install bzip2
```

Step 4 Build and install GCC.

1. Go to the directory where the source code package **gcc-7.3.0.tar.gz** is located and run the following command to extract it:

```
tar -zxvf gcc-7.3.0.tar.gz
```

2. Go to the extracted directory and run the following command to download the GCC dependency packages:

```
cd gcc-7.3.0  
./contrib/download_prerequisites
```

If an error is reported during the command execution, run the following commands in the **gcc-7.3.0/** directory to download the dependency packages:

```
wget http://gcc.gnu.org/pub/gcc/infrastructure/gmp-6.1.0.tar.bz2  
wget http://gcc.gnu.org/pub/gcc/infrastructure/mpfr-3.1.4.tar.bz2  
wget http://gcc.gnu.org/pub/gcc/infrastructure/mpc-1.0.3.tar.gz  
wget http://gcc.gnu.org/pub/gcc/infrastructure/isl-0.16.1.tar.bz2
```

After downloading the preceding dependency packages, run the following command:

```
./contrib/download_prerequisites
```

If the verification fails, check whether the dependency package is repeatedly downloaded. The package should be downloaded at a time.

3. Run the following commands for configuration, build, and installation.

```
./configure --enable-languages=c,c++ --disable-multilib --with-system-zlib --prefix=/usr/local/  
linux_gcc7.3.0  
make -j15 # The value 15 indicates the number of CPUs, which is configurable and can be queried  
by running grep -w processor /proc/cpuinfo|wc -l.  
make install
```



CAUTION

The **--prefix** parameter is used to specify the `linux_gcc7.3.0` installation path, which is configurable. Do not set it to **/usr/local** or **/usr**, which is the default installation path for the GCC installed by using the software source.

Otherwise, a conflict occurs and the original GCC compilation environment of the system is damaged. In this example, the installation path is set to **/usr/local/linux_gcc7.3.0**.

Step 5 Set the environment variable.

The build environment after GCC upgrade is required for training. Therefore, you need to configure the following environment variable in the training script:

```
export LD_LIBRARY_PATH=${install_path}/lib64:${LD_LIBRARY_PATH}
```

\${install_path} indicates the GCC 7.3.0 installation path configured in 3. In this example, the GCC 7.3.0 installation path is **/usr/local/gcc7.3.0/**.

NOTE

The environment variable needs to be configured only when you need to use the build environment after the GCC upgrade.

----End

12.2 Change History

Date	Description
2021-01-15	This issue is the first official release.